COMP 576: Assignment 1 Report

In this report, I will be documenting my implementation of the assignment. I have split the documentation according to the labeled sections given in the assignment's specification.

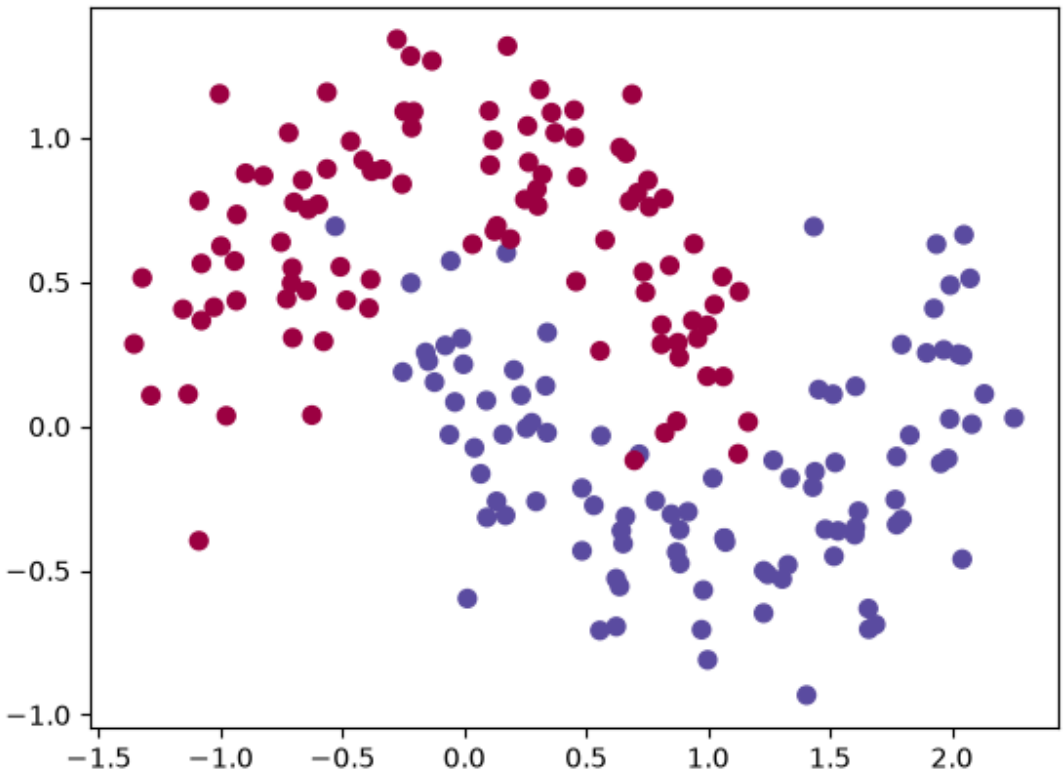
Backpropagation in a Simple Neural Network

a) Dataset

In addition to the given code to generate and visualize the Make-Moons dataset, I also added the following command:

```python
plt.savefig("make_moons.png")
```

So I could save the generated visualization to include in this report. Here is the visualizatiion that was produced:



From my understanding of the code, this dataset consists of 200 data points that have been generated with a noise value of 0.20, used to pretube the datapoints across the axis. We have two input features and two classes that we are going to be using neural network functionality to assign these datapoints to.

b) Activation Function

1. In section b) we are asked to implement the activation functions from the set: {‘Tanh’, ‘Sigmoid’, ‘ReLU’} in the function actFun(self, z, type), where z is a neuron’s input and type is the set mentioned above.

ReLu is not differentiable when z=0, so for this implementation I have resorted to returning 0 when this is the case.

I have included my actFun(self, z, type) implementation below. For safety, I added code that would bring all characters of ‘type’ to lower-case, just in-case there was a test where the input is not as expected - with that same mindset, I added a ValueError, just in case a malformed or incorrect string was inputted into the function:

```python
def actFun(self, z, type):
    '''
    actFun computes the activation functions
    :param z: net input
    :param type: Tanh, Sigmoid, or ReLU
    :return: activations
    '''
    type = type.lower()

    if type == 'tanh':
        return np.tanh(z)
    elif type == 'sigmoid':
        return 1.0 / (1.0 + np.exp(-z))
    elif type == 'relu':
        return np.maximum(0, z)
    else:
        raise ValueError("Activation function type must be 'Tanh', 'Sigmoid', or
'ReLU'")
```

I also added the following test in main(), just to make sure that the functionality is working as expected - I won't do this for every function but, since this was the first function we are implementing in this assignment, I just wanted to wrap my head around how testing could look like:

```python
# I added the following just to test the functionality of actFun(self, z, type):
model = NeuralNetwork(nn_input_dim=2, nn_hidden_dim=3, nn_output_dim=2)
z = np.array([-2.0, -0.5, 0.0, 0.5, 2.0])
for activation in ('tanh', 'sigmoid', 'relu'):
    output = model.actFun(z, type=activation)
    print(f"{activation}: {np.round(output, 4)}")
```

Which gave the following output:

```shell
tanh: [-0.964  -0.4621  0.      0.4621  0.964 ]
sigmoid: [0.1192 0.3775 0.5    0.6225 0.8808]
relu: [0.  0.  0.  0.5 2. ]
```

2. Derivations of the activation functions. The activation functions are their derivatives can be seen in the table below:

Activation Function Name	**Function: f(z)**	**Derivative: f'(z)**
Tanh	$tanh(z)$	$1 - tanh^2(z)$
Sigmoid	$\frac{1}{1+e^{-z}}$	$f(z)(1 - f(z))$
ReLU	$max(0, z)$	$0\ if\ z < 0,\ and\ 1\ if\ z > 0$

For tanh:

$tanh(z) = \frac{(e^z - e^{-z})}{(e^z + e^{-z})}$, we would need to apply the quotient rule to derive tanh:

$$\frac{d}{dz} tanh(z) = \frac{(e^z + e^{-z})^2 - (e^z - e^{-z})^2}{(e^z + e^{-z})^2} = 1 - tanh^2(z)$$

For sigmoid:

$f(z) = (1 + e^{-z})^{-1}$, we would need to apply the chain rule to derive sigmoid:

$f'(z) = \frac{e^{-z}}{(1+e^{-z})^2} = \frac{1}{1+e^{-z}}(1 - \frac{1}{1+e^{-z}}) = f(z)(1 - f(z))$

For ReLU:

ReLU is $f(z) = 0$ when $z < 0$ and $f(z) = z$ when $z > 0$, so its derivative is 0 and 1 respectively. The derivative is undefined when $z = 0$.

3. I have implemented the diff_actFun(z, type) functionality below. It has followed a very similar format to the actFun(z, type) functionality. For Sigmoid, you can see the use of the actFun(z, type) functionality:

```python
def diff_actFun(self, z, type):
    '''
    diff_actFun compute the derivatives of the activation functions wrt the net input
    :param z: net input
    :param type: Tanh, Sigmoid, or ReLU
    :return: the derivatives of the activation functions wrt the net input
    '''
    type = type.lower()

    if type == 'tanh':
        return 1.0 - np.tanh(z) ** 2
    elif type == 'sigmoid':
        activation = self.actFun(z, type="sigmoid")
        return activation * (1.0 - activation)
    elif type == 'relu':
        return (np.asarray(z) > 0).astype(float)
    else:
        raise ValueError("Activation function type must be 'Tanh', 'Sigmoid', or 'ReLU'")
```

c) Build the Neural Network

The given constructor for the Neural Network already creates the weights and biases for the network - can be seen by the variables W1, W2, b1, and b2. We can make use of these calculated weights and biases in the feedforward(X, actFun) functionality. This forward pass function is required to not only create the 3-layered network, but to also transform the input coordinates into class probabilities for the classes ‘male’ and ‘female’.

Since we are making use of the NumPy library, for the matrix multiplication that is required in this stage of neural network creation, I decided to go with ‘@’ syntax, which performs matrix multiplication between two variables.

1. feedforward(self, X, actFun)

The input for layer 1 (z1) is described as:

$$z_1 = W_1 x + b_1$$

This is where each neuron essentially computes a weighted combination of the two input values, and then adds its bias.Which can be calculated by doing:

```python
self.z1 = X @ self.W1 + self.b1
```

The output for layer 1 (a1) is described as:

$$a_1 = actFun(z_1)$$

Which can be calculated by doing:

```python
self.a1 = actFun(self.z1)
```

The input for layer 2 (z2) is described as:

$$z_2 = W_2 a_1 + b_2$$

This input layer produces two scores per input value, for each class. Which can be calculated by doing:

```python
self.z2 = self.a1 @ self.W2 + self.b2
```

And finally, the output for layer 2 (a2, or also described as self.prob in the assignment specification and given code) is described as:

$$a2 = \hat{y} = softmax(z_2)$$

From my understanding, softmax() is a function that is applied to input values, which converts these values into a value in the range of 0 to 1, where the sum of all of the input values is equal to 1. This means that softmax() essentially converts the input values into probabilities. The function of softmax() itself, can be described as the following:

$$\frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$$

Here all we are doing is exponentiate each score, and then divide it by the sum of all exponentiated scores. Which can be calculated by doing:

```python
shifted_scores = self.z2 - np.max(self.z2, axis=1, keepdims=True)
exp_scores = np.exp(shifted_scores)
self.probs = exp_scores / np.sum(exp_scores, axis=1, keepdims=True)
```

With the help of softmax, we are able to calculate the probability of the neural networks input belonging to either of the two classes.

Just to make sure I was properly understanding the network construction up to this point, I decided to create a small test in main:

```python
# I added the following just to test the functionality of feedforward(self, X, actFun):
    # Here you can see I copied the network-layer dimensionality that is displayed in Figure 1 on page 3.
    model = NeuralNetwork(nn_input_dim=2, nn_hidden_dim=3, nn_output_dim=2)
    # Here I selected a point's two coordinates from the Make Moons dataset,as well as calling actFun for z.
    model.feedforward(X[:1], lambda z: model.actFun(z, type=model.actFun_type))
    # Below are some outputs to validate that the functionality is working as expected.
    print("Input point:", X[0])
    print("Class probabilities:", np.round(model.probs[0], 4))
    print("Probability sum:", model.probs[0].sum())
```

2. calculate_loss(self, X, y)

Given that we have N training examples, and C classes, then the loss function for the prediction ŷ with respect to the original labels y is:

$$L(y, \hat{y}) = -\frac{1}{N}\sum_{n\epsilon N}\sum_{i\epsilon C} y_{n,i} log(\hat{y}_{n,i})$$

Here we are using one-hot-encoding vectors, where y is the one-hot-encoding vector and ŷ is the vector of probability. $y_{n,i}$ is 1 for the correct class, and 0 for the incorrect class.

This can be calculated by the following calculate_loss(self, X, y) implementation:

```python
def calculate_loss(self, X, y):
    '''
    calculate_loss compute the loss for prediction
    :param X: input data
    :param y: given labels
    :return: the loss for prediction
    '''
    num_examples = len(X)
    self.feedforward(X, lambda x: self.actFun(x, type=self.actFun_type))
    # Calculating the loss
    shifted_scores = self.z2 - np.max(self.z2, axis=1, keepdims=True)
    log_probs = shifted_scores - np.log(np.sum(np.exp(shifted_scores), axis=1,
keepdims=True))
    data_loss = -np.sum(log_probs[np.arange(num_examples), y])

    # Add regulatization term to loss (optional)
    data_loss += self.reg_lambda / 2 * (np.sum(np.square(self.W1)) +
np.sum(np.square(self.W2)))
    return (1. / num_examples) * data_loss
```

The above function implementation works from the values of z2, before the softmax implementation of adjusting the values into probabilities.

‘num_examples’: This was given. This finds N, which is the number of examples, used in the loss function: $\frac{1}{N}$.

‘self.feedforward(X, lambda x: self.actFun(x, type=self.actFun_type))’: This was given. This will calculate ŷ, which is stored in self.probs, as well as storing the output of z2 in self.z2.

‘shifted_scores’: Shifted scores is used to subtract each row’s maximum for numerical stability.

‘log_probs’: This is used to calculate;$log(\hat{y}_{n,i})$, the log of the predicted probability for class i.

‘log_probs[np.arange(num_examples), y]’: This is used for selecting the correct class for each example. Essentially having the same effect as multiplying by one-hoten-encoded value $y_{n,i}$ and summing this over classes i.

‘data_loss’: The functionality here is to negate the value of log_probs. I then sum up all of the example losses that are associated with the correct class. The weight penalty is then added to data_loss (given in code). The function then returns 1 being divided by the sum of losses into their average.

‘return (1. / num_examples) * data_loss’: Applies $\frac{1}{N}$ to the loss function.

d) Backward Pass - Backpropagation

1. Derivation of the back propagation gradients

Let x be a row-vector and y be its one-hot label:

$$z_1 = x\ W_1 + b_1, \qquad a_1 = f(z_1), \qquad z_2 = a_1 W_2 + b_2, \qquad p_n = softmax(z_2).$$

The cross-entropy loss for this example is $l = -\sum_i y_i log p_i$. Cross-entropy loss is essentially a rule that measures the difference between the predicted probability distribution of a model, and the true distribution of class labels in the model. The paths that need to be differentiated are:

$$W_2 \text{ or } b_2 \rightarrow z_2 \rightarrow loss, \text{ and } W_1 \text{ or } b_1 \rightarrow z_1 \rightarrow a_1 \rightarrow z_2 \rightarrow loss$$

Starting with the output layer, since $p_i = \frac{e^{z_{2,i}}}{\sum_r e^{z_{2,r}}}$, substituting $softmax$ into the loss will give us:

$$l = -\sum_i y_i z_{2,i} + log(\sum_r e^{z_{2,i}}),$$

Where we used $\sum_i y_i = 1$.

Differentiating with respect to $z_{2,i}$;

$$\delta_{2,i} := \frac{\partial l}{\partial z_{2,i}} = -y_i + \frac{e^{z_{2,i}}}{\sum_r e^{z_{2,r}}} = p_i - y_i$$

Where $\delta_{2,i}$ is the error passed back from the output layer. We now work back to W_2 and b_2,

where the output score is: $z_{2,i} = \sum_j a_{1,j} W_{2,ji} + b_{2,i}$:

$$\frac{\partial l}{\partial W_{2,ji}} = \frac{\partial l}{\partial z_{2,i}} \frac{\partial z_{2,i}}{\partial W_{2,ji}} = \delta_{2,i} a_{1,j}, \qquad \frac{\partial l}{\partial b_{2,i}} = \delta_{2,i}$$

Now we can pass the error through the 'hidden layer'. The hidden layer activation is connected to every output score, we therefore need to add the contribution from each output path:

$$\frac{\partial l}{\partial a_{1,j}} = \sum_i \frac{\partial l}{\partial z_{2,ii}} \frac{\partial z_{2,i}}{\partial a_{1,j}} = \sum_i \delta_{2,i} W_{2,ji}$$

Since $a_{1,j} = f(z_{1,j})$, the chain-rule gives the hidden layer error:

$$\delta_{1,j} := \frac{\partial l}{\partial z_{1,j}} = (\sum_i \delta_{2,i} W_{2,ji}) f'(z_{1,j})$$

We can now work back to W_1 and b_1, where the output score is: $z_{1,j} = \sum_k x_k W_{1,kj} + b_{1,j}$:

$$\frac{\partial l}{\partial W_{1,kj}} = \frac{\partial l}{\partial z_{1,j}} \frac{\partial z_{1,j}}{\partial W_{1,kj}} = \delta_{1,j} x_k,$$

$$\frac{\partial l}{\partial b_{1,j}} = \frac{\partial l}{\partial z_{1,j}} \frac{\partial z_{1,j}}{\partial b_{1,j}} = \delta_{1,j}$$

The equations that define the network are averages over N, example: $L = \frac{1}{N} \sum_{n=1}^{N} l_n$. With examples stored as rows, allow us to define:

$D_2 = \frac{P-Y}{N}$, $\quad D_1 = (D_2 W_2^T) \odot f'(Z_1)$, where $\odot$ = element-wise multiplication.

<u>The four gradients are therefore:</u>

$$\frac{\partial L}{\partial W_2} = A_1^T D_2, \qquad \frac{\partial L}{\partial b_2} = \sum_n D_{2,n},$$

$$\frac{\partial L}{\partial W_1} = X^T D_1, \qquad \frac{\partial L}{\partial b_1} = \sum_n D_{1,n}$$

2. Backpropagation implementation

I have included comments for both implementation and test for the backprop(self, X, y) function:
Implementation:

```python
def backprop(self, X, y):
    '''
    backprop run backpropagation to compute the gradients used to update the parameters in the backward step
    :param X: input data
    :param y: given labels
    :return: dL/dW1, dL/b1, dL/dW2, dL/db2
    '''
    # For backprop, we are making use of the results from feedforward - as shwon in the fit_model function.
    # Here, we are essentially just supplying the gradeints that fit_model uses.
    num_examples = len(X)
    delta2 = self.probs.copy()
    # We do -= 1 here (making it (P-Y)/N) so we don't have to create a separate one-hot label matrix.
    delta2[np.arange(num_examples), y] -= 1
    delta2 /= num_examples

    dW2 = self.a1.T @ delta2
    db2 = np.sum(delta2, axis=0, keepdims=True)

    delta1 = (delta2 @ self.W2.T) * self.diff_actFun(self.z1, self.actFun_type)
    dW1 = X.T @ delta1
    db1 = np.sum(delta1, axis=0, keepdims=True)

    return dW1, dW2, db1, db2
```

Test:

```python
# I added the following to test out the functionality for backprop
# We make use of a single Make Moons point and its label, run feedforward ffirst, and then print out
# the gradients and check each has the same shap as its parameter.
model = NeuralNetwork(nn_input_dim=2, nn_hidden_dim=3, nn_output_dim=2)
model.feedforward(X[:1], lambda z: model.actFun(z, type=model.actFun_type))
dW1, dW2, db1, db2 = model.backprop(X[:1], y[:1])
print("Correct class:", y[0])
for name, gradient, parameter in (
    ("dW1", dW1, model.W1), ("dW2", dW2, model.W2),
    ("db1", db1, model.b1), ("db2", db2, model.b2),
):
    assert gradient.shape == parameter.shape
    print(f"{name} ({gradient.shape}):\n{np.round(gradient, 4)}")
```

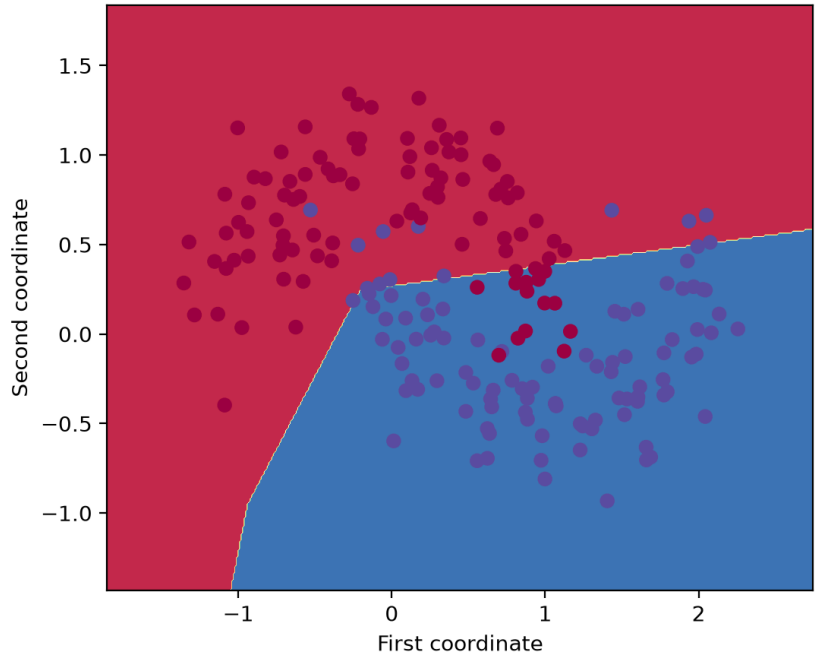
Regarding the code given to us in main() for training, I added a few variables so that we can describe and analyse the results a lot better. Here is the updated code in main():

```python
for activation in ("tanh", "sigmoid", "relu"):
    model = NeuralNetwork(2, 3, 2, actFun_type=activation, seed=0)
    initial_loss = model.calculate_loss(X, y)
    model.fit_model(X, y, print_loss=False)
    final_loss = model.calculate_loss(X, y)
    accuracy = np.mean(model.predict(X) == y)
    model.visualize_decision_boundary(X, y)
    print(
        f"{activation}: initial loss={initial_loss:.4f}, "
        f"final loss={final_loss:.4f}, training accuracy={accuracy:.1%}, "
        f"figure={figure_path}"
    )
```

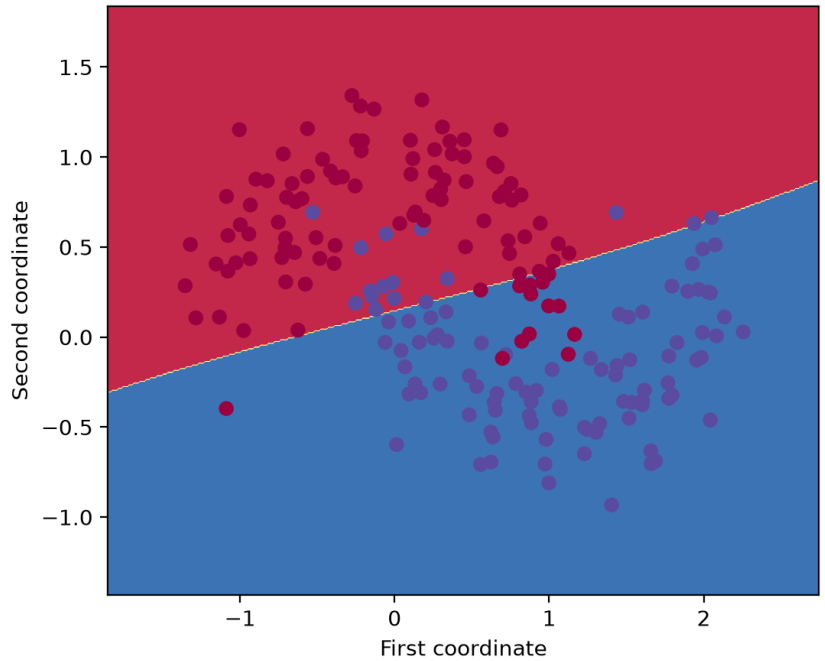
e) Time to Have Fun - Training!

1. Training the Activation Functions

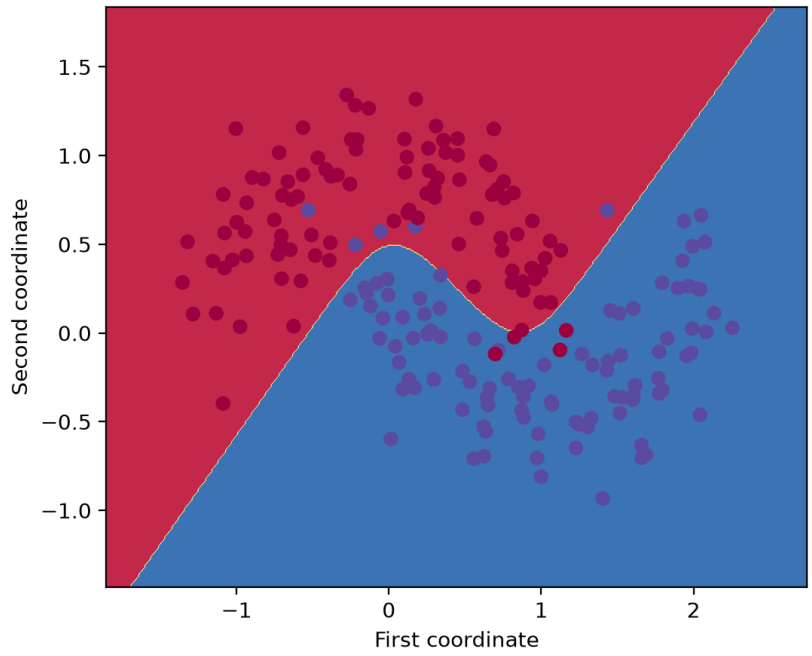
ReLU Results:



Sigmoid Results:



Tanh Results:



I also constructed the following table of results, for a more clear visualization:

Activation	**Final Loss**	**Accuracy***
ReLU	0.2724	86.0%
Sigmoid	0.3055	85.0%
Tanh	0.1486	95.5%

*Accuracy is based on the 200 training points in Make Moons.

Based on the above results, we can see that Tanh produced the most curved decision boundary, which resulted in the highest accuracy for the Make Moons dataset (95.5%), compared with Sigmoid (85.0%) and ReLU (86.0%). The shape of the class-distribution within the Make Moons dataset allowed Tanh to produce the highest results due to the nature of its activation function.

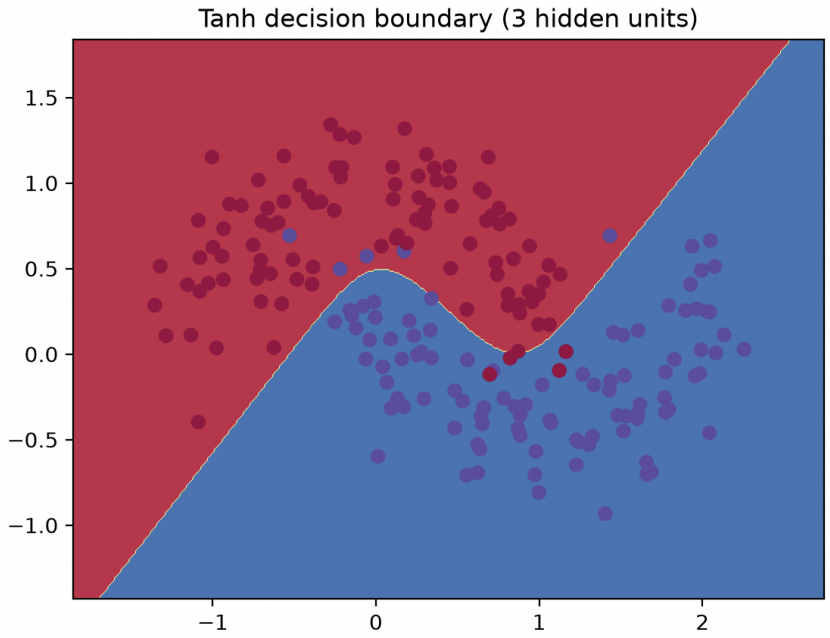
2. Training Tanh with Increased Hidden Units

For training Tanh, I decided to make use of the following hidden unit values: {3, 6, 12}. With that being said, I added code to the main() function, in order to produce the results of the training. Here is the code I made use of:

```python
# The below section is to compare hidden-layer widths using Tanh.
# I made use of the following hidden dimension values: {3, 6, 12}
for hidden_dim in (3, 6, 12):
    model = NeuralNetwork(2, hidden_dim, 2, actFun_type="tanh", seed=0)
    initial_loss = model.calculate_loss(X, y)
    model.fit_model(X, y, print_loss=False)
    final_loss = model.calculate_loss(X, y)
    accuracy = np.mean(model.predict(X) == y)
    print(
        f"Tanh, {hidden_dim} hidden units: "
        f"initial loss={initial_loss:.4f}, "
        f"final loss={final_loss:.4f}, "
        f"training accuracy={accuracy:.1%}",
        flush=True,
    )
    plt.figure()
    plt.title(f"Tanh decision boundary ({hidden_dim} hidden units)")
    model.visualize_decision_boundary(X, y)
    plt.close()
```

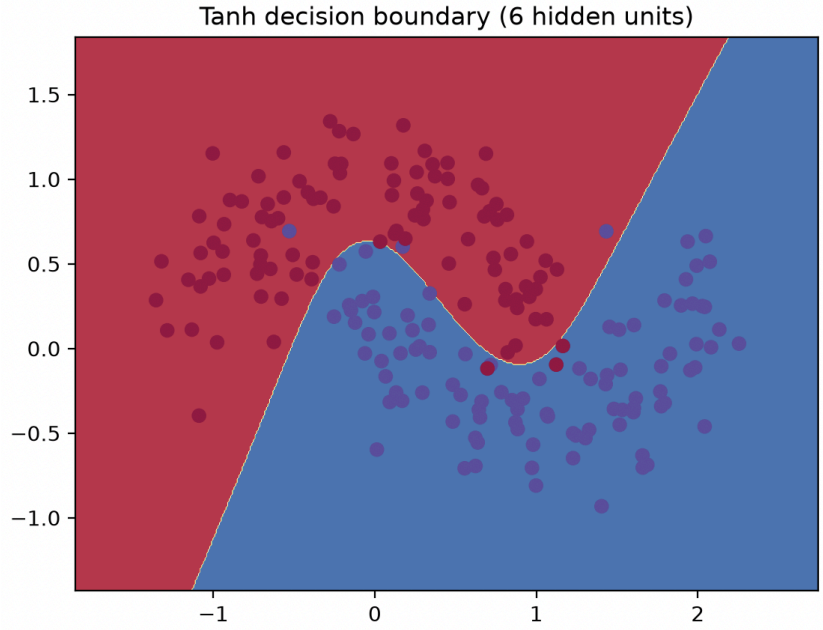
Results are on the following page.

Based on the above code, I was able to obtain the following results:
Hidden Unit = 3:



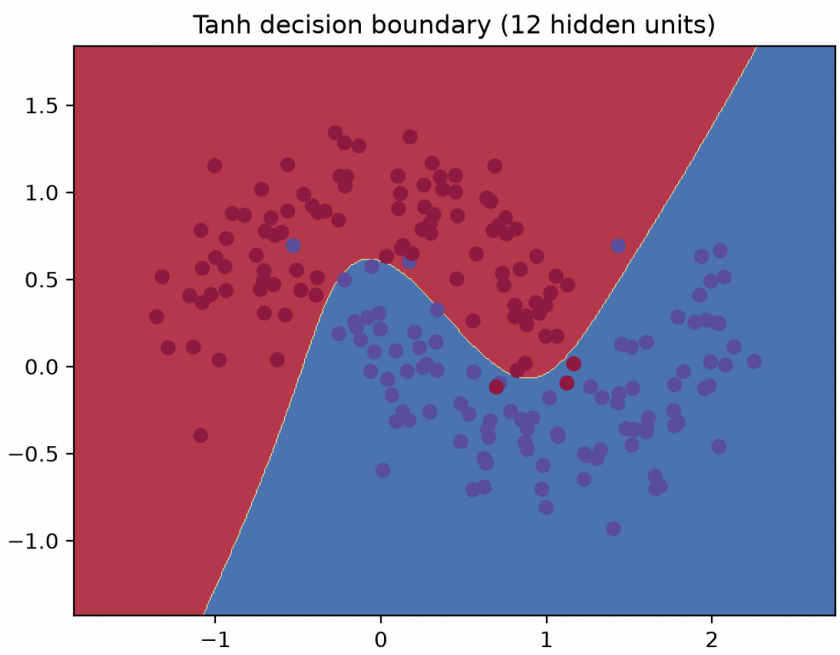
Results: Initial Loss=0.5839, Final Loss=0.1486, Training Accuracy=95.5%

Hidden Unit = 6:



Results: Initial Loss=0.5459, Final Loss=0.1000, Training Accuracy=97.0%

Hidden Unit = 12:



Results: Initial Loss=0.7687, Final Loss=0.1053, Training Accuracy=97.0%

The results of the three respective hidden unit values for the Tanh activation function show that increasing the hidden layer from 3 to 6 units raised training accuracy from 95.5% to 97.0% and reduced loss from 0.1486 to 0.1000. More hidden units give the network more flexibility to form a curved boundary around the two moons, which may explain the initial improvement.
Increasing the width to 12 units did not improve training accuracy, suggesting that 6 units were already sufficient to classify most of these training points under our settings. The 12-unit loss was slightly higher (0.1053), so additional units did not guarantee a better result.

f) Even More Fun - Training a Deeper Network!!!

1. DeepNeuralNetwork

I implemented DeepNeuralNetwork as a subclass of NeuralNetwork. This allows the deeper network to reuse the existing activation functions and their derivatives. I first started with the constructor, where it can be seen to accept the number of layers, the size of each of these layers as a list, the activation type, the regularization strength, as well as random seed - very similar to the constructor from NeuralNetwork. The constructor checks if length of layer_sizes is equal num_layers, then creates a weight matrix, as well as a bias vector for each connection between the layers:

```python
def __init__(self, num_layers, layer_sizes, actFun_type='tanh', reg_lambda=0.01,
seed=0):
    if num_layers != len(layer_sizes):
        raise ValueError(
            f"num_layers != len(layer_sizes). Invalid construction."
        )

    self.num_layers = num_layers
    self.layer_sizes = layer_sizes
    self.actFun_type = actFun_type
    self.reg_lambda = reg_lambda

    self.weights = []
    self.biases = []

    np.random.seed(seed)

    for i in range(len(layer_sizes) - 1):
        input_size = layer_sizes[i]
        output_size = layer_sizes[i + 1]

        W = np.random.randn(input_size, output_size)
        b = np.zeros((1, output_size))

        self.weights.append(W)
        self.biases.append(b)
```

2. Functions adapted to DeepNeuralNetwork

I created new implementations of feedforward, backprop, calculate_loss, and fit_model since the methods in NeuralNetwork only function for a fixed pair of weight matrices, whereas we need to allow these methods to function for N number of layers, therefore a non-fixed number of weight matrices. The DeepNeuralNetwork implementation of these methods iterates over lists of weights and biases, meaning that it works for different values and sizes of layers. I also re-implemented predict to work respectively with the new feedforward.

Python

```python
def feedforward(self, X):
    self.z_values = []
    self.activations = [X]

    for i in range(len(self.weights)):
        z = self.activations[i] @ self.weights[i] + self.biases[i]
        self.z_values.append(z)

        if i == len(self.weights) - 1:
            shifted_scores = z - np.max(z, axis=1, keepdims=True)
            exp_scores = np.exp(shifted_scores)
            self.probs = exp_scores / np.sum(
                exp_scores, axis=1, keepdims=True
            )
            self.activations.append(self.probs)
        else:
            a = self.actFun(z, type=self.actFun_type)
            self.activations.append(a)

    return None

def predict(self, X):
    self.feedforward(X)
    return np.argmax(self.probs, axis=1)
```

Python

```python
def backprop(self, X, y):
    dW = [None] * len(self.weights)
    db = [None] * len(self.biases)

    num_examples = len(X)
    delta = self.probs.copy()
    delta[np.arange(num_examples), y] -= 1
    delta /= num_examples

    for i in range(len(self.weights) -1, -1, -1):
        dW[i] = self.activations[i].T @ delta
        db[i] = np.sum(delta, axis=0, keepdims=True)

        if i > 0:
            delta = (delta @ self.weights[i].T) * self.diff_actFun(
                self.z_values[i-1], type=self.actFun_type
            )

    return dW, db

def calculate_loss(self, X, y):
    num_examples = len(X)
    self.feedforward(X)

    scores = self.z_values[-1]
    shifted_scores = scores - np.max(
        scores, axis=1, keepdims=True
    )

    log_probs = shifted_scores - np.log(
        np.sum(np.exp(shifted_scores), axis=1, keepdims=True)
    )
    data_loss = -np.sum(log_probs[np.arange(num_examples), y])
    weight_penalty = (self.reg_lambda / 2) * sum(
        np.sum(W ** 2) for W in self.weights
    )

    return (data_loss + weight_penalty) / num_examples
```

```python
def fit_model(self, X, y, epsilon=0.01, num_passes=20000, print_loss=True):
    for i in range(0, num_passes):
        self.feedforward(X)
        dW, db = self.backprop(X, y)

        for j in range(len(self.weights)):
            dW[j] += (self.reg_lambda / len(X)) * self.weights[j]
            self.weights[j] += -epsilon * dW[j]
            self.biases[j] += -epsilon * db[j]

        if print_loss and i % 1000 == 0:
            print("Loss after iteration %i: %f" % (i, self.calculate_loss(X, y)))
```

3. Choice of Layer Representation

The assignment suggested creating a separate Layer class, however I instead represented the layers with ordered lists of weight matrices and bias vectors. For index i, weights[i] and biases[i] describe a single connection between consecutive layers. This keeps the parameters and the calculations for all of the layers in a single class, while still allowing a variable network depth.

4. Forward Propagation

The forward pass in DeepNeuralNetwork starts with the input data, $A_0 = X$, and processes the weight matrices in order. For each connection, it calculates:

$$Z_{i+1} = A_i W_{i+1} + b_{i+1}$$

Within the hidden layers, forward propagation apples the selected activation function $A_{i+1} = f(Z_{i+})$. At the final layer, it applies softmax to obtain the class possibilities. The method stores each Z and activation because they are needed during backpropagation. It subtracts the largest output score before exponentiation to make the softmax calculation numerically more stable.

5. Backpropagation

The backpropagation function in DeepNeuralNetwork begins with the output error:

$$D_{out} = \frac{(P-Y)}{N}$$

Where P contains the predicted probabilities, Y contains the labels in one-hot form, and N is the number of examples. The method visits the layers in reverse order. At each layer it computes:

$$\frac{\partial L}{\partial W_{i+1}} = A_i^T D_{i+1}, \qquad \frac{\partial L}{\partial b_{i+1}} = \sum_{examples} D_{i+1}$$

For each preceding hidden layer, it propagates the error backward through the current weights and multiplies by the derivative of the hidden activation function:

$$D_i = (D_{i+1} W_{i+1}^T) \odot f'(Z_i)$$

6. Loss, Regularization, and Training

The loss combines average softmax cross-entropy with an L2 penalty on every weight matrix:

$$L = -\frac{1}{N} \sum_{n=1}^{N} logP_{n,y_n} + \frac{\lambda}{2N} \sum_i ||W_i||_F^2$$

The fit_model method runs a forward pass and backpropagation on the training data, adds $(\lambda/N)W_i$ to each weight gradient, and then updates every weight matrix and bias using gradient descent. Biases receive no L2 penalty. The method can print the loss periodically to monitor training.

Training DeepNeuralNetwork on Make_Moons

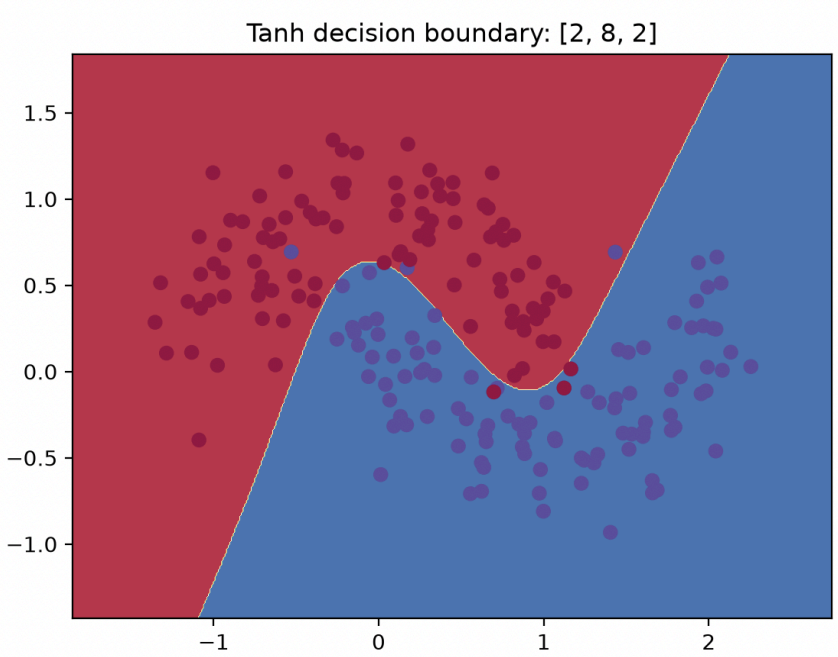
“Be curious and creative!!!” inspired me to run four separate training experiments using n_layer_neural_network. I decided to experiment with the depth, width, mixed widths, as well as different activation functions for specific architectures. The results can be seen below.

Depth Experiment

For the depth experiment, I made use of the following architectures, with the Tanh activation function:

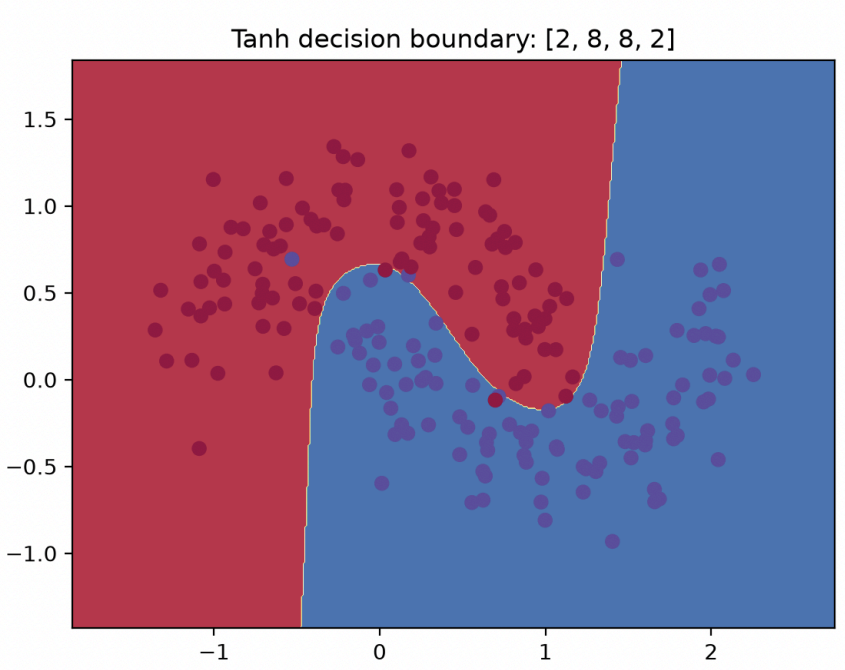
```python
architectures = [
    [2, 8, 2],
    [2, 8, 8, 2],
    [2, 8, 8, 8, 2],
    [2, 8, 8, 8, 8, 2],
]
```

Depth of 3



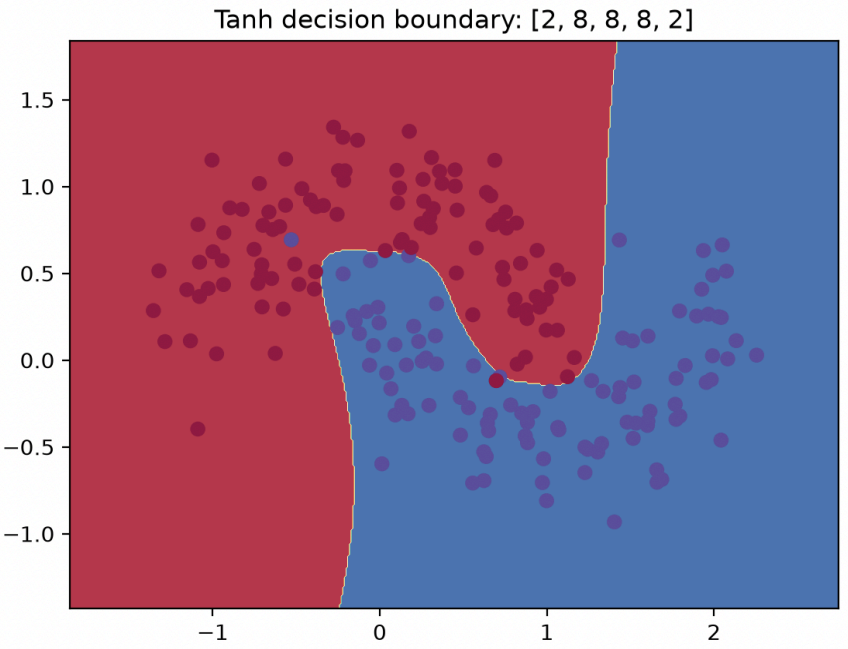
3 Layers [2, 8, 2]: Initial Loss=2.4012, Final Loss=0.0962, Training Accuracy=97.0%

Depth of 4



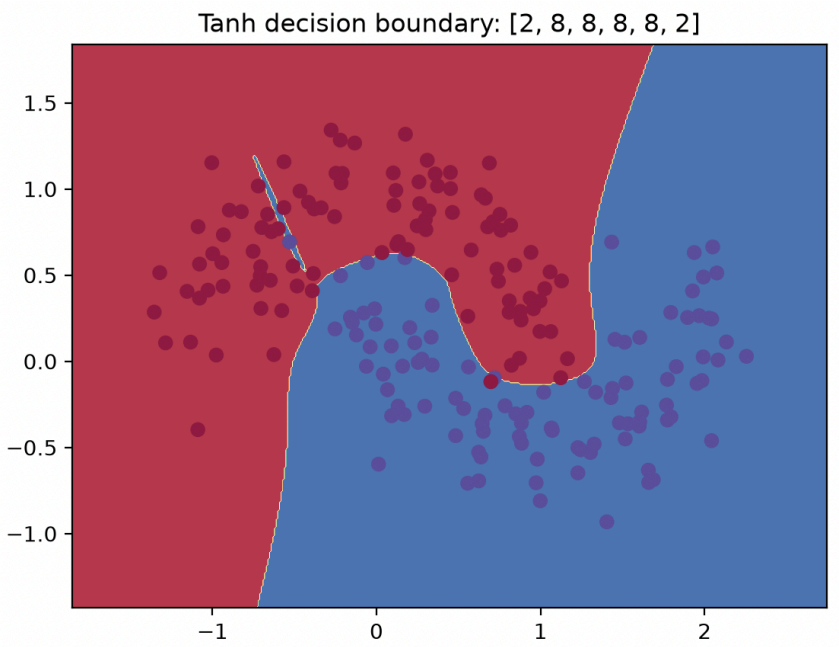
4 Layers [2, 8, 8, 2]: Initial Loss=1.0920, Final Loss=0.0644, Training Accuracy=98.0%

Depth of 5



5 Layers [2, 8, 8, 8, 2]: Initial Loss=3.7995, Final Loss=0.0446, Training Accuracy=99.0%

Depth of 6



6 Layers [2, 8, 8, 8, 8, 2]: Initial Loss=0.7145, Final Loss=0.0257, Training Accuracy=99.5%

Observations & Results:

No. Layers	Initial Loss	Final Loss	Training Accuracy
3	2.4012	0.0962	97.0%
4	1.0920	0.0644	98.0%
5	3.7995	0.0446	99.0%
6	0.7145	0.0257	99.5%

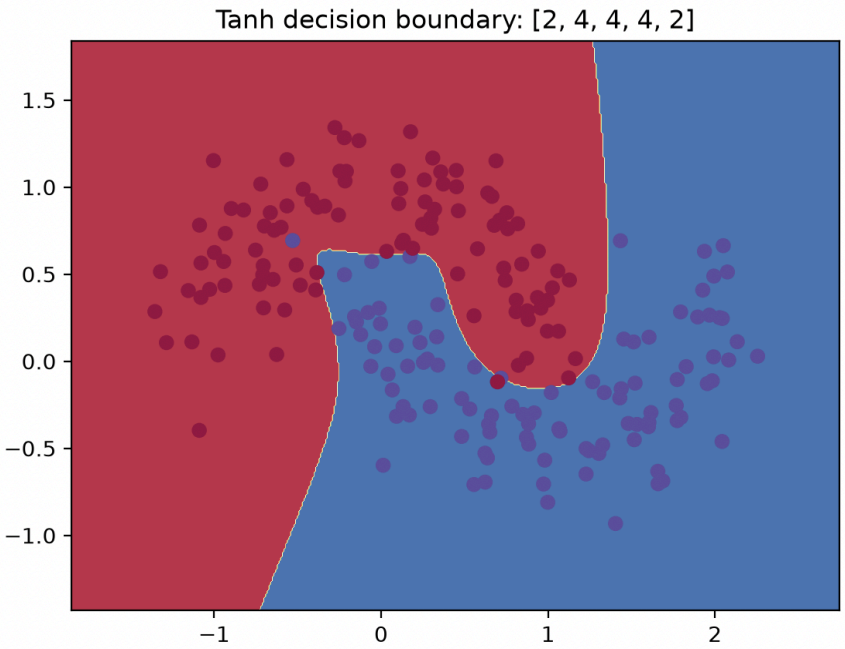
Increasing the network from 3 to 6 total layers raised the training accuracy from 97.0% to 99.5%, as well as reducing the final loss from 0.0962 to 0.0257. It can be seen that all 4 decision boundaries managed to separate the two moons, however the deeper networks followed the moon's curved shape more closely. The 6-layered network can be seen to include a narrow blue-zone extension into the red-zone, as an attempt to maximize point coverage. There is no observable pattern for the initial losses produced by these networks, so the clearest trend is the trained results. These findings have shown us that with depth, comes improved fitting of classes.

Width Experiment

For the width experiment, I made use of the following architectures, with the Tanh activation function:

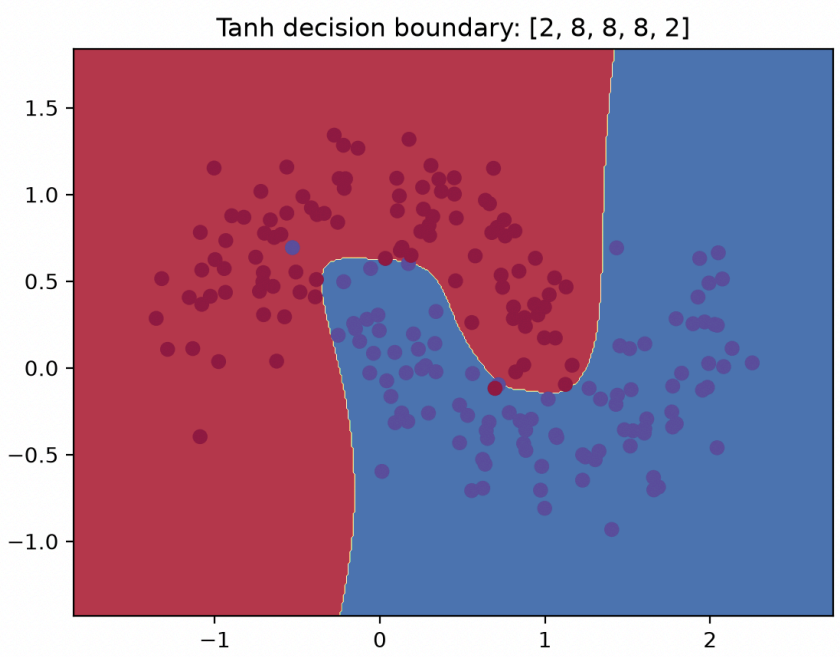
```python
architectures = [
    [2, 4, 4, 4, 2],
    [2, 8, 8, 8, 2],
    [2, 16, 16, 16, 2],
    [2, 32, 32, 32, 2],
]
```

Width of 4



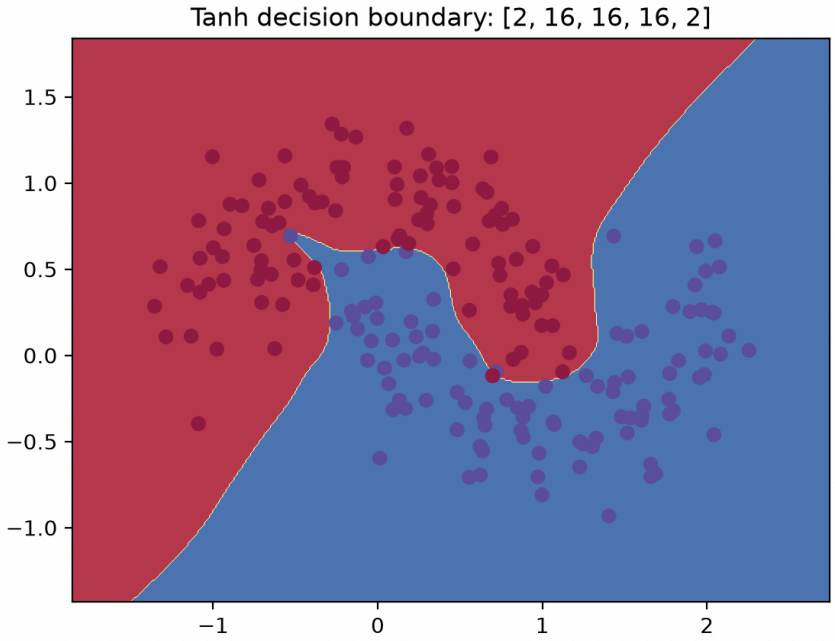
5 Layers [2, 4, 4, 4, 2]: Initial Loss=0.9281, Final Loss=0.0385, Training Accuracy=98.5%

Width of 8



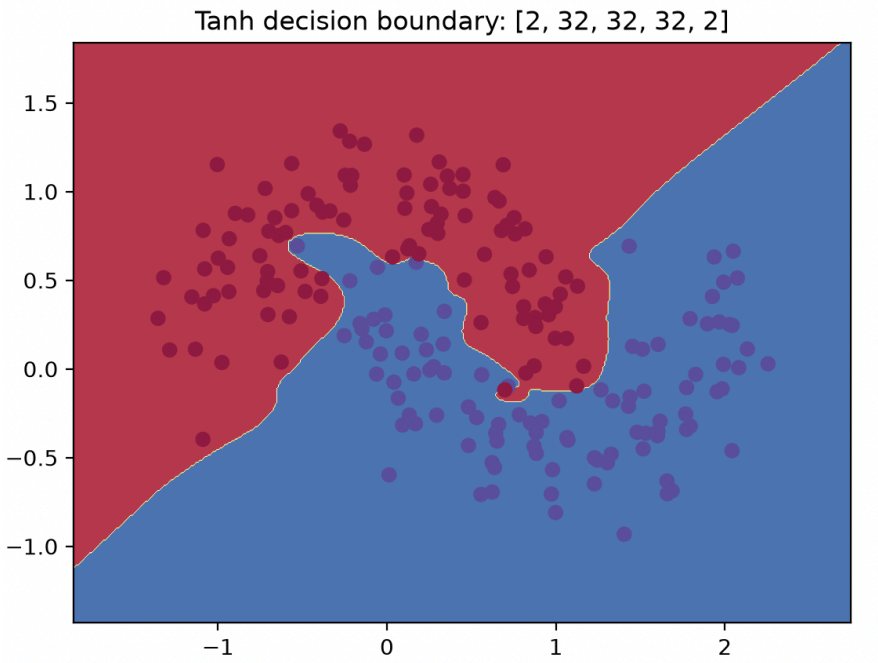
5 Layers [2, 8, 8, 8, 2]: Initial Loss=3.7995, Final Loss=0.0446, Training Accuracy=99.0%

Width of 16



5 Layers [2, 16, 16, 16, 2]: Initial Loss=1.5539, Final Loss=0.0361, Training Accuracy=99.0%

Width of 32



5 Layers [2, 32, 32, 32, 2]: Initial Loss=2.4502, Final Loss=0.0614, Training Accuracy=100.0%

Observations & Results

Hidden Layer Width	Initial Loss	FInal Loss	Training Accuracy
4	0.9281	0.0385	98.5%
8	3.7995	0.0446	99.0%
16	1.5539	0.0361	99.0%
32	2.4502	0.0614	100.0%

Increasing the width from 4 to 32 raised training accuracy from 98.5% to 100%, but final loss evidently did not decrease consistently at all: the 32-width network ended up having the highest final loss despite classifying every training point correctly. The boundaries of the 32-depth network also bend more sharply around the individual points than the narrower models' boundaries. This nature suggests that extra width gives the network more flexibility to fit the training data, but higher training accuracy does not necessarily mean lower loss or better performance on unseen points - this could be due to the flexibility of the class definition, it is exposed to more loss when determining the classes.

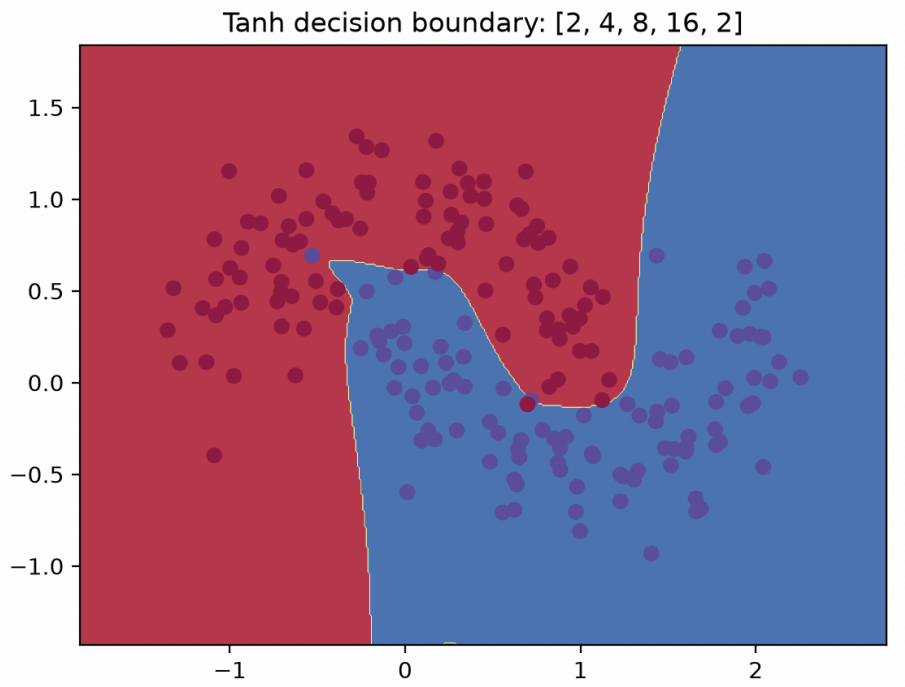
Mixed-Width Experiment

For the mixed-width experiment, I made use of the following architectures, with the Tanh activation function:

```python
architectures = [
    [2, 4, 8, 16, 2],
    [2, 16, 8, 4, 2],
    [2, 16, 4, 16, 2],
    [2, 8, 8, 8, 2],
]
```

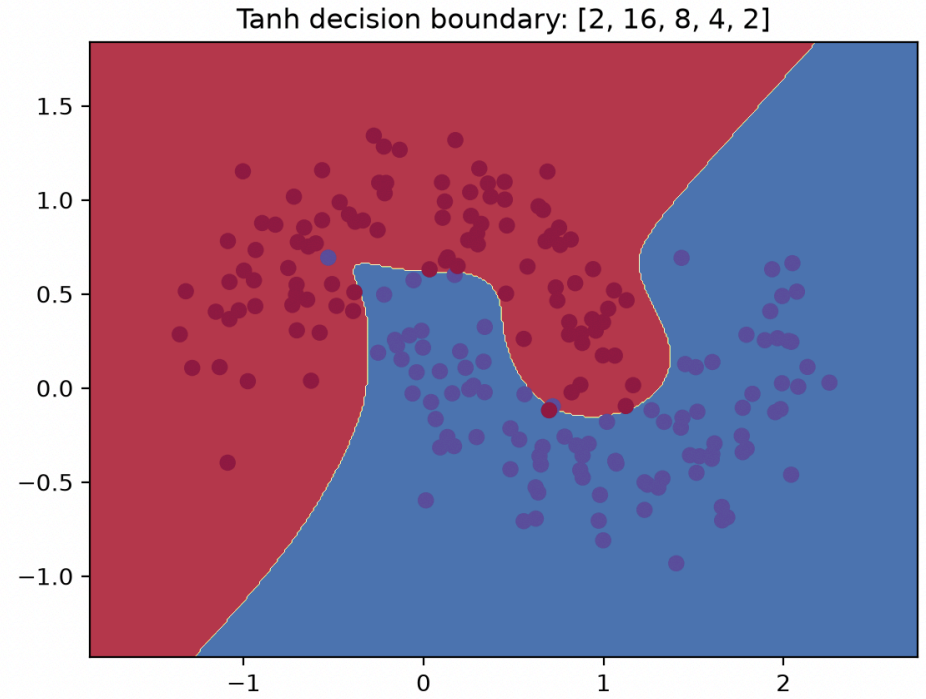
Above you can see I made use of the following architecture: [2, 8, 8, 8, 2]. This is so that we can compare our mixed-width results with a network of fixed hidden-layer width in our observations and results section.

[2, 4, 8, 16, 2]



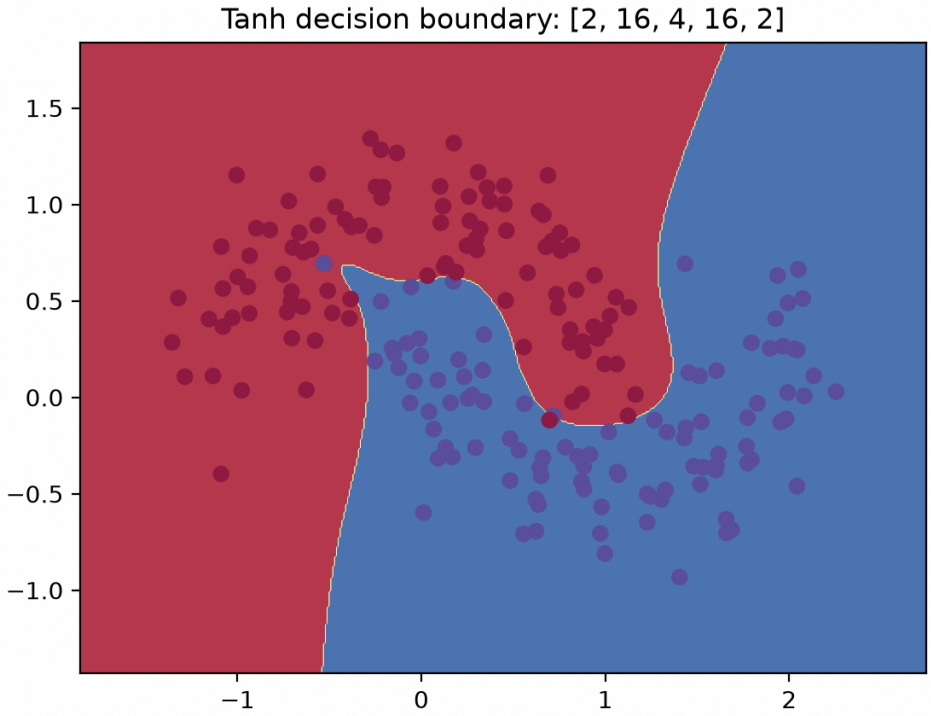
5 Layers [2, 4, 8, 16, 2]: Initial Loss=2.5058, Final Loss=0.0367, Training Accuracy=98.5%

[2, 16, 8, 4, 2]



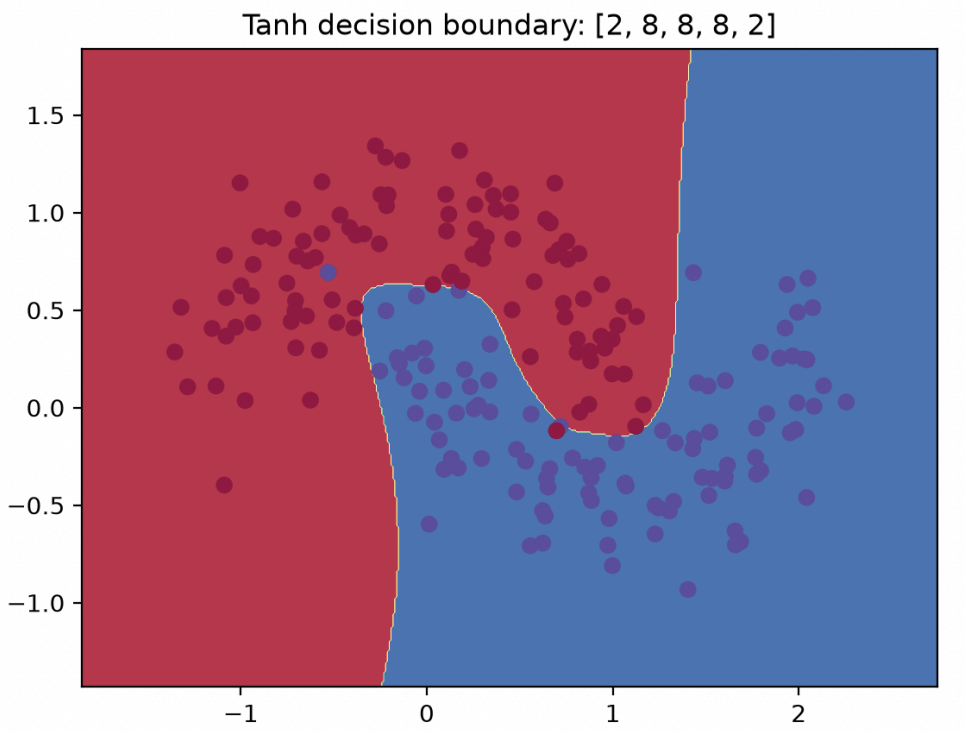
5 Layers [2, 16, 8, 4, 2]: Initial Loss=0.5168, Final Loss=0.0381, Training Accuracy=98.5%

[2, 16, 4, 16, 2]



5 Layers [2, 16, 4, 16, 2]: Initial Loss=4.1423, Final Loss=0.0327, Training Accuracy=98.5%

```
[2, 8, 8, 8, 2]
```



5 Layers [2, 8, 8, 8, 2]: Initial Loss=3.7995, Final Loss=0.0446, Training Accuracy=99.0%

Observations & Results

Architecture	Pattern	Initial Loss	Final Loss	Training Accuracy
[2, 4, 8, 16, 2]	Widening	2.5058	0.0367	98.5%
[2, 16, 8, 4, 2]	Narrowing	0.5168	0.0381	98.5%
[2, 16, 4, 16, 2]	Bottleneck	4.1423	0.0327	98.5%
[2, 8, 8, 8, 2]	Uniform Width	3.7995	0.0446	99.0%

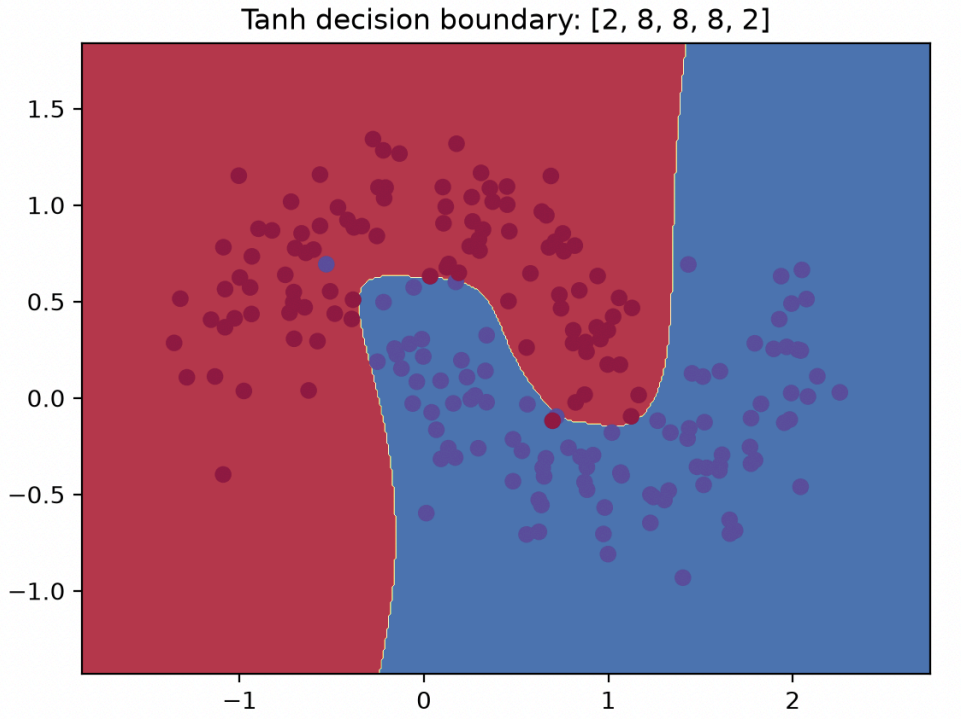
The three mixed-width architectures achieved the same training accuracy of 98.5%. The bottleneck architecture ended up having the lowest final loss of 0.0327. The uniform-width architecture had the highest final loss of 0.0446, but a higher training accuracy of 99.0%. The plots show us many differences based on the architecture, how boundaries extend beyond the observed points where the training data provided little guidance. However, this experiment does not show a clear advantage for a uniform width architecture.

Activations Experiment

For the activations experiment, I made use of the following architecture, for the Tanh, Sigmoid, and ReLU activation functions:

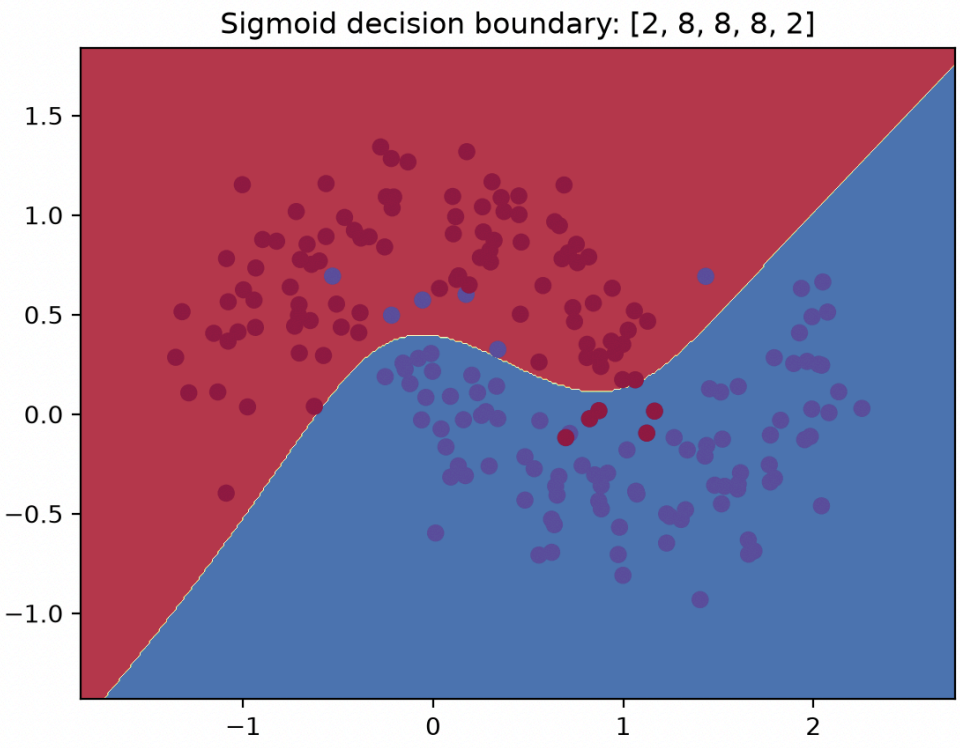
```python
layer_sizes = [2, 8, 8, 8, 2]
```

Tanh Activation



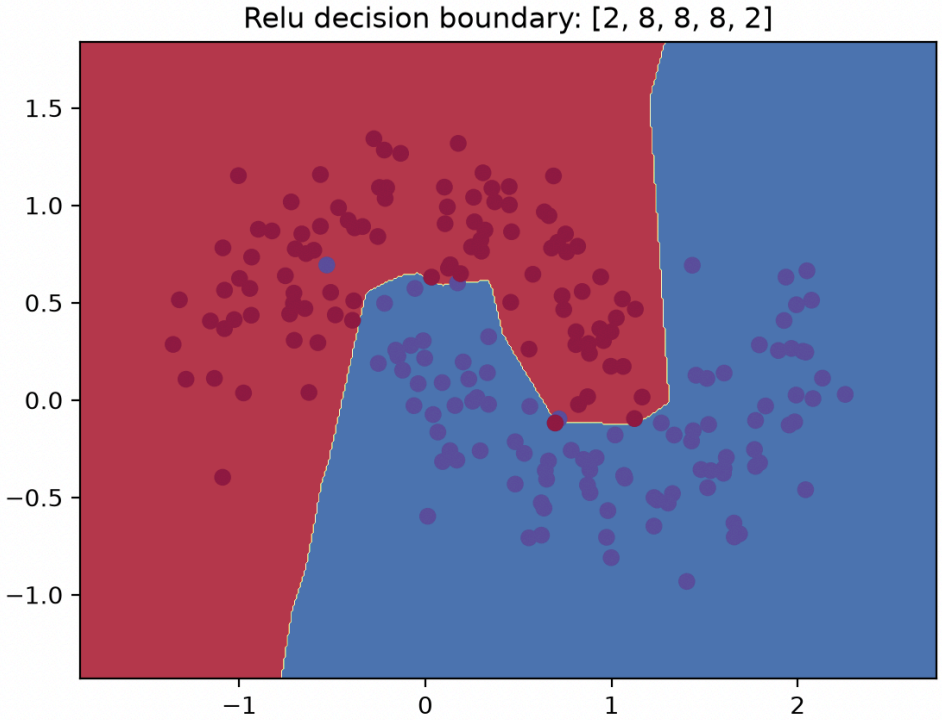
Tanh, 5 Layers [2, 8, 8, 8, 2]: Initial Loss=3.7995, Final Loss=0.0446, Training Accuracy=99.0%

Sigmoid Activation



Sigmoid, 5 Layers [2, 8, 8, 8, 2]: Initial Loss=0.7684, Final Loss=0.1813, Training Accuracy=94.5%

ReLU Activation



Relu, 5 Layers [2, 8, 8, 8, 2]: Initial Loss=1.4175, Final Loss=0.0324, Training Accuracy=98.0%

Observations & Results

Activation	Initial Loss	Final Loss	Training Accuracy
Tanh	3.7995	0.0446	99.0%
Sigmoid	0.7684	0.1813	94.5%
ReLU	1.4175	0.0324	98.0%

Tanh and ReLU produced decision boundaries that closely follow the moon dataset, however Sigmoid produced a much broader boundary that misclassified a lot of points. Tanh was able to achieve the highest training accuracy of 99.0%. ReLU achieved the lowest final loss (0.0324) despite having a slightly lower accuracy (98.0). Sigmoid's higher final loss and 94.5% training accuracy indicates that it trained less effectively under these settings, potentially because sigmoid gradients can become small in deeper networks. The differing ranks for loss and accuracy are possible because loss also reflects prediction confidence, and the weight penalty. Accuracy counts only correct classifications.

Conclusion

Overall, greater depth improved the network's fit to the Make Moons training data, while greater width and different hidden-layer arrangements changed the decision boundaries without consistently reducing loss. Tanh and ReLU performed better than Sigmoid under the tested settings. These results show how architecture and activation affect training performance, but a test set is needed to determine whether the more flexible models also perform better on unseen data.

Wine Recognition Dataset

For the next section, I decided to make use of the Wine Recognition datatset. This dataset contains 178 samples, each with 13 chemical measurements and a label identifying one of three wine classes. I decided to choose it because its classification labels work with our network's softmax output and let us test the model on three classes. Since we are already on page 34 of this report, I decided to keep the experiments a bit more condensed. We will still test for depth, width, and activation functions.

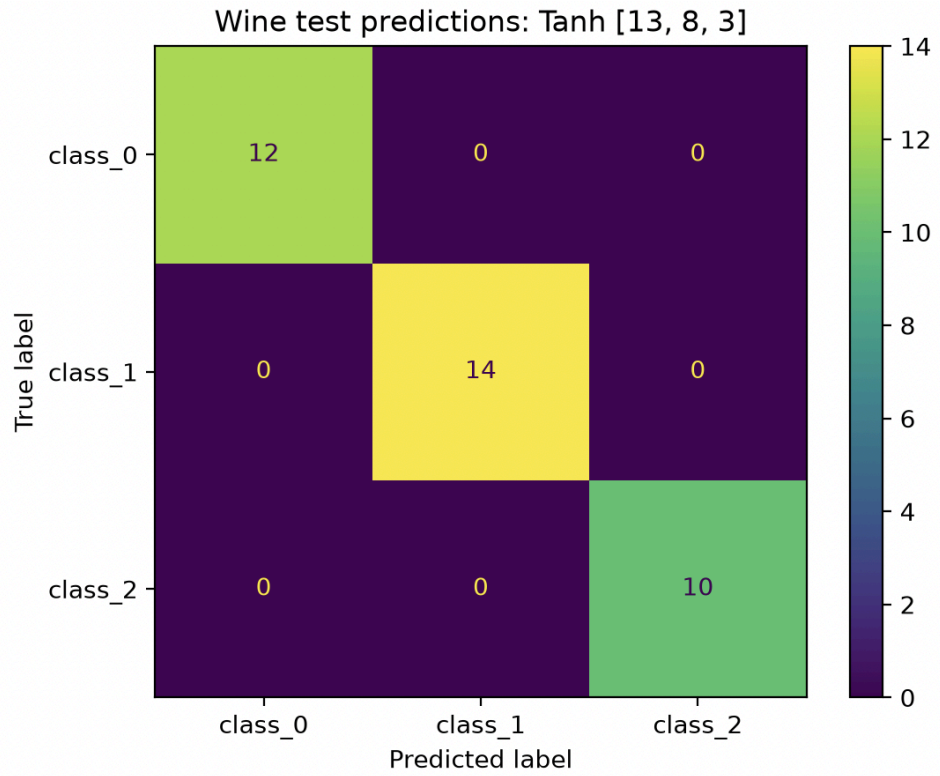
For the results of this dataset, wines are classified into 3 classes: 'class_0', 'class_1', and class_2'. We make use of a confusion matrix to best display the class results, where a darker grid is a stronger classification.

Depth Experiment

For the depth experiment, I made use of the following architectures, with the Tanh activation function:

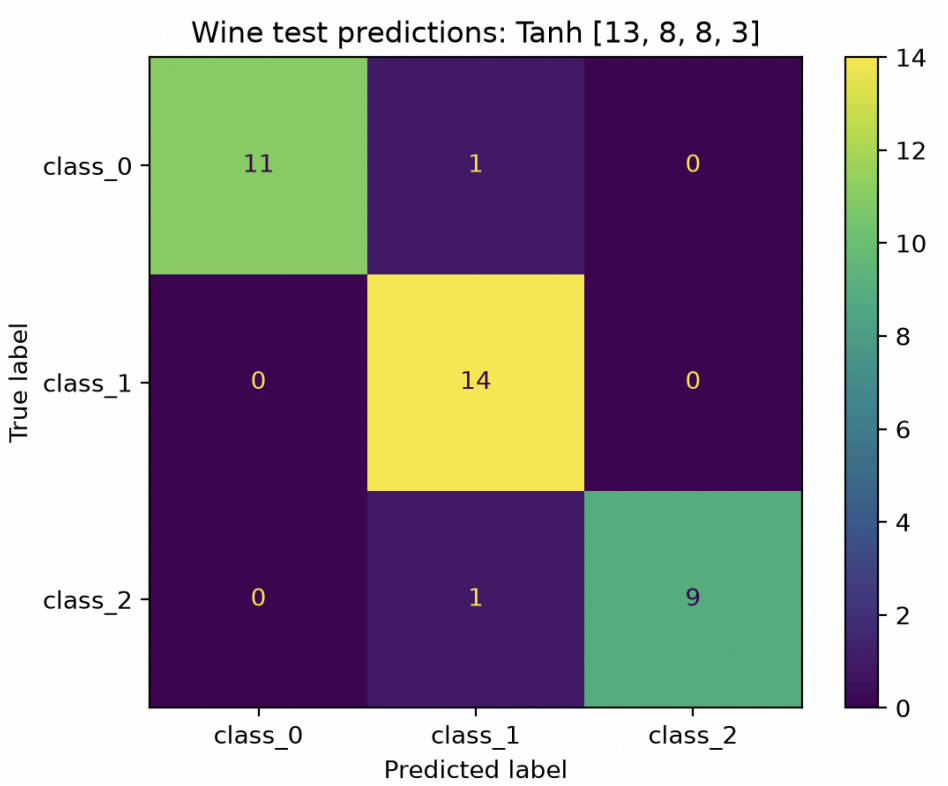
```python
configurations = [
    ([13, 8, 3], "tanh"),
    ([13, 8, 8, 3], "tanh"),
    ([13, 8, 8, 8, 3], "tanh"),
]
```

Depth of 3



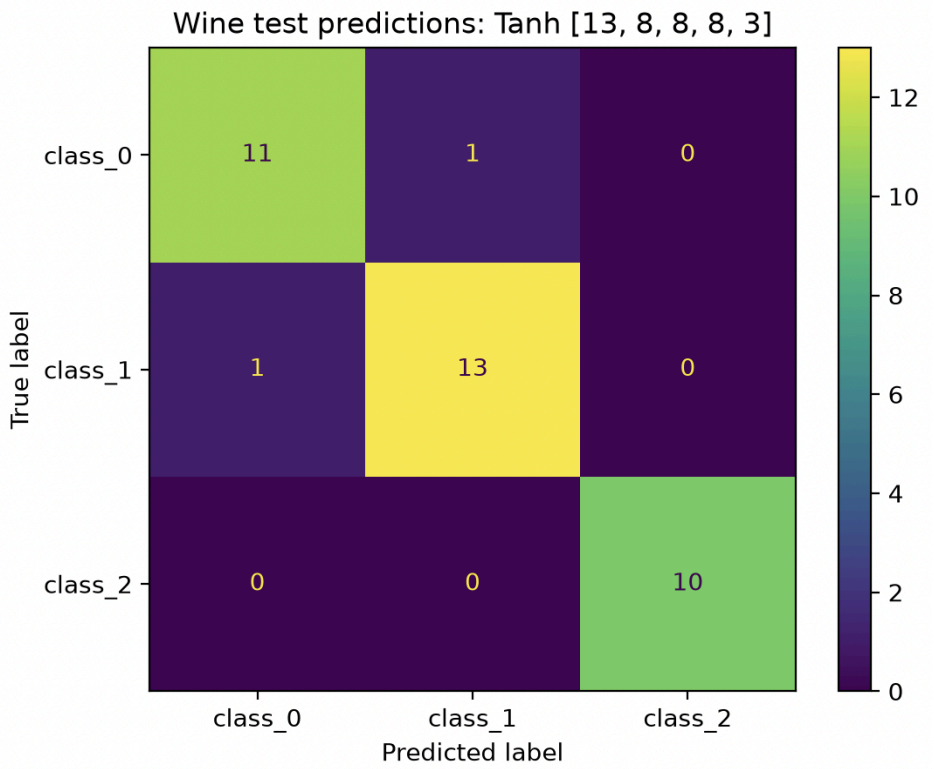
Tanh, [13, 8, 3]: Initial Loss=4.0665, Final Loss=0.0111, Training Accuracy=100.0%, Test Accuracy=100.0%

Depth of 4



Tanh, [13, 8, 8, 3]: Initial Loss=3.2868, Final Loss=0.0105, Training Accuracy=100.0%, Test Accuracy=94.4%

Depth of 5



Tanh, [13, 8, 8, 8, 3]: Initial Loss=2.2582, Final Loss=0.0142, Training Accuracy=100.0%, Test Accuracy=94.4%

Observations

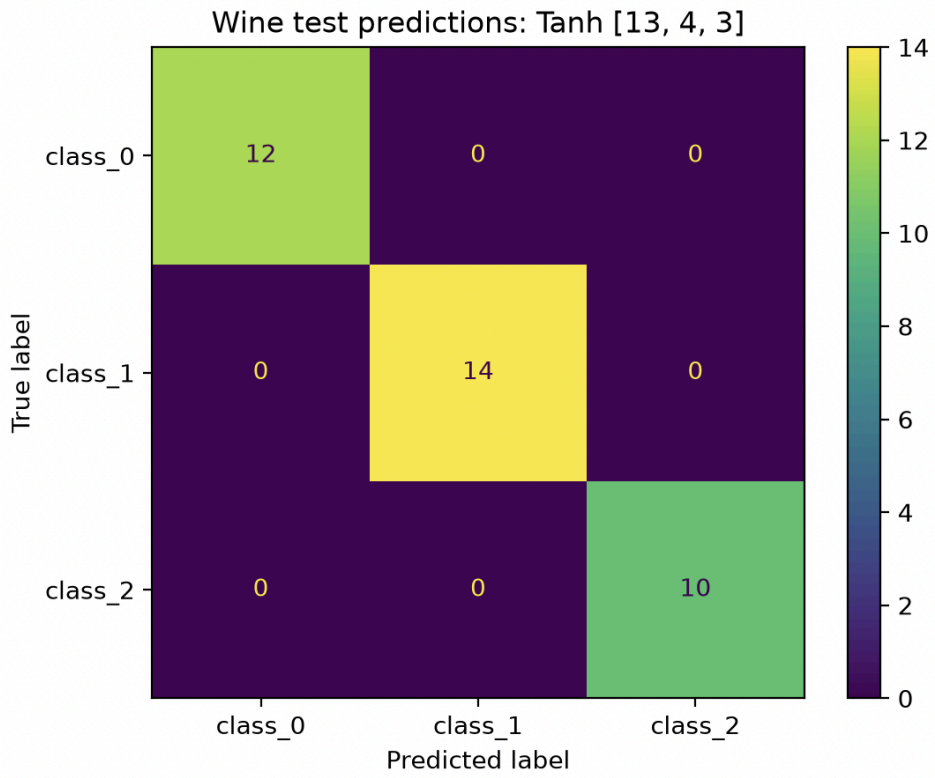
All three Tanh networks achieved 100% training accuracy, but the three-layer network also achieved the highest test accuracy (100%, versus 94.4% for both deeper networks). Adding depth did not improve performance on this test split, even though the four-layer network had a slightly lower final training loss.

Width Experiment

For the width experiment, I made use of the following architectures, with the Tanh activation function:

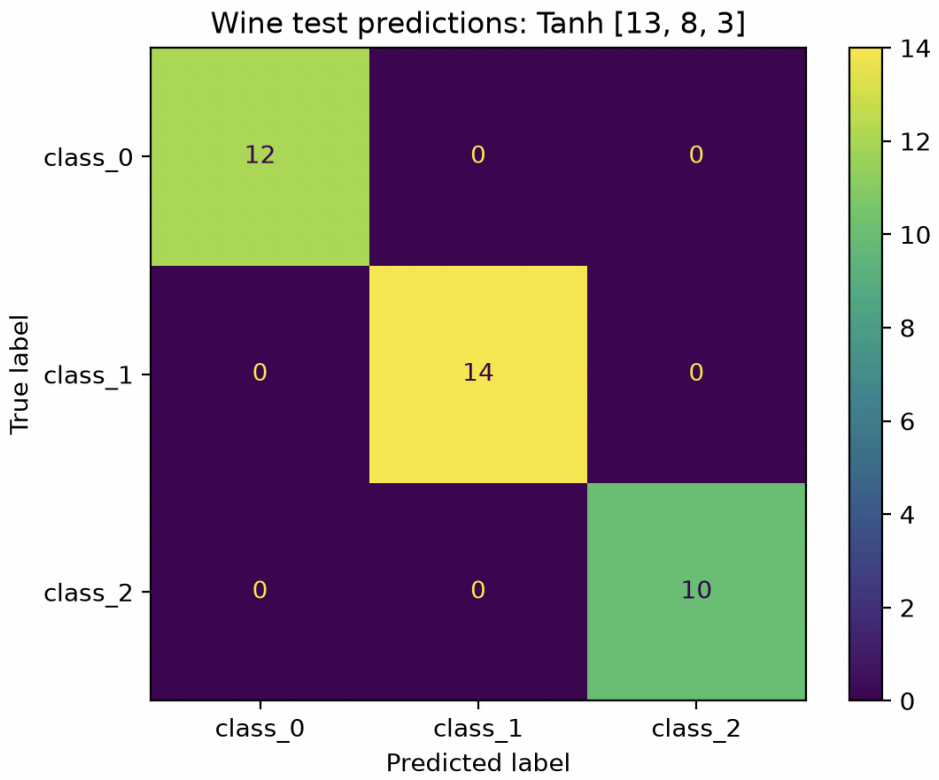
```python
configurations = [
    ([13, 4, 3], "tanh"),
    ([13, 8, 3], "tanh"),
    ([13, 16, 3], "tanh"),
]
```

Width of 4



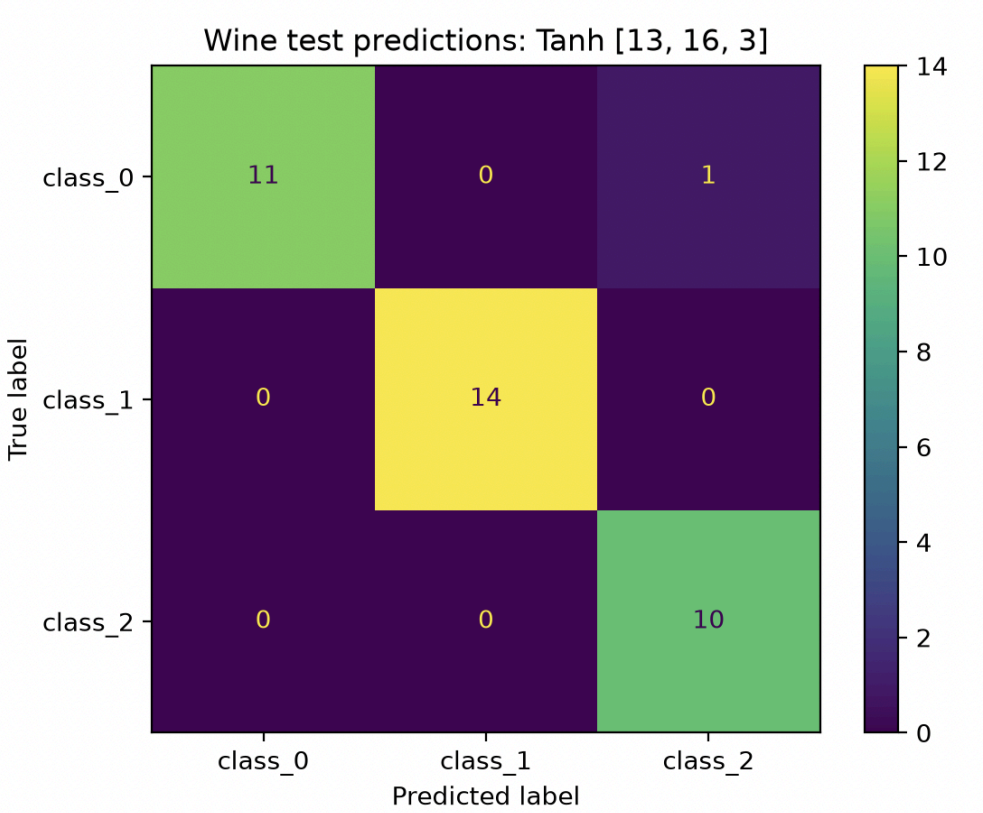
Tanh, [13, 4, 3]: Initial Loss=0.9103, Final Loss=0.0162, Training Accuracy=100.0%, Test Accuracy=100.0%

Width of 8



Tanh, [13, 8, 3]: Initial Loss=4.0665, Final Loss=0.0111, Training Accuracy=100.0%, Test Accuracy=100.0%

Width of 16



Tanh, [13, 16, 3]: Initial Loss=1.4913, Final Loss=0.0121, Training Accuracy=100.0%, Test Accuracy=97.2%

Observations

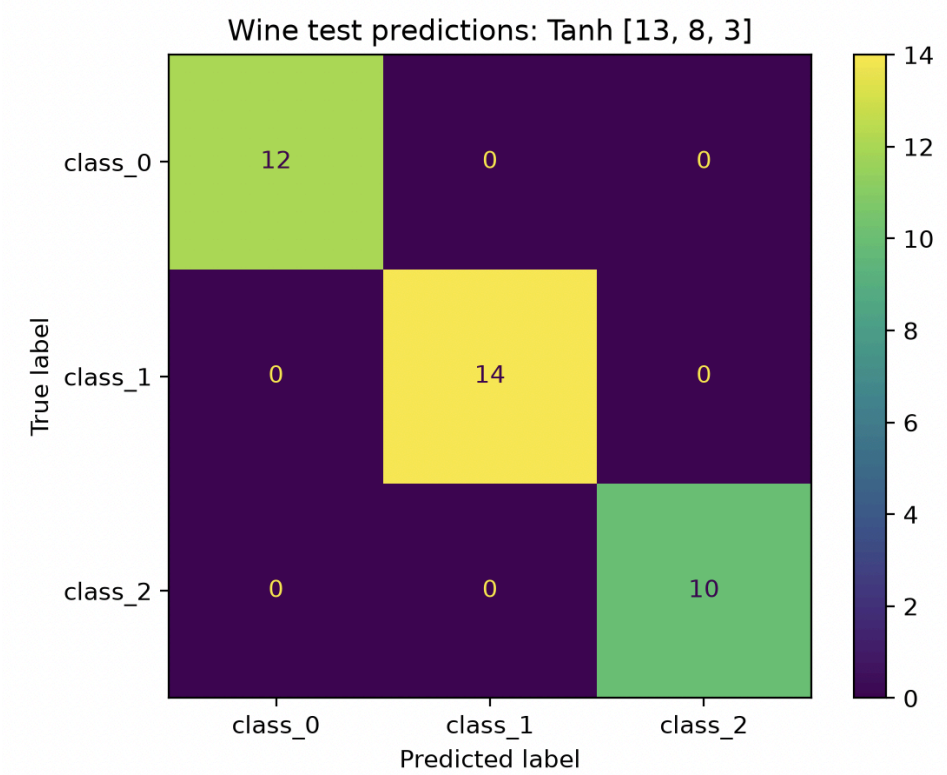
All three widths achieved 100% training accuracy, while the 4 and 8-unit networks also classified every test sample correctly. Increasing width beyond 8 did not improve results on this split: the 16-unit network had 97.2% test accuracy, and the 8-unit network had the lowest final training loss.

Activations Experiment

For the activations experiment, I made use of the following architecture, for the Tanh, Sigmoid, and ReLU activation functions:

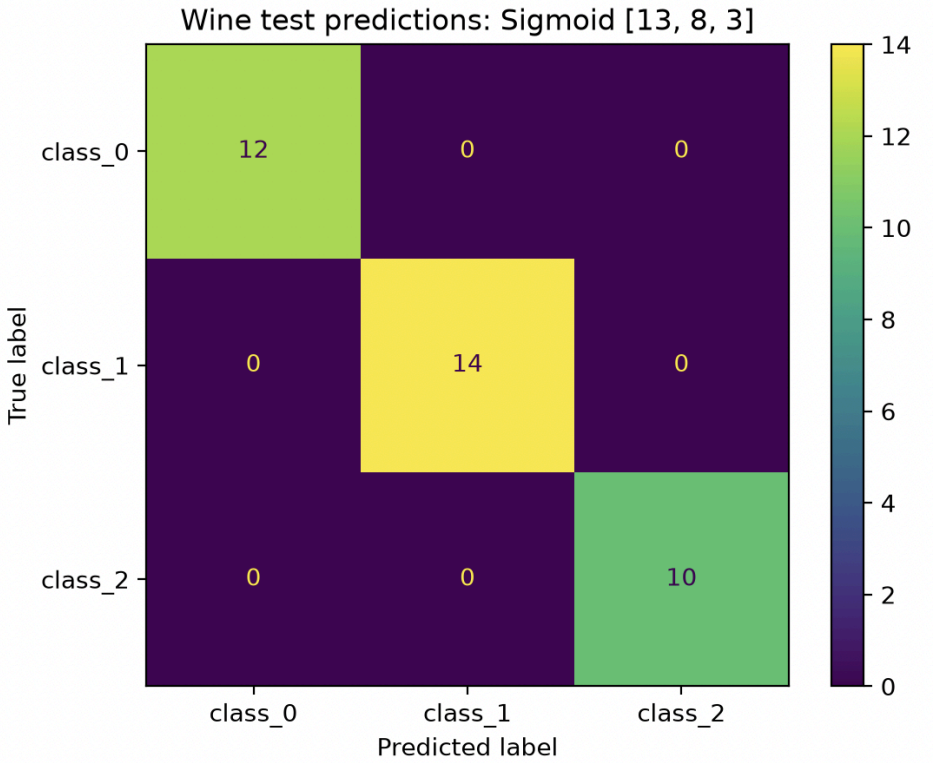
```python
configurations = [
    ([13, 8, 3], "tanh"),
    ([13, 8, 3], "sigmoid"),
    ([13, 8, 3], "relu"),
]
```

Tanh Activation



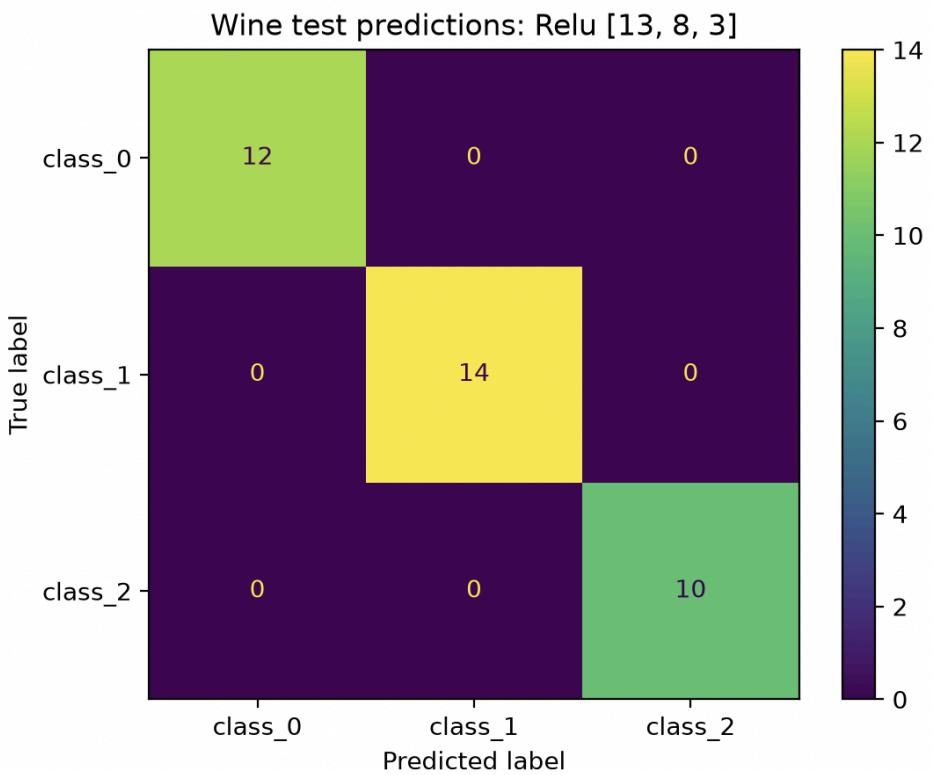
Tanh, [13, 8, 3]: Initial Loss=4.0665, Final Loss=0.0111, Training Accuracy=100.0%, Test Accuracy=100.0%

Sigmoid Activation



Sigmoid, [13, 8, 3]: Initial Loss=2.1077, Final Loss=0.0265, Training Accuracy=100.0%, Test Accuracy=100.0%

ReLU Activation



Relu, [13, 8, 3]: Initial Loss=10.7437, Final Loss=0.0073, Training Accuracy=100.0%, Test Accuracy=100.0%

Observations

All three activations achieved 100% accuracy on both the training and test sets for [13, 8, 3]. ReLU had the lowest final training loss (0.0073), followed by Tanh (0.0111) and Sigmoid (0.0265), but none improved classification accuracy over the others on this split.

Training a Simple Deep Convolutional Network on MNIST

a) Build and Train a 4-layer DCN

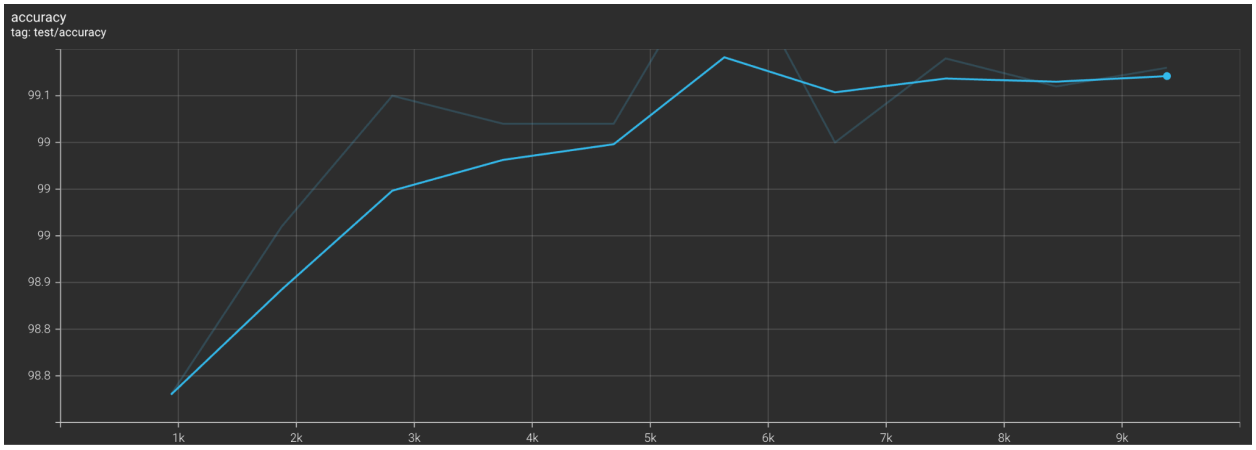
MNIST contains 28 × 28 grayscale images labeled with digits 0–9. I trained the network using two convolution–ReLU–max-pooling stages, a 1,024-unit fully connected layer, 50% dropout, and a ten-class softmax output. Training used batches of 64 images, the ‘Adam’ optimizer with a learning rate of 0.001, and my Mac’s MPS device for 10 epochs. Training loss decreased substantially over the run. The final test loss was 0.0342 and final test accuracy was 99.26%; the highest accuracy observed during the run was 99.26% at epoch 10. These results show that the network learned to classify the handwritten digits effectively.

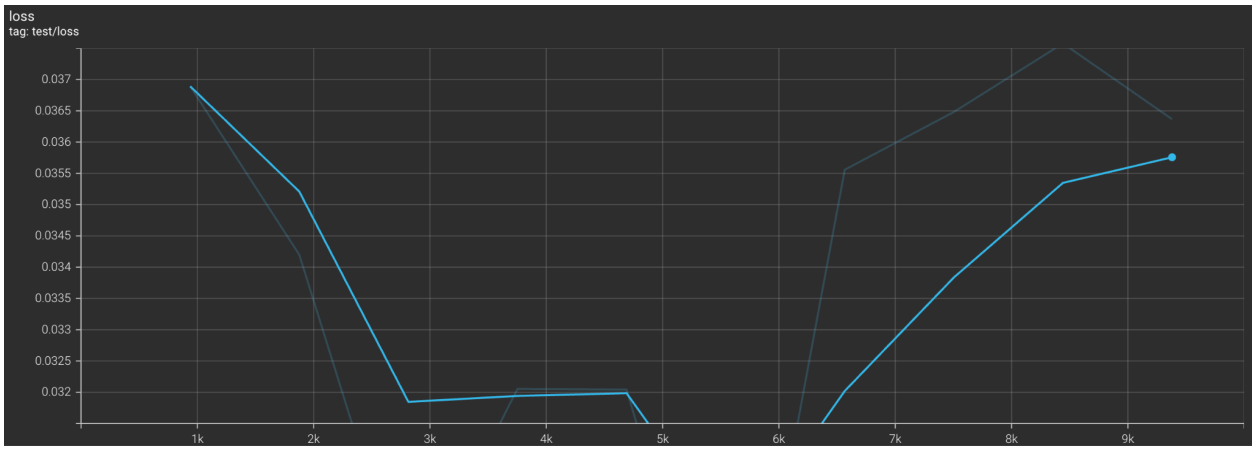
Overall results for 10 epochs:

Epoch	Test Loss	Test Accuracy
1	0.0365	98.87%
2	0.0339	99.00%
3	0.0253	99.22%
4	0.0336	99.05%
5	0.0309	99.14%
6	0.0297	99.21%
7	0.0290	99.21%
8	0.0349	99.05%
9	0.0414	98.97%
10	0.0342	99.26%

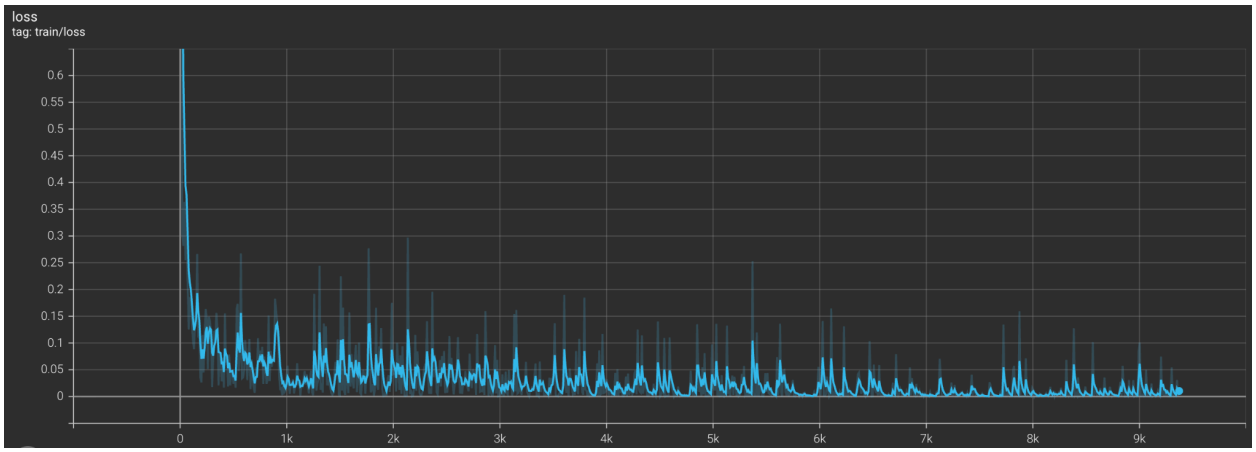
Below you find all respective results for the convolutional neural network (over 10 epochs).

Test Results:





Training Results:

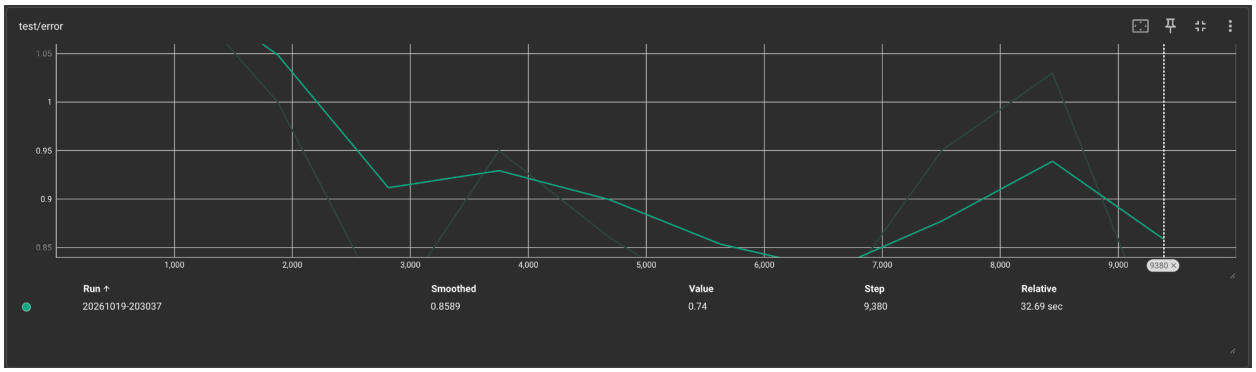


Convolution Layer Comparison



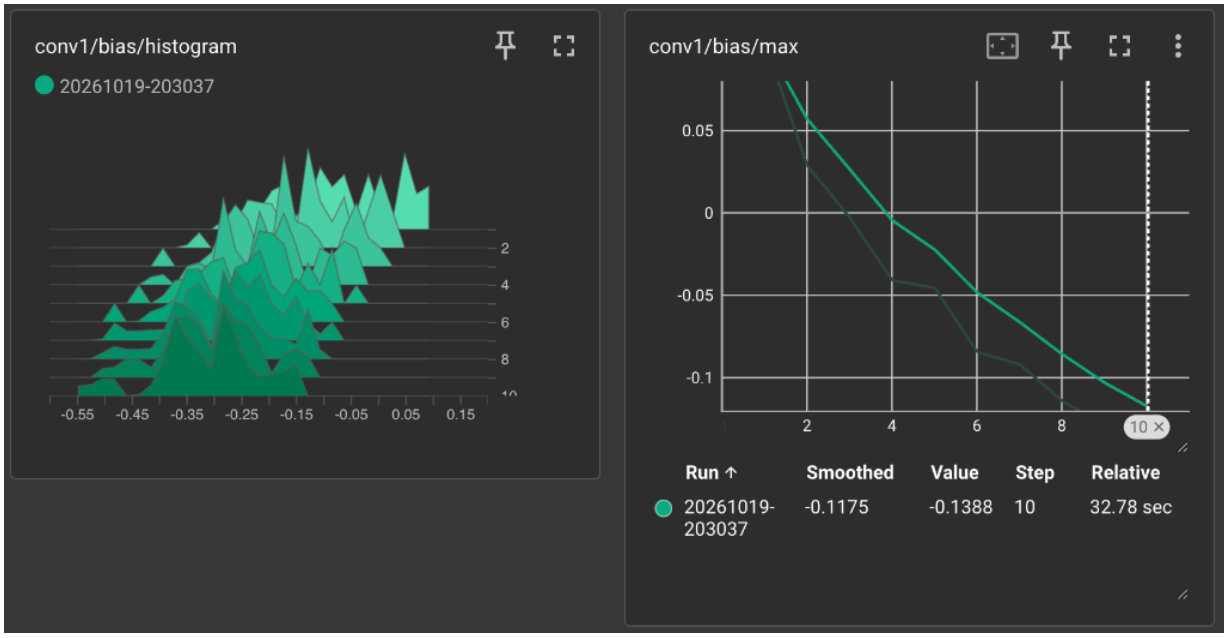
b) More on Visualizing Your Training

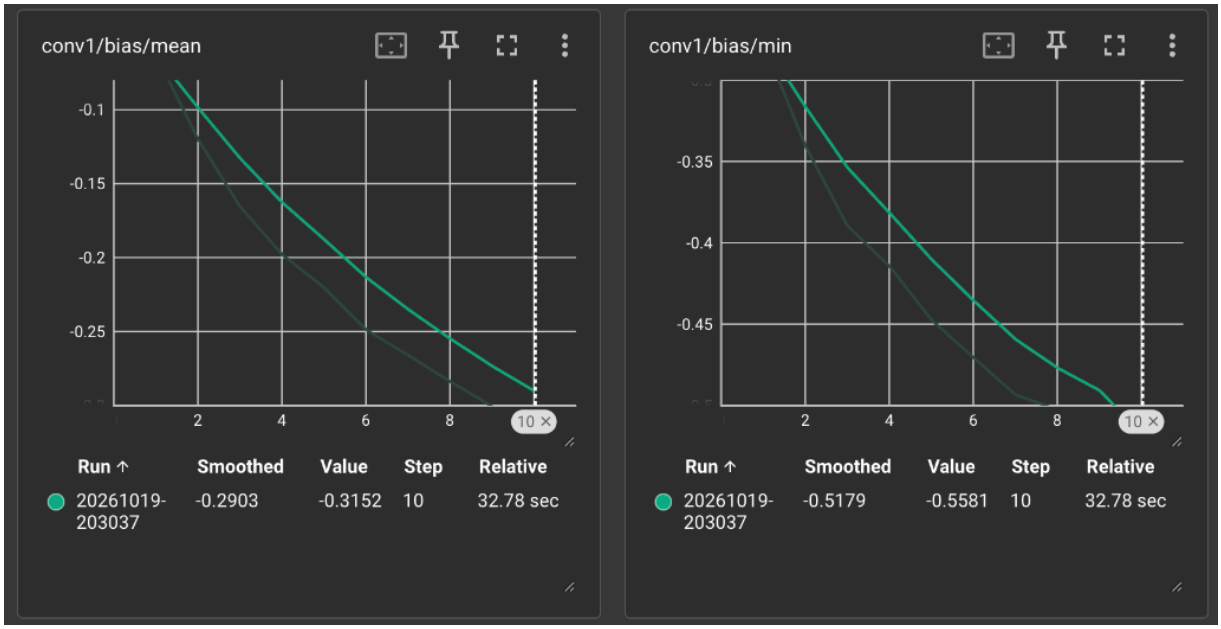
Test Error Across 10 Epochs

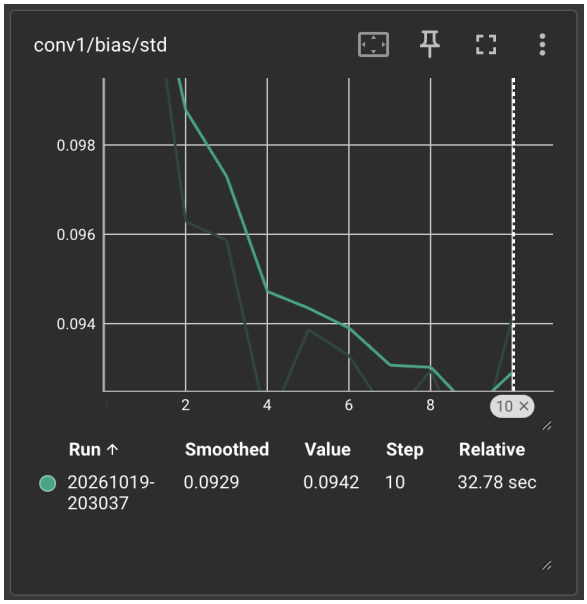


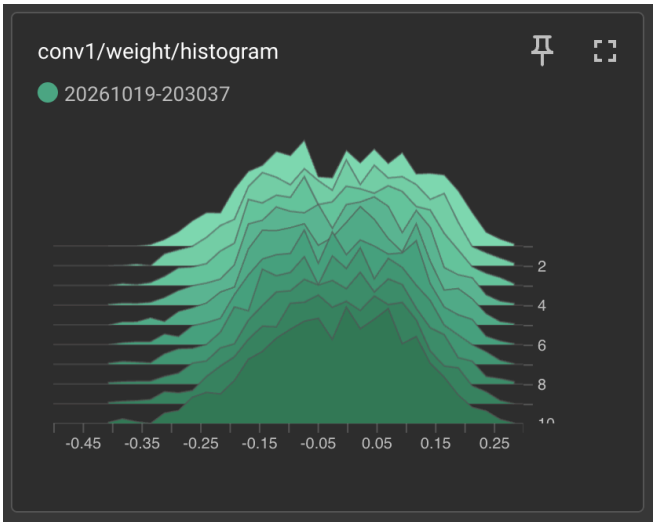
Weights and Biases Across Layers

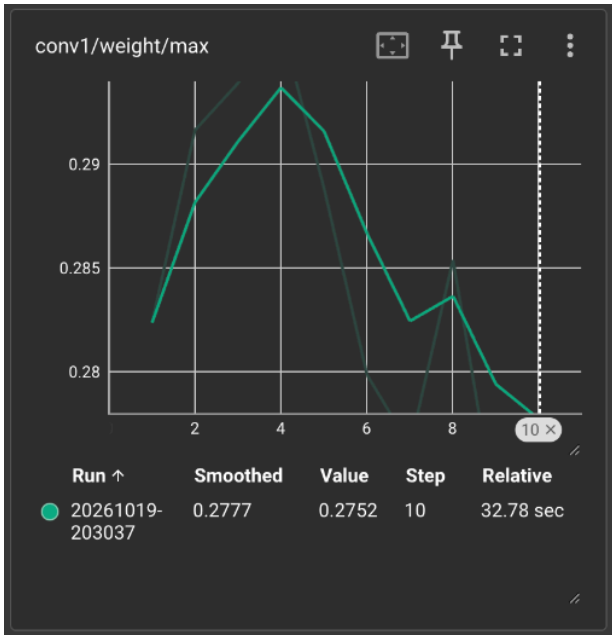
Convolution Layer 1

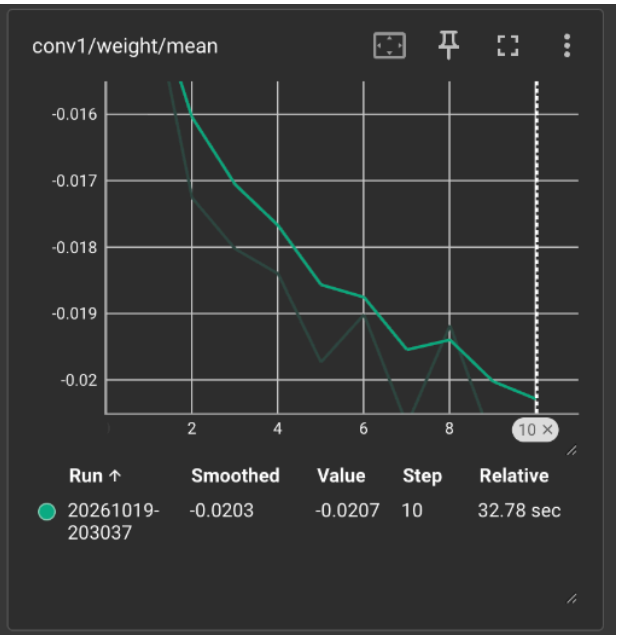


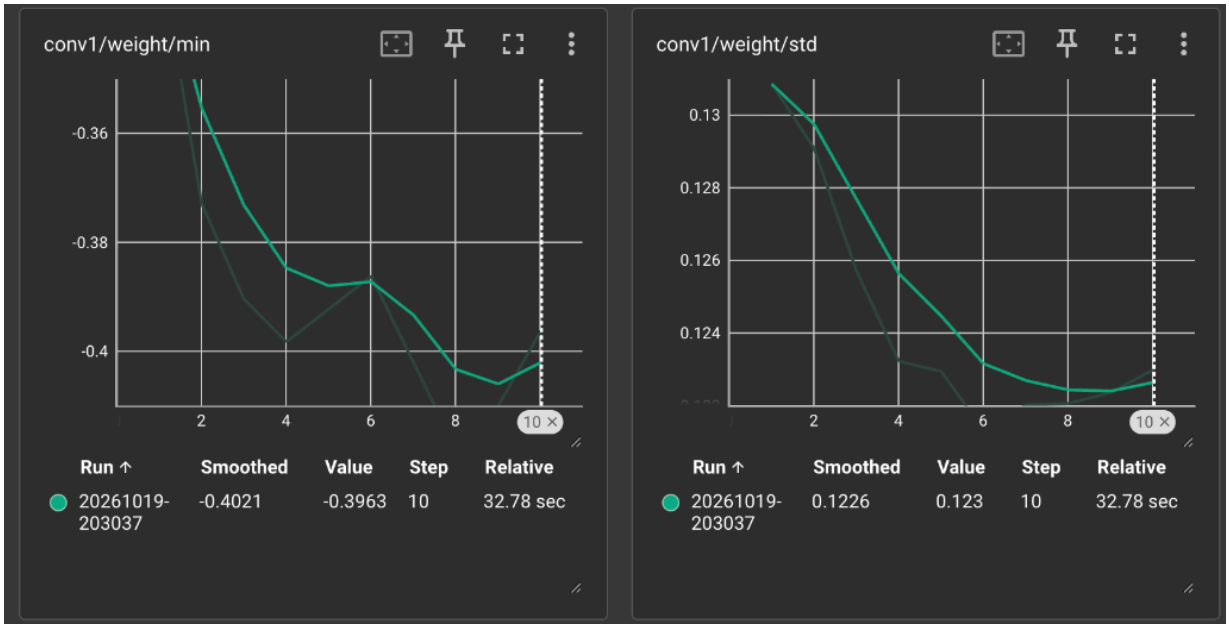



conv1/bias/std
0.098
0.096
0.094
2
4
6
8
10
Run
Smoothed
Value
Step
Relative
20261019-203037
0.0929
0.0942
10
32.78 sec

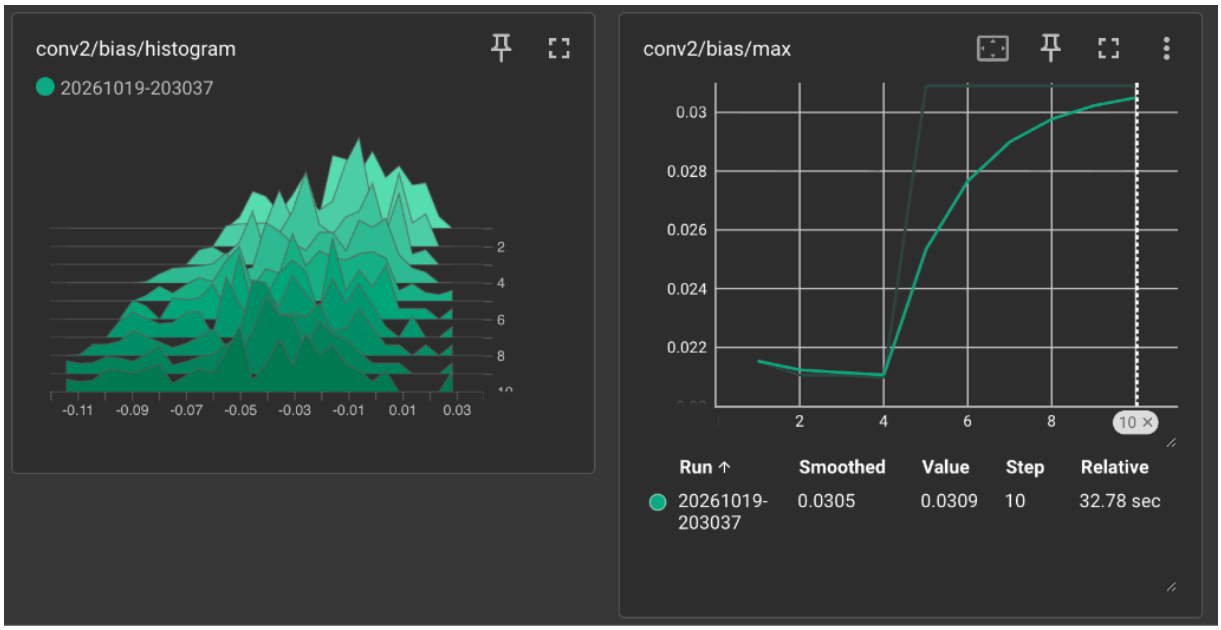

conv1/weight/histogram
20261019-203037
2
4
6
8
10
-0.45
-0.35
-0.25
-0.15
-0.05
0.05
0.15
0.25

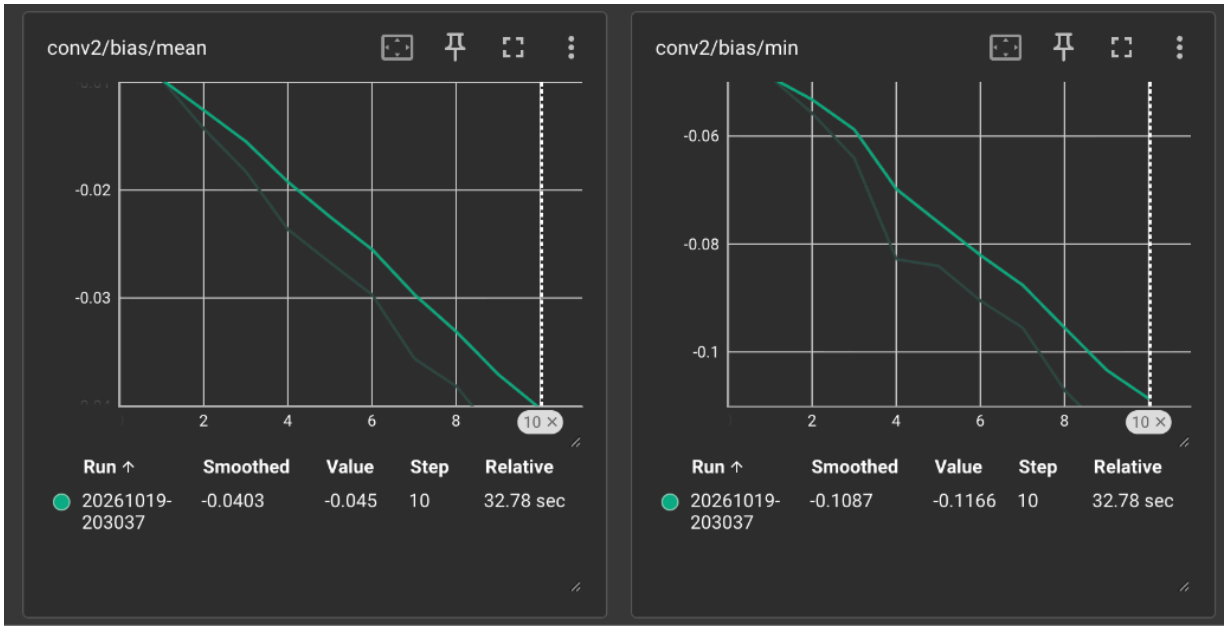

conv1/weight/max
0.29
0.285
0.28
2
4
6
8
10
Run
Smoothed
Value
Step
Relative
20261019-203037
0.2777
0.2752
10
32.78 sec

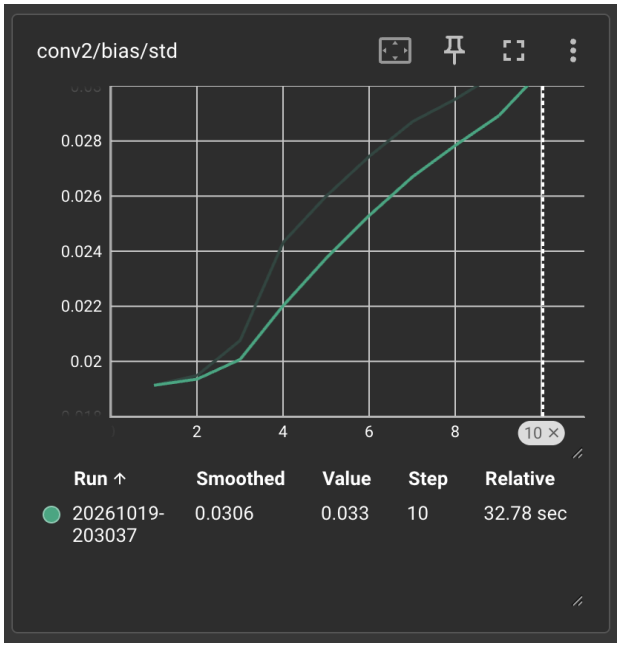

conv1/weight/mean
-0.016
-0.017
-0.018
-0.019
-0.02
2
4
6
8
10
Run
Smoothed
Value
Step
Relative
20261019-203037
-0.0203
-0.0207
10
32.78 sec

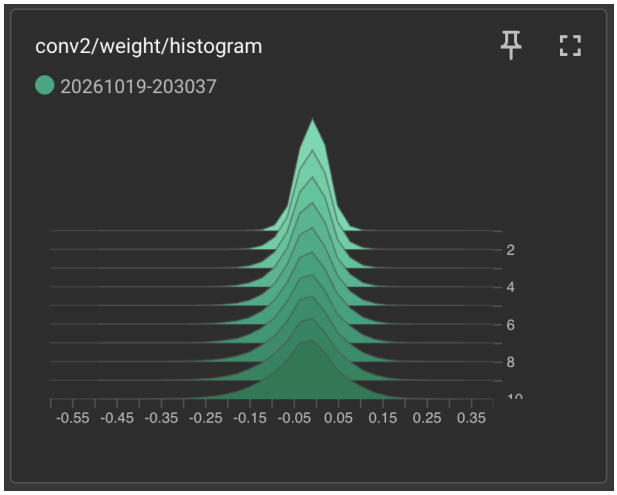


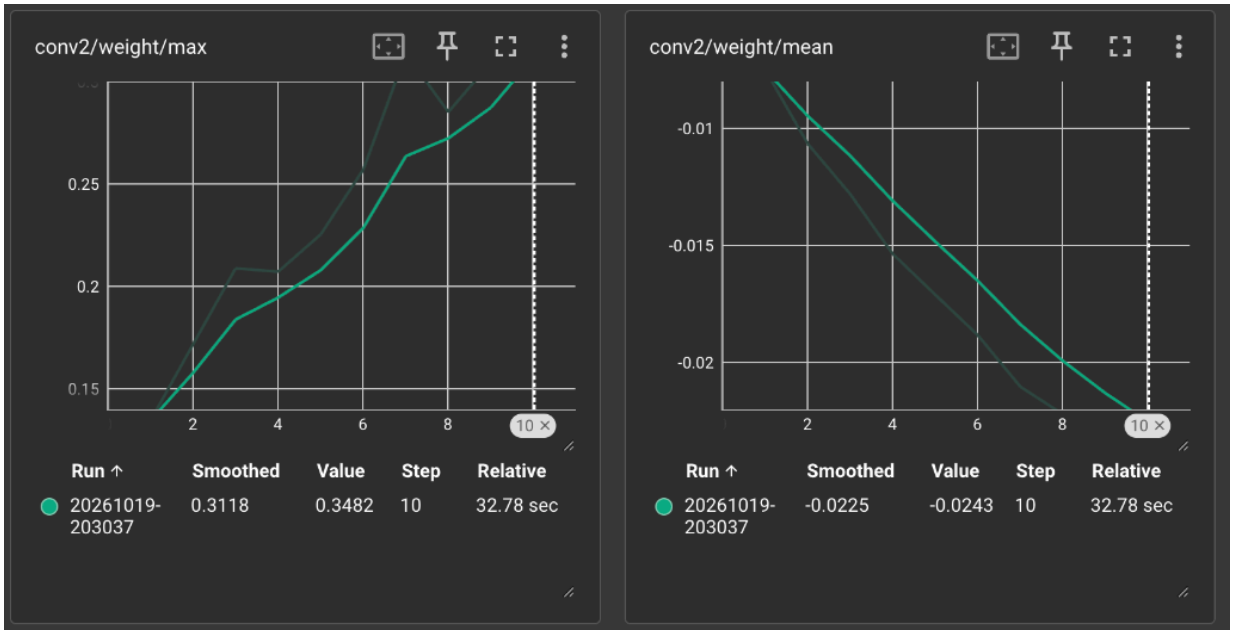
Convolution Layer 2

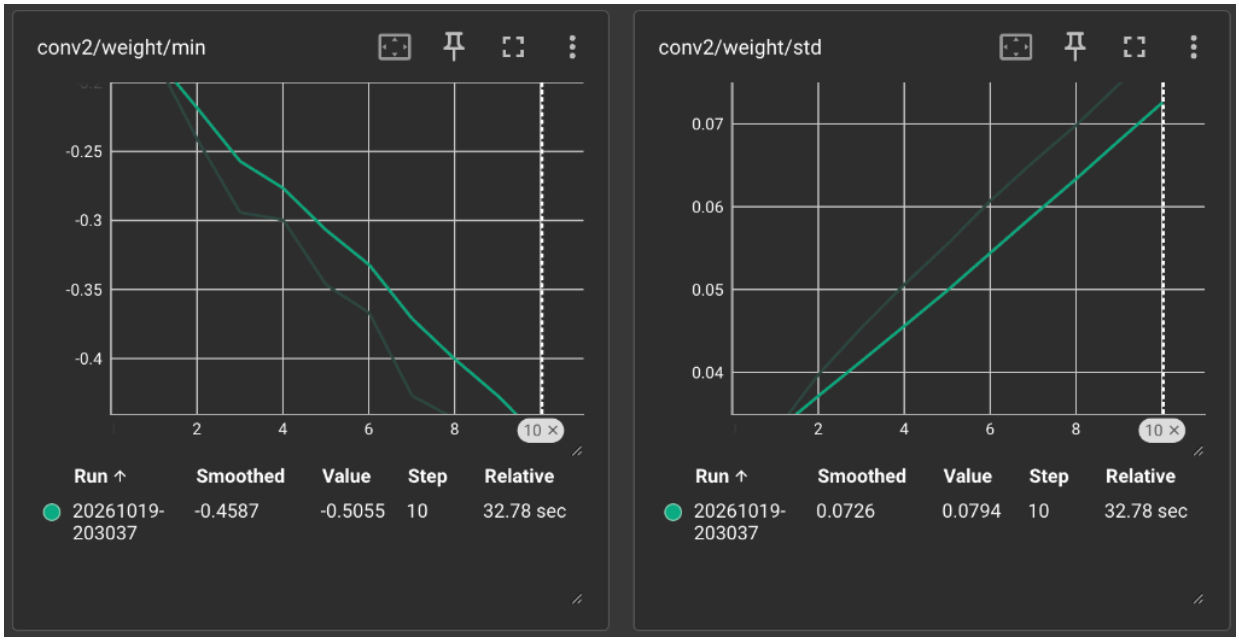



conv2/bias/mean
-0.02
-0.03
2
4
6
8
10
Run
Smoothed
Value
Step
Relative
20261019-203037
-0.0403
-0.045
10
32.78 sec
conv2/bias/min
-0.06
-0.08
-0.1
2
4
6
8
10
Run
Smoothed
Value
Step
Relative
20261019-203037
-0.1087
-0.1166
10
32.78 sec

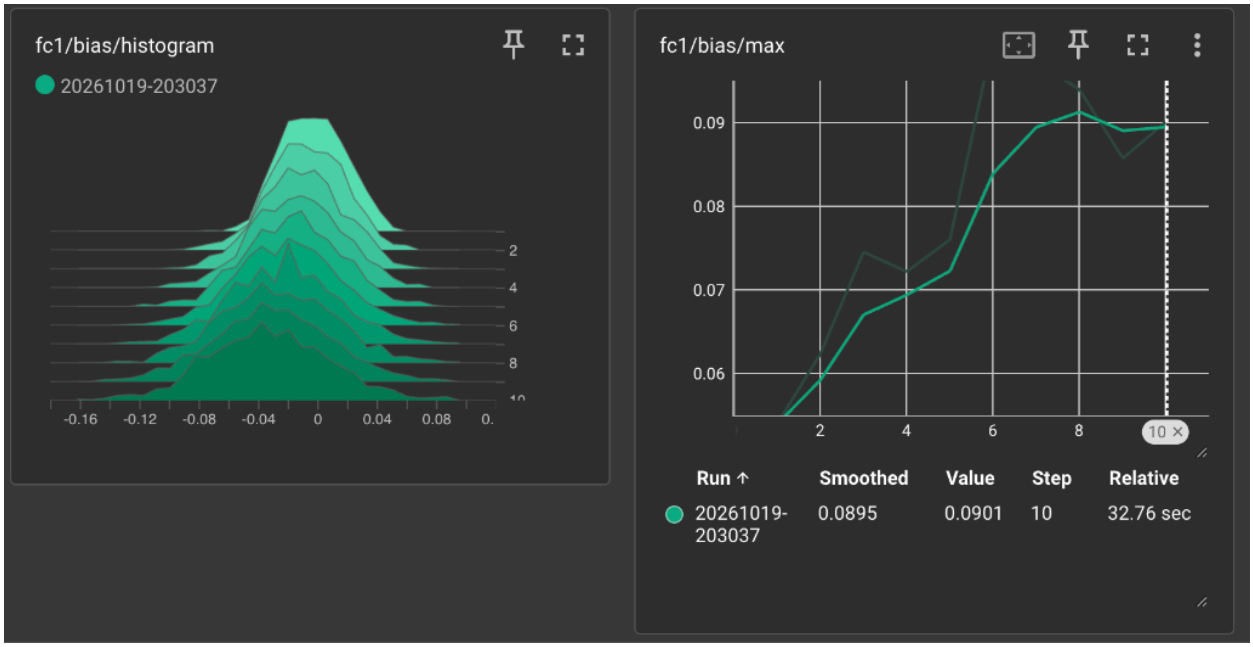

conv2/bias/std
0.028
0.026
0.024
0.022
0.02
2
4
6
8
10
Run
Smoothed
Value
Step
Relative
20261019-203037
0.0306
0.033
10
32.78 sec

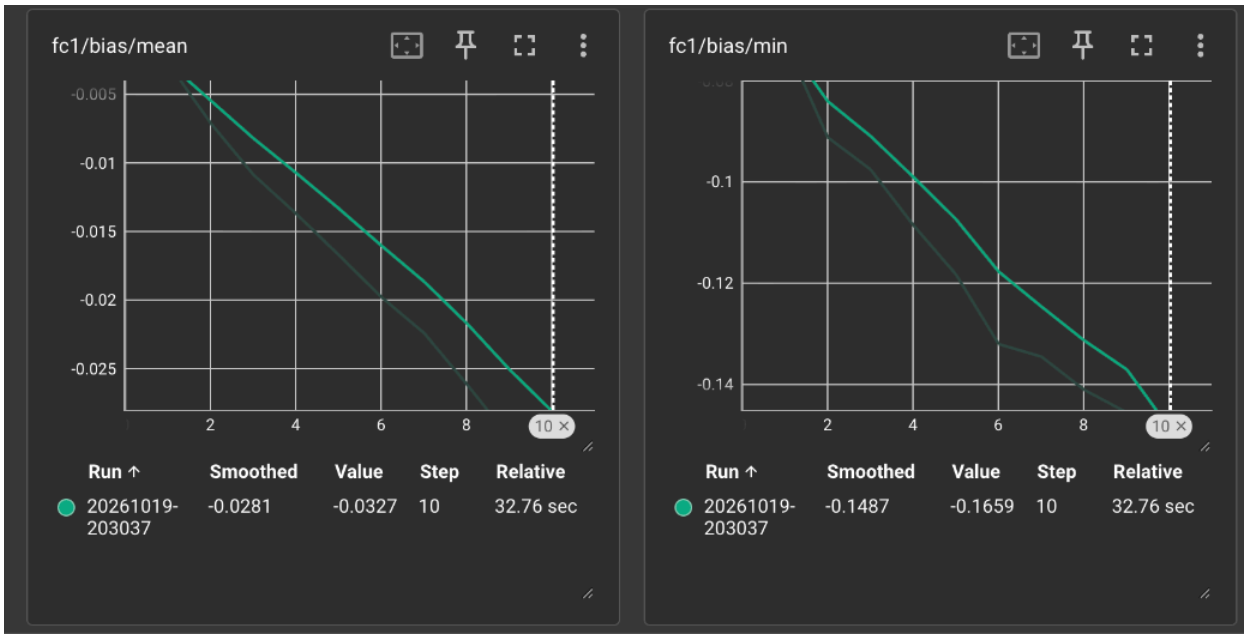

conv2/weight/histogram
20261019-203037
2
4
6
8
10
-0.55
-0.45
-0.35
-0.25
-0.15
-0.05
0.05
0.15
0.25
0.35

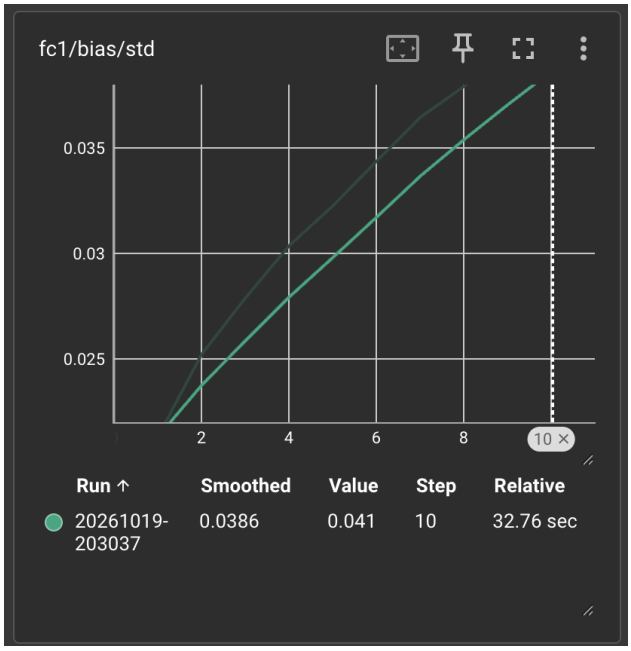

conv2/weight/max
0.25
0.2
0.15
2
4
6
8
10 ×
Run ↑ Smoothed Value Step Relative
20261019-203037 0.3118 0.3482 10 32.78 sec
conv2/weight/mean
-0.01
-0.015
-0.02
2
4
6
8
10 ×
Run ↑ Smoothed Value Step Relative
20261019-203037 -0.0225 -0.0243 10 32.78 sec

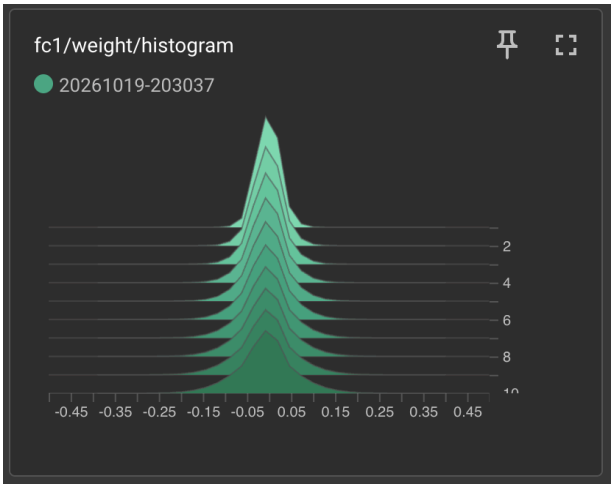

conv2/weight/min
-0.25
-0.3
-0.35
-0.4
2
4
6
8
10 ×
Run ↑ Smoothed Value Step Relative
20261019-203037 -0.4587 -0.5055 10 32.78 sec
conv2/weight/std
0.07
0.06
0.05
0.04
2
4
6
8
10 ×
Run ↑ Smoothed Value Step Relative
20261019-203037 0.0726 0.0794 10 32.78 sec

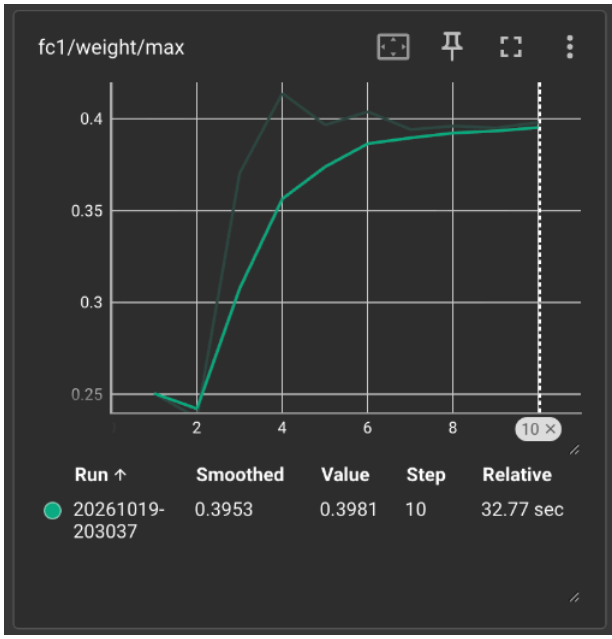
Fully Connected Layer 1

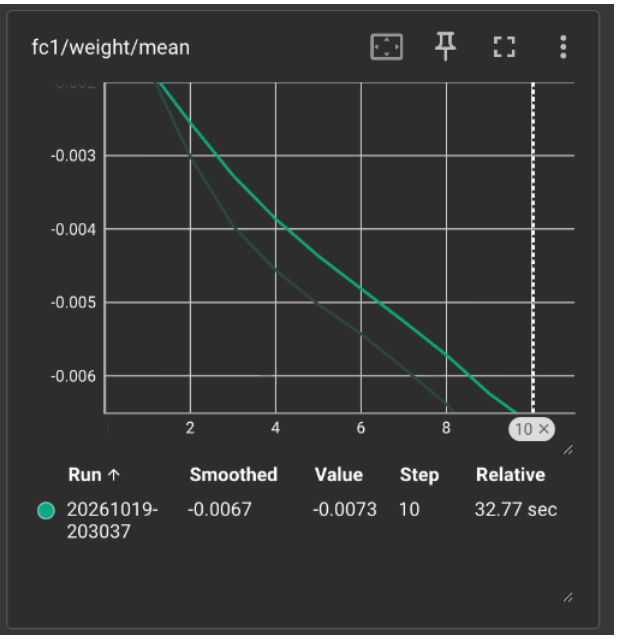


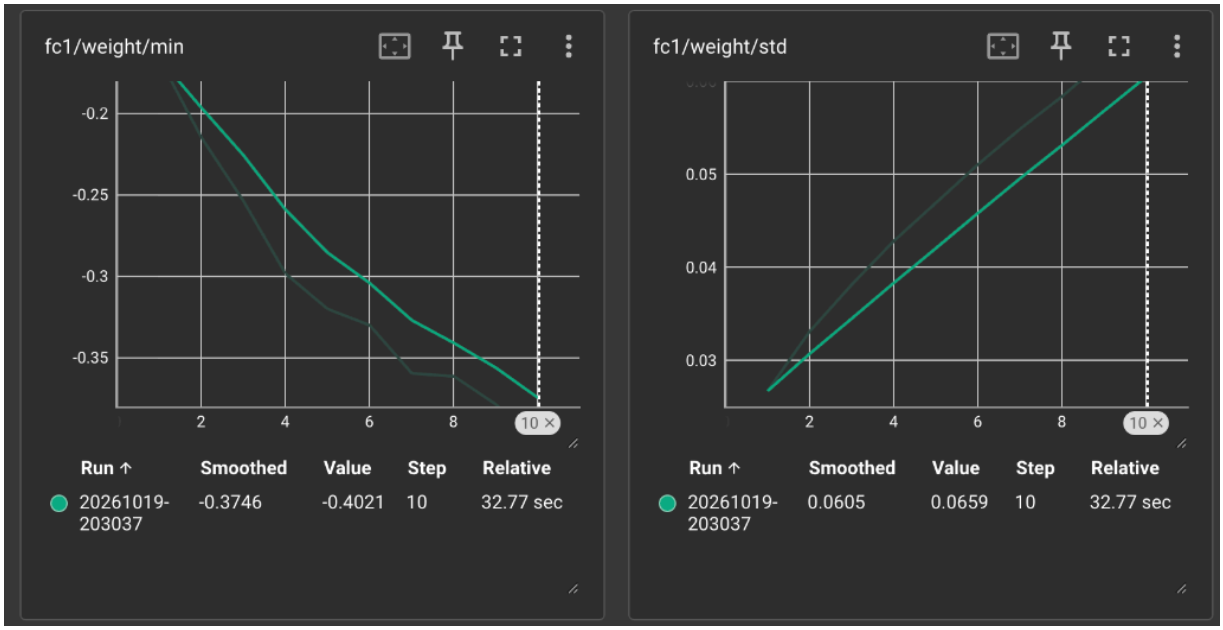



fc1/bias/std
0.035
0.03
0.025
2
4
6
8
10
Run
Smoothed
Value
Step
Relative
20261019-203037
0.0386
0.041
10
32.76 sec

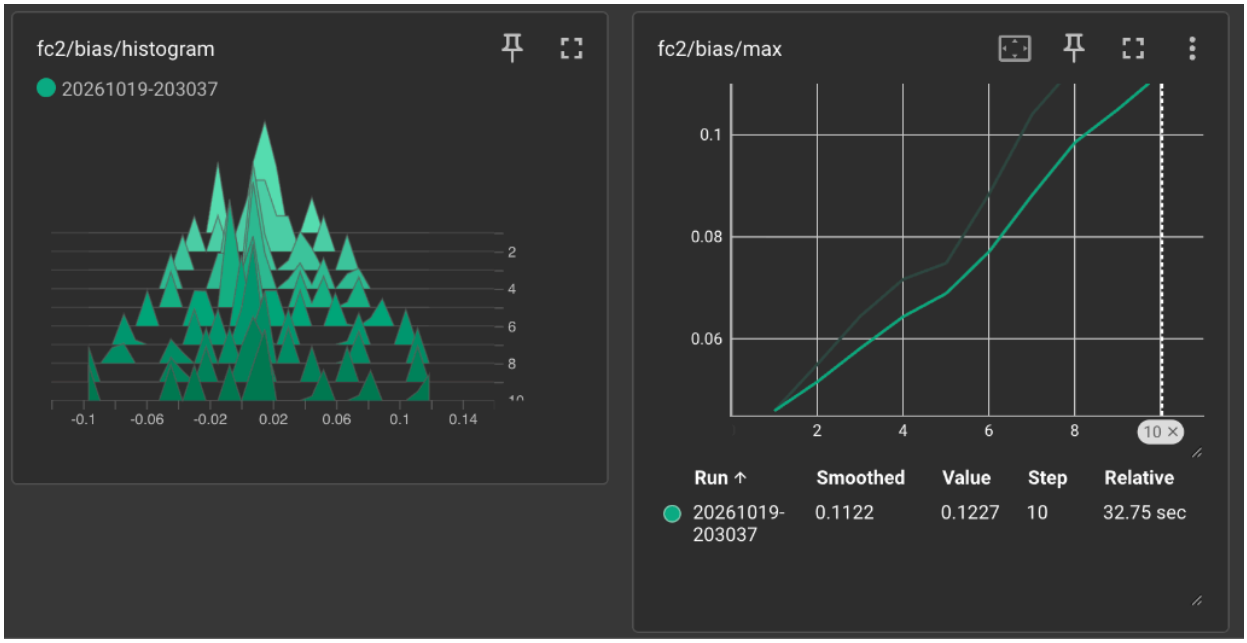

fc1/weight/histogram
20261019-203037
2
4
6
8
-0.45
-0.35
-0.25
-0.15
-0.05
0.05
0.15
0.25
0.35
0.45

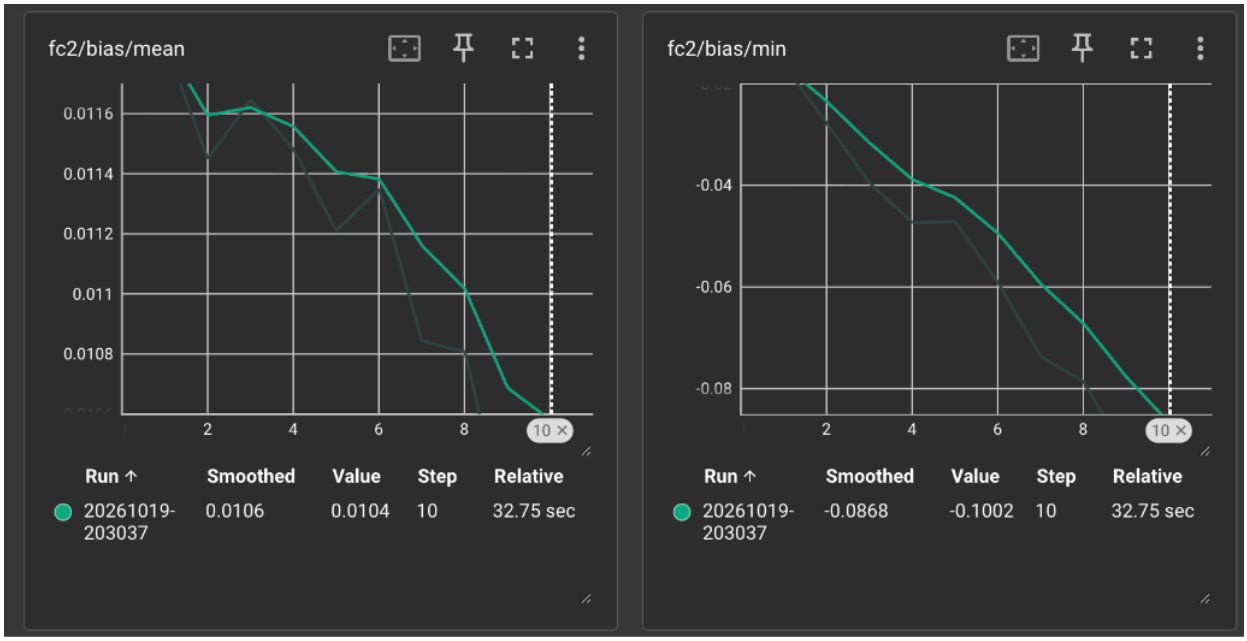

fc1/weight/max
0.4
0.35
0.3
0.25
2
4
6
8
10
Run
Smoothed
Value
Step
Relative
20261019-203037
0.3953
0.3981
10
32.77 sec

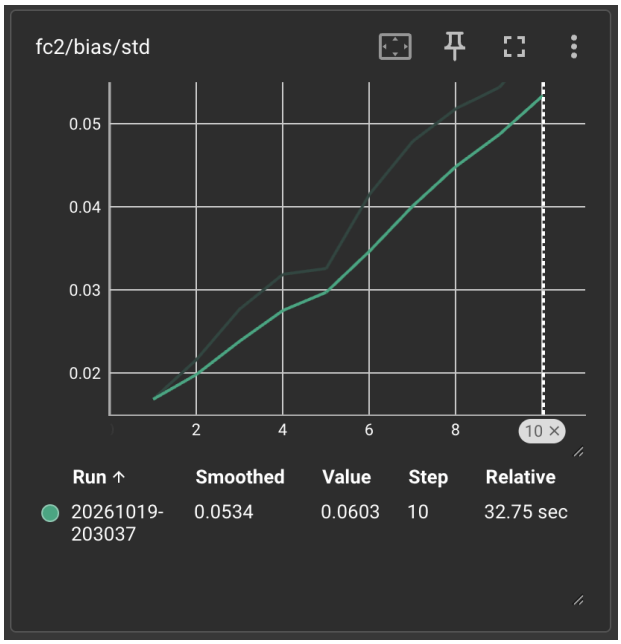

fc1/weight/mean
-0.003
-0.004
-0.005
-0.006
2
4
6
8
10
Run
Smoothed
Value
Step
Relative
20261019-203037
-0.0067
-0.0073
10
32.77 sec

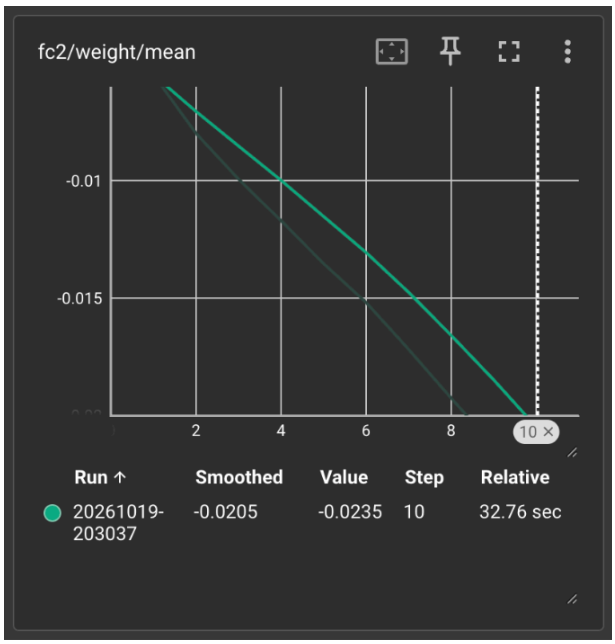

fc1/weight/min
-0.2
-0.25
-0.3
-0.35
2
4
6
8
10
Run
Smoothed
Value
Step
Relative
20261019-203037
-0.3746
-0.4021
10
32.77 sec
fc1/weight/std
0.05
0.04
0.03
2
4
6
8
10
Run
Smoothed
Value
Step
Relative
20261019-203037
0.0605
0.0659
10
32.77 sec

Fully Connected Layer 2



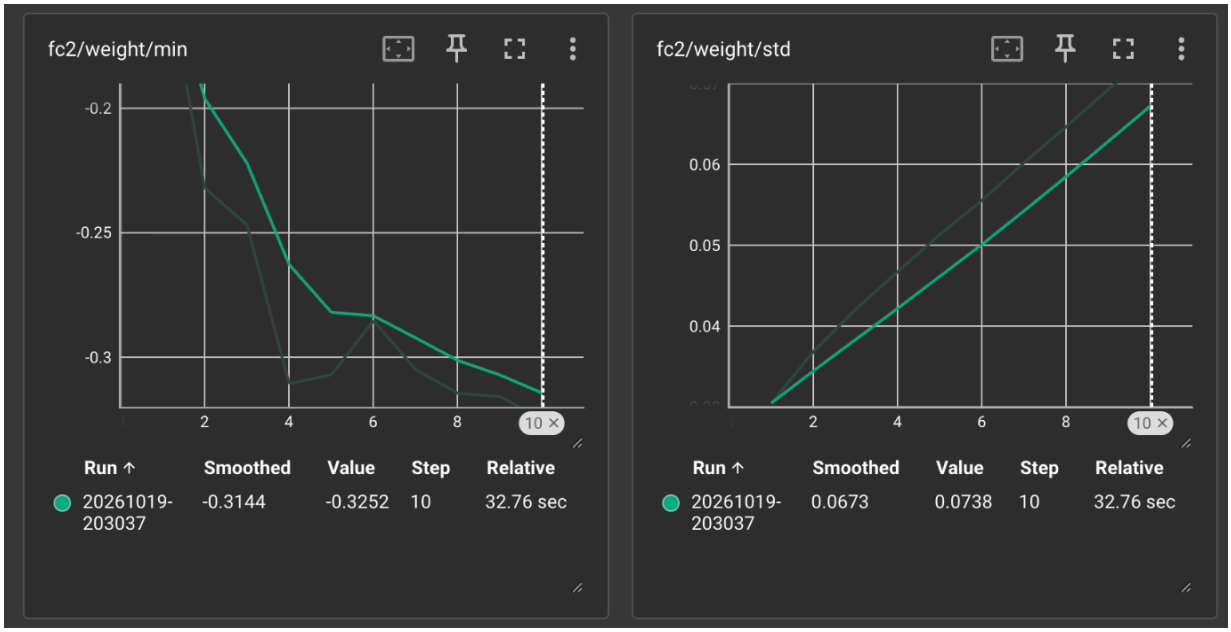



fc2/bias/std
0.05
0.04
0.03
0.02
2
4
6
8
10
Run
Smoothed
Value
Step
Relative
20261019-203037
0.0534
0.0603
10
32.75 sec


fc2/weight/mean
-0.01
-0.015
2
4
6
8
10
Run
Smoothed
Value
Step
Relative
20261019-203037
-0.0205
-0.0235
10
32.76 sec

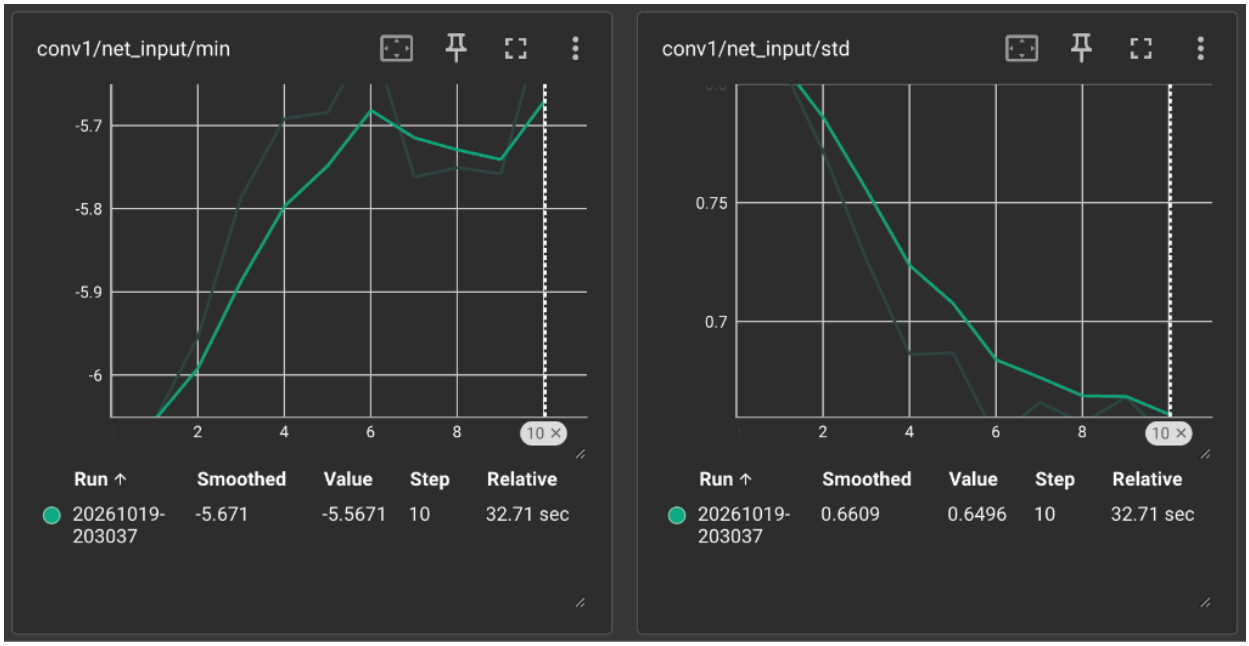

fc2/weight/histogram
20261019-203037
2
4
6
8
-0.3
-0.2
-0.1
0
0.1
0.2

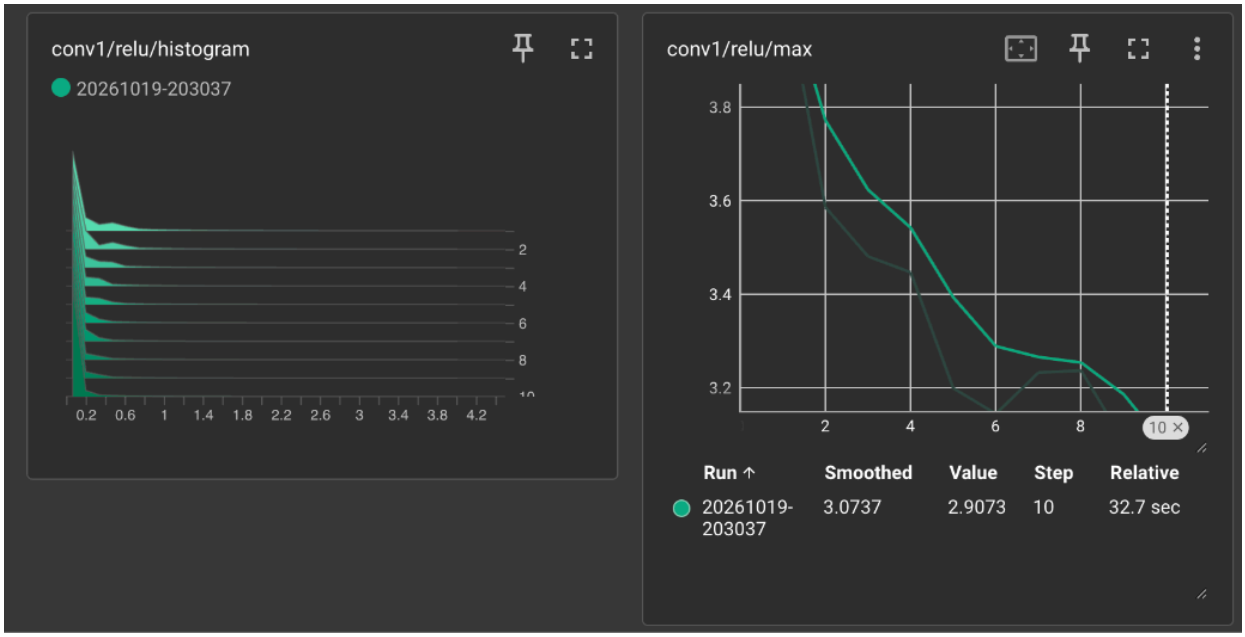

fc2/weight/max
0.2
0.18
0.16
2
4
6
8
10
Run
Smoothed
Value
Step
Relative
20261019-203037
0.2264
0.2427
10
32.76 sec



First Convolution: Net Input and ReLU Activation

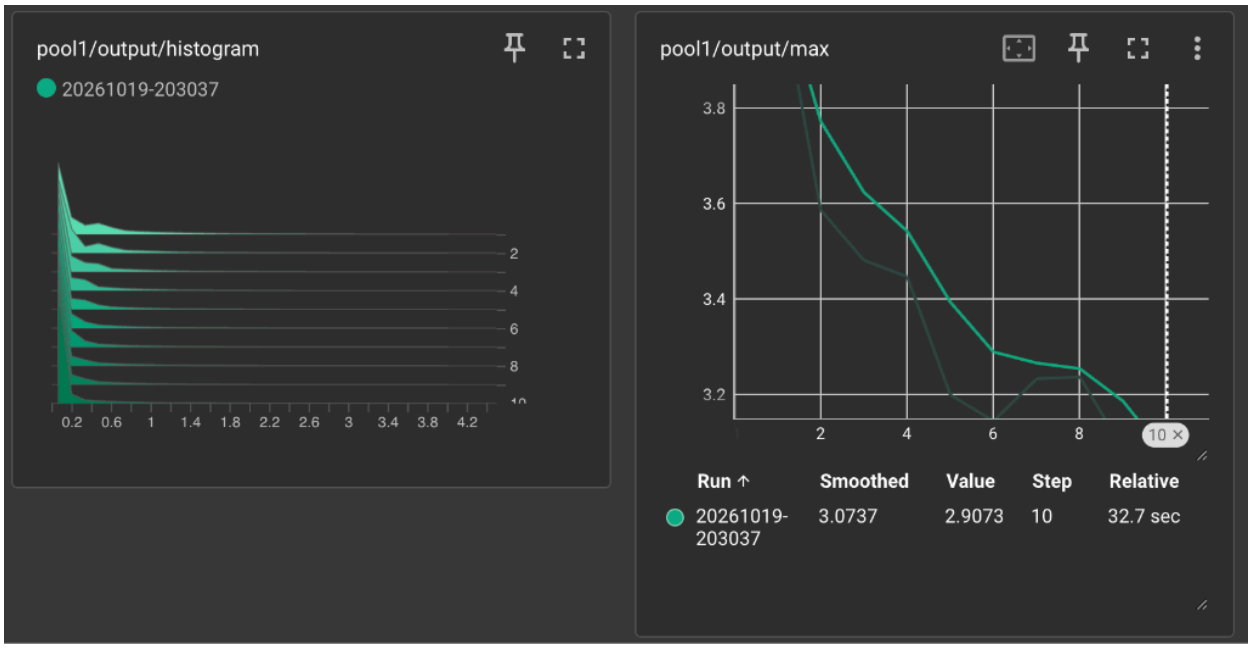


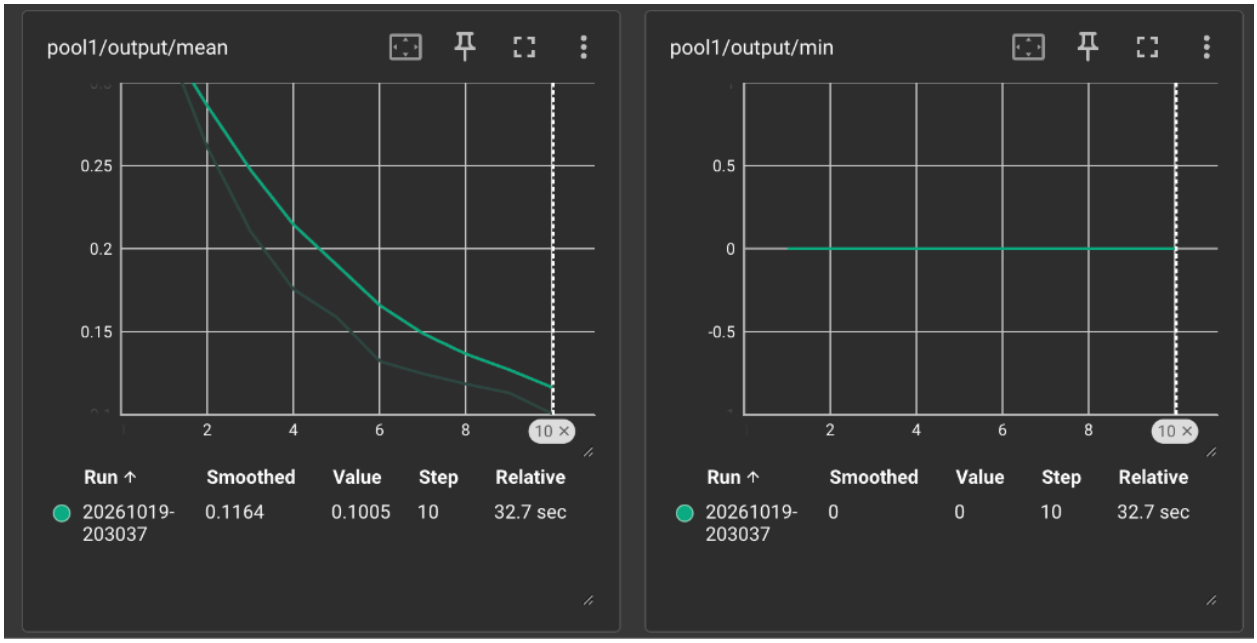

conv1/net_input/min
-5.7
-5.8
-5.9
-6
2
4
6
8
10 ×
Run ↑ Smoothed Value Step Relative
20261019-203037 -5.671 -5.5671 10 32.71 sec
conv1/net_input/std
0.75
0.7
2
4
6
8
10 ×
Run ↑ Smoothed Value Step Relative
20261019-203037 0.6609 0.6496 10 32.71 sec

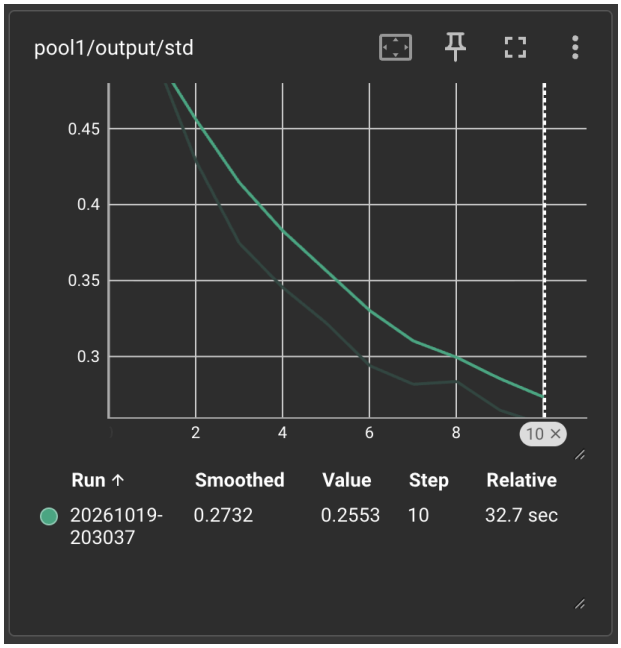

conv1/relu/histogram
20261019-203037
2
4
6
8
0.2 0.6 1 1.4 1.8 2.2 2.6 3 3.4 3.8 4.2
conv1/relu/max
3.8
3.6
3.4
3.2
2
4
6
8
10 ×
Run ↑ Smoothed Value Step Relative
20261019-203037 3.0737 2.9073 10 32.7 sec


conv1/relu/mean
0.15
0.1
2
4
6
8
10 ×
Run ↑ Smoothed Value Step Relative
20261019-203037 0.0645 0.054 10 32.7 sec
conv1/relu/min
0.5
0
-0.5
2
4
6
8
10 ×
Run ↑ Smoothed Value Step Relative
20261019-203037 0 0 10 32.7 sec
conv1/relu/std
0.35
0.3
0.25
0.2
2
4
6
8
10 ×
Run ↑ Smoothed Value Step Relative
20261019-203037 0.1898 0.1748 10 32.7 sec

First Max-Pooling Output

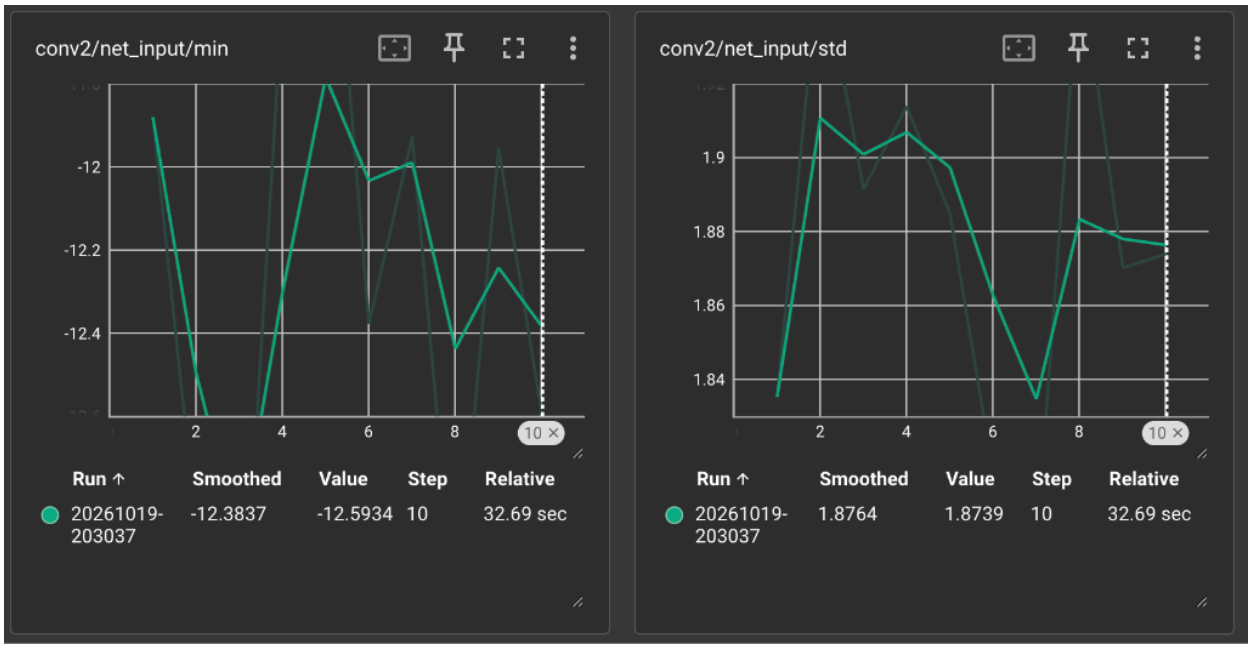


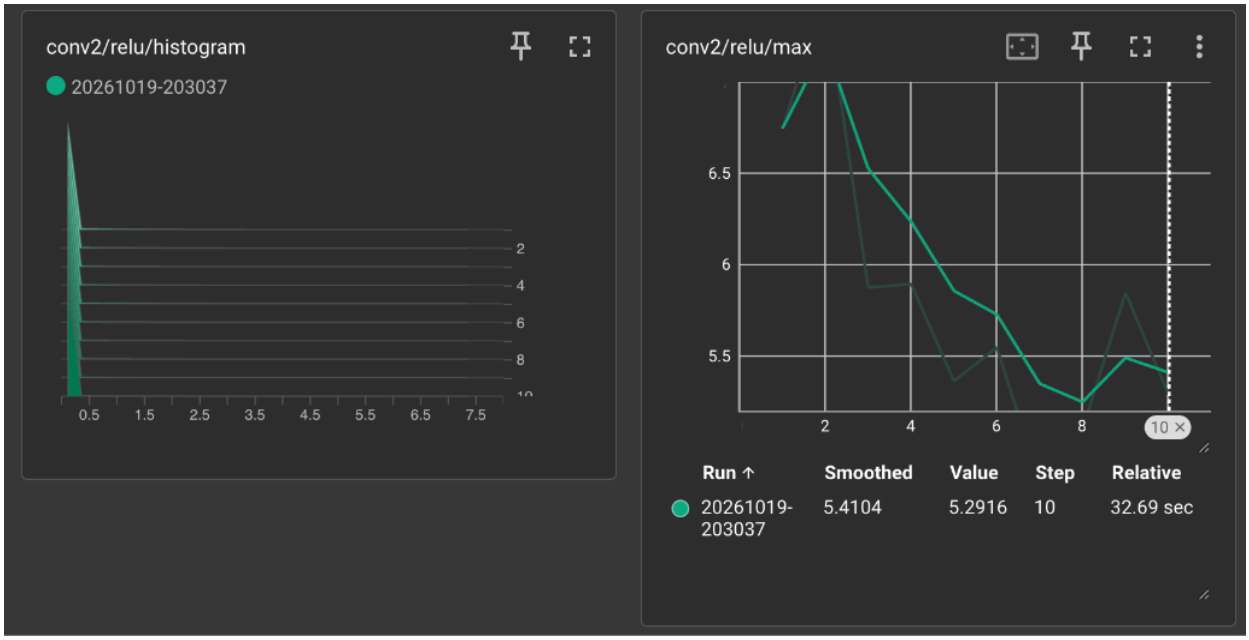


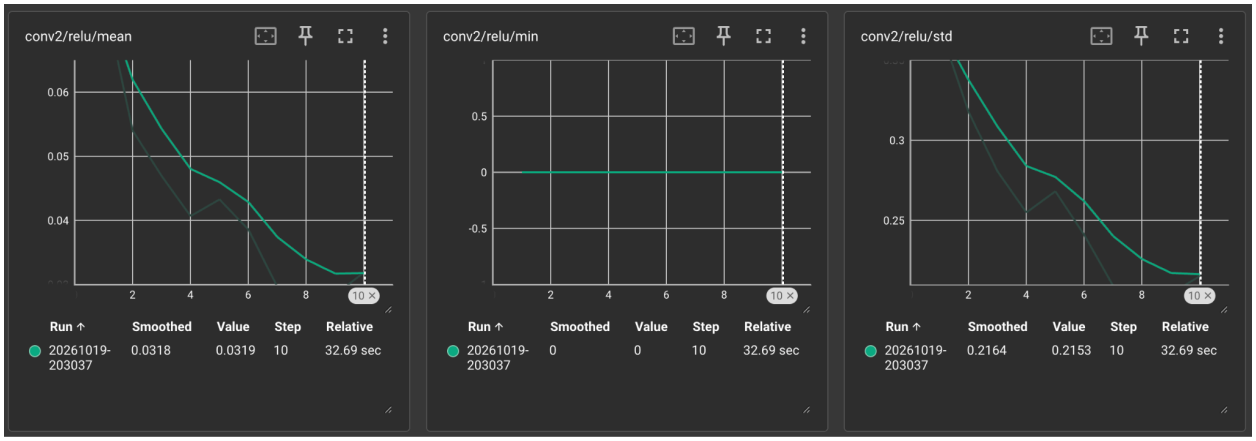

pool1/output/std
0.45
0.4
0.35
0.3
2
4
6
8
10
Run
Smoothed
Value
Step
Relative
20261019-203037
0.2732
0.2553
10
32.7 sec

Second Convolution: Net Input and ReLU Activation

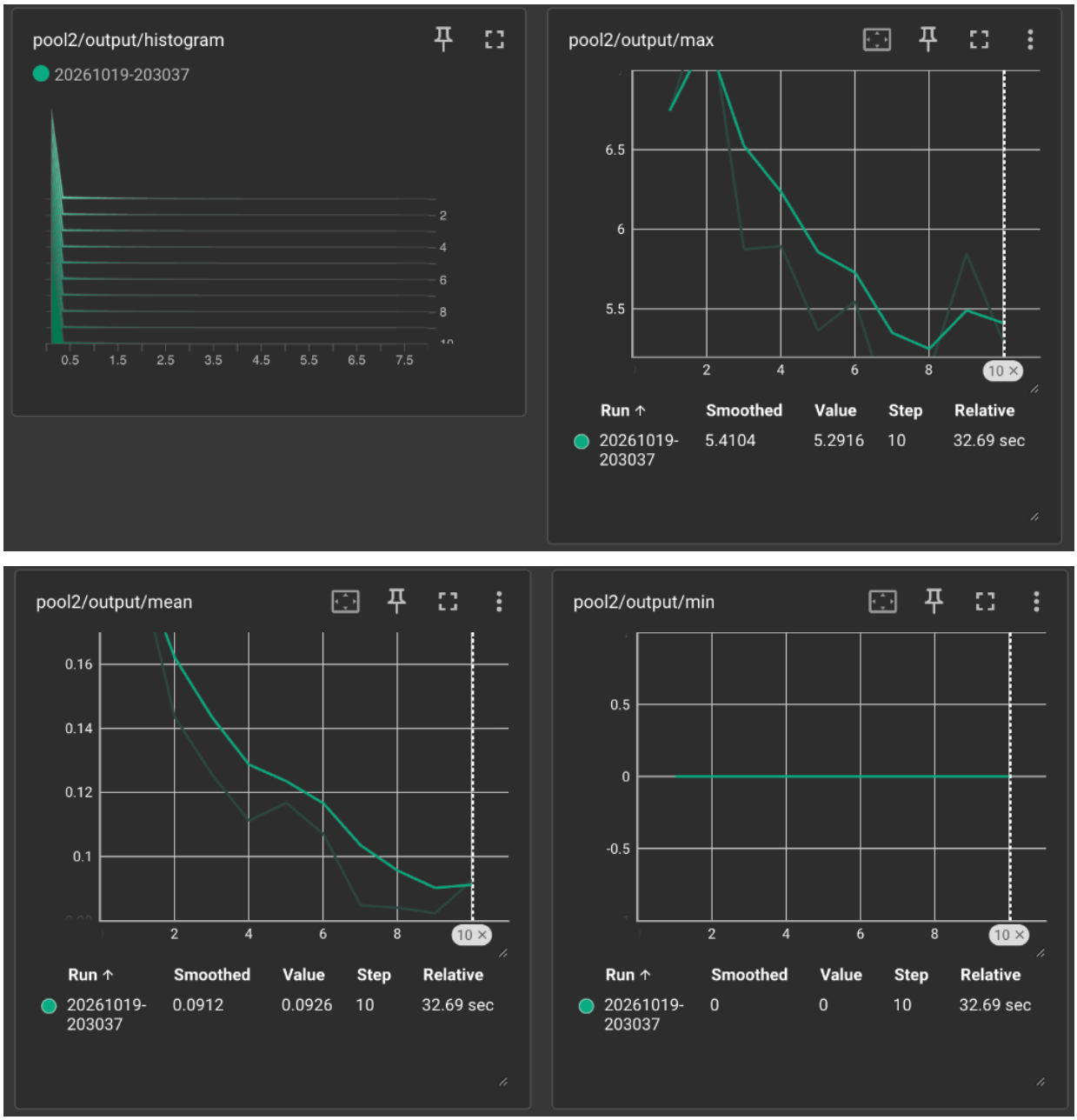


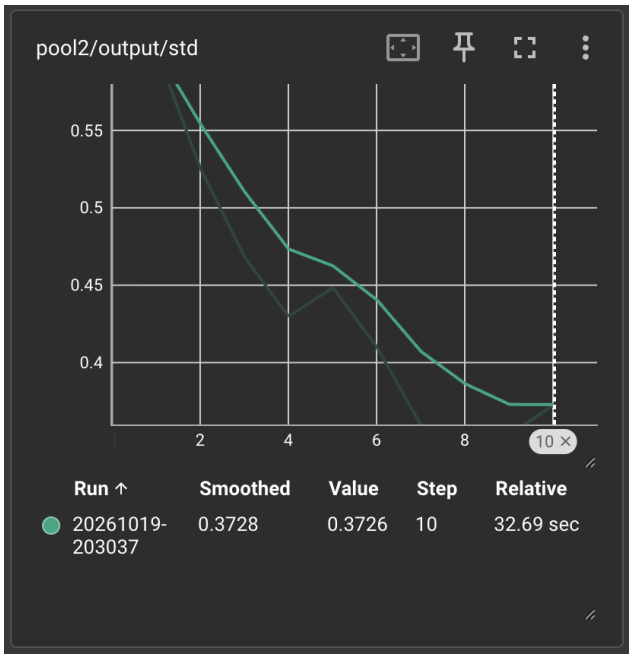



conv2/relu/histogram
20261019-203037
0.5
1.5
2.5
3.5
4.5
5.5
6.5
7.5
2
4
6
8
conv2/relu/max
6.5
6
5.5
2
4
6
8
10 ×
Run ↑
Smoothed
Value
Step
Relative
20261019-203037
5.4104
5.2916
10
32.69 sec


conv2/relu/mean
0.06
0.05
0.04
2
4
6
8
10 ×
Run ↑
Smoothed
Value
Step
Relative
20261019-203037
0.0318
0.0319
10
32.69 sec
conv2/relu/min
0.5
0
-0.5
2
4
6
8
10 ×
Run ↑
Smoothed
Value
Step
Relative
20261019-203037
0
0
10
32.69 sec
conv2/relu/std
0.3
0.25
2
4
6
8
10 ×
Run ↑
Smoothed
Value
Step
Relative
20261019-203037
0.2164
0.2153
10
32.69 sec

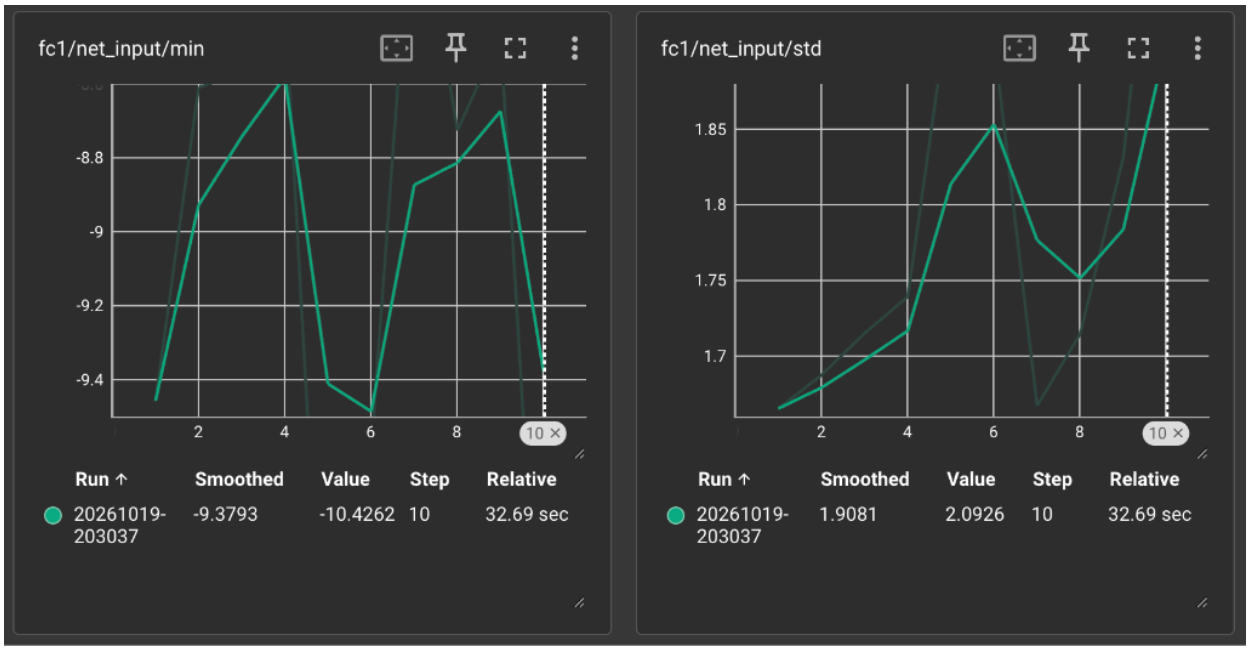
Second Max-Pooling Output

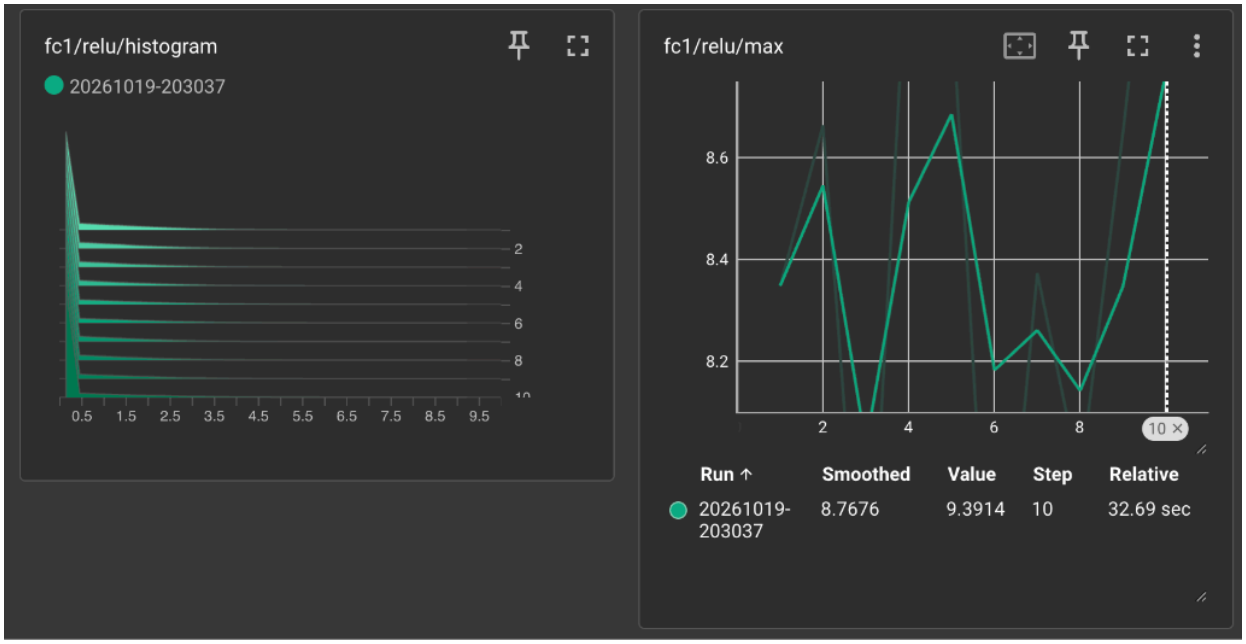



pool2/output/std
0.55
0.5
0.45
0.4
2
4
6
8
10
Run
Smoothed
Value
Step
Relative
20261019-203037
0.3728
0.3726
10
32.69 sec

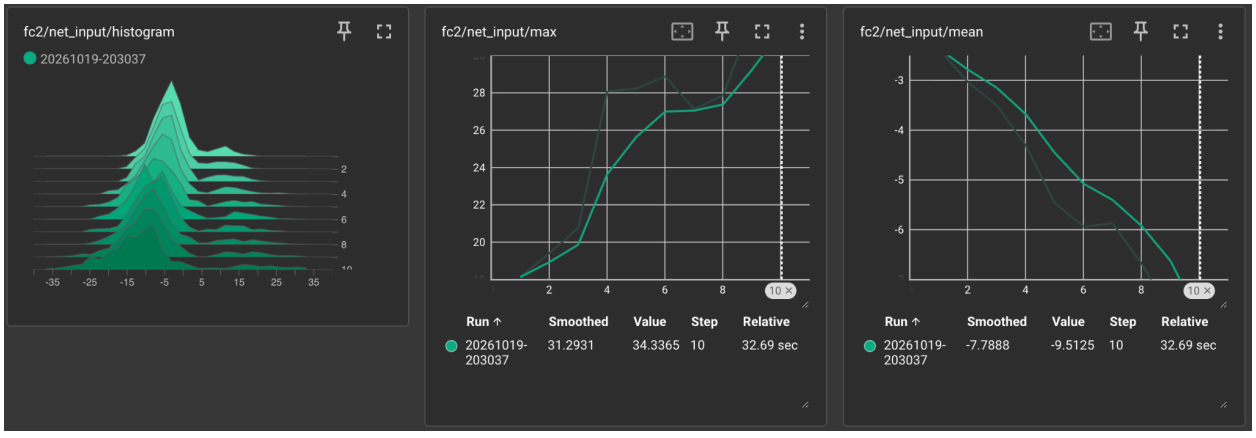
Fully Connected Layers: Net Inputs and ReLU Activation






fc1/relu/histogram
20261019-203037
0.5 1.5 2.5 3.5 4.5 5.5 6.5 7.5 8.5 9.5
2
4
6
8
fc1/relu/max
8.6
8.4
8.2
2 4 6 8 10
Run ↑ Smoothed Value Step Relative
20261019-203037 8.7676 9.3914 10 32.69 sec


fc1/relu/mean
0.42
0.4
0.38
0.36
0.34
2 4 6 8 10
Run ↑ Smoothed Value Step Relative
20261019-203037 0.3421 0.3597 10 32.69 sec
fc1/relu/min
0.5
0
-0.5
2 4 6 8 10
Run ↑ Smoothed Value Step Relative
20261019-203037 0 0 10 32.69 sec
fc1/relu/std
0.86
0.84
0.82
2 4 6 8 10
Run ↑ Smoothed Value Step Relative
20261019-203037 0.8516 0.9105 10 32.69 sec


fc2/net_input/histogram
20261019-203037
-35 -25 -15 -5 5 15 25 35
2
4
6
8
fc2/net_input/max
28
26
24
22
20
2 4 6 8 10
Run ↑ Smoothed Value Step Relative
20261019-203037 31.2931 34.3365 10 32.69 sec
fc2/net_input/mean
-3
-4
-5
-6
2 4 6 8 10
Run ↑ Smoothed Value Step Relative
20261019-203037 -7.7888 -9.5125 10 32.69 sec

Observations

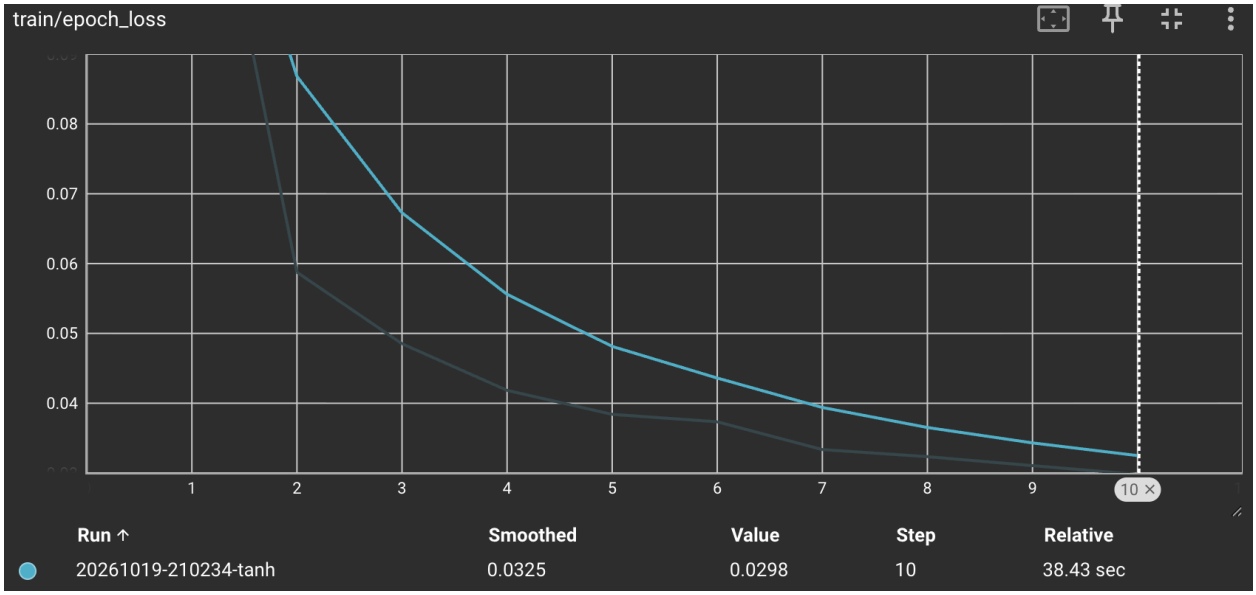
During the 10 epoch run, test error fell from approximately 1.13% after the first epoch to 0.74% after the final epoch, although it fluctuated between epochs. The TensorBoard plots show that the learned weights and biases changed across all four parameterized layers: the first convolutional layer's weight spread narrowed slightly, while the spreads in the second convolutional and fully connected layers generally increased. Net inputs included both positive and negative values, whereas outputs after ReLU were non-negative, as expected. Max-pooling selected stronger local responses, giving its outputs higher mean values than the preceding ReLU outputs. The final fully connected layer's net inputs spread further apart during training, alongside the decrease in test error. These plots describe how the network's internal values evolved during this run; they do not, on their own, establish that any one layer change caused the improvement.

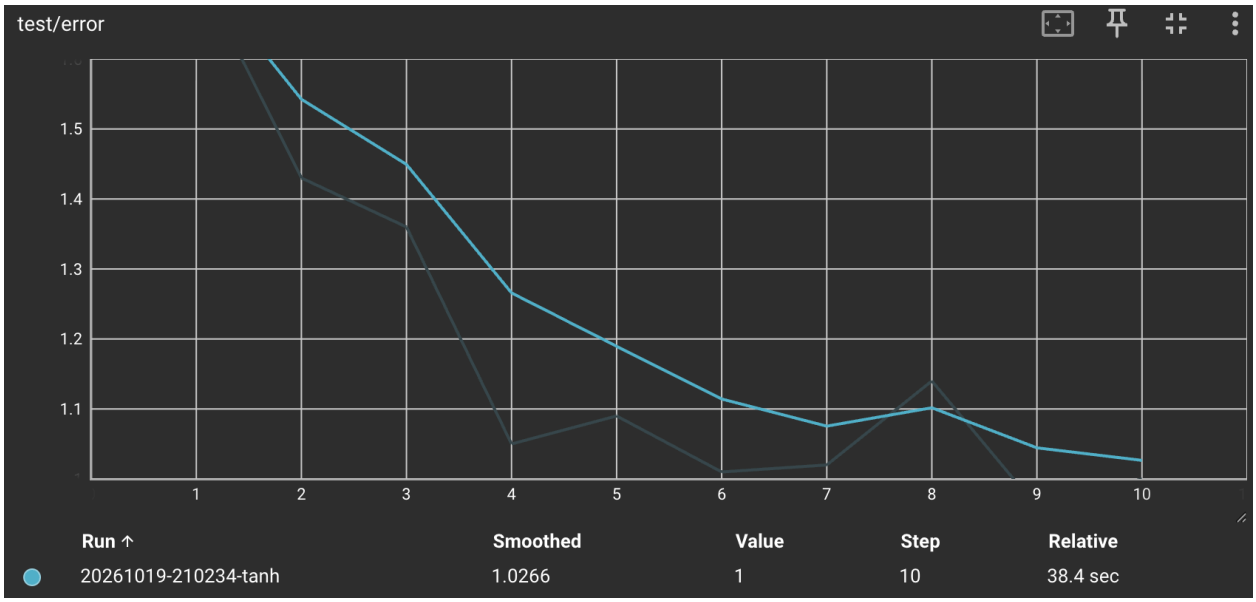
c) Time For More Fun!!!

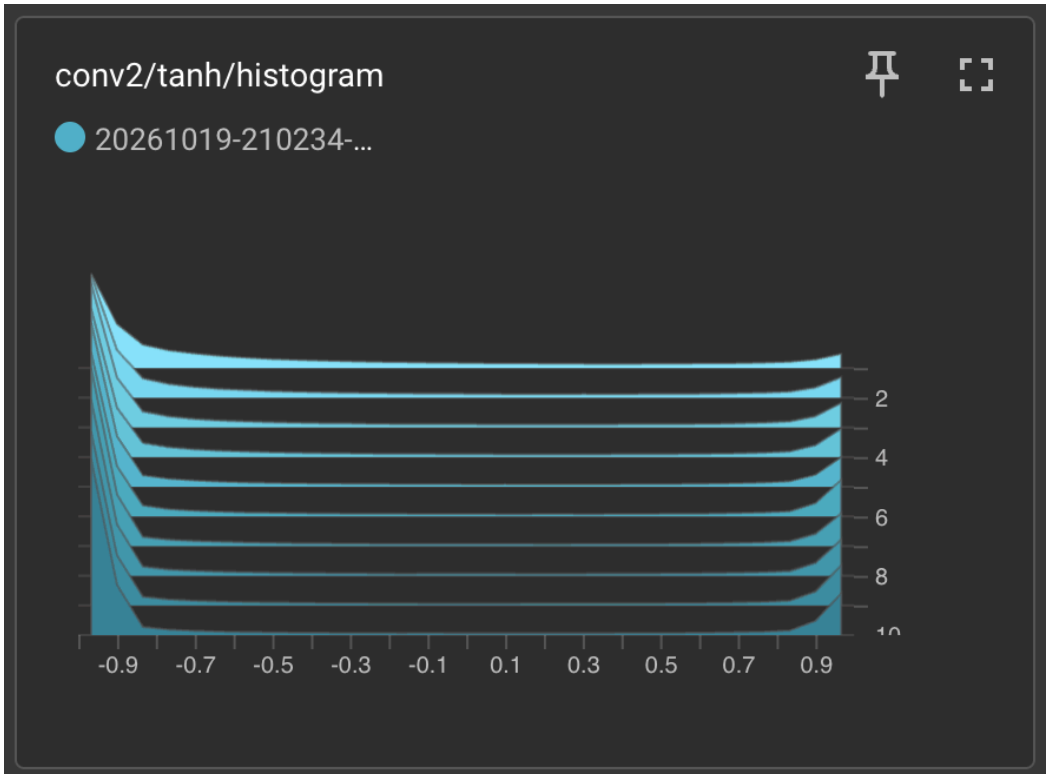
For this section, I ran my CNN for each of the suggestions given in the question itself. All results can be seen below. For each run, I got the graphs for epoch loss during training, as well as the error form testing.

Non-Linearities

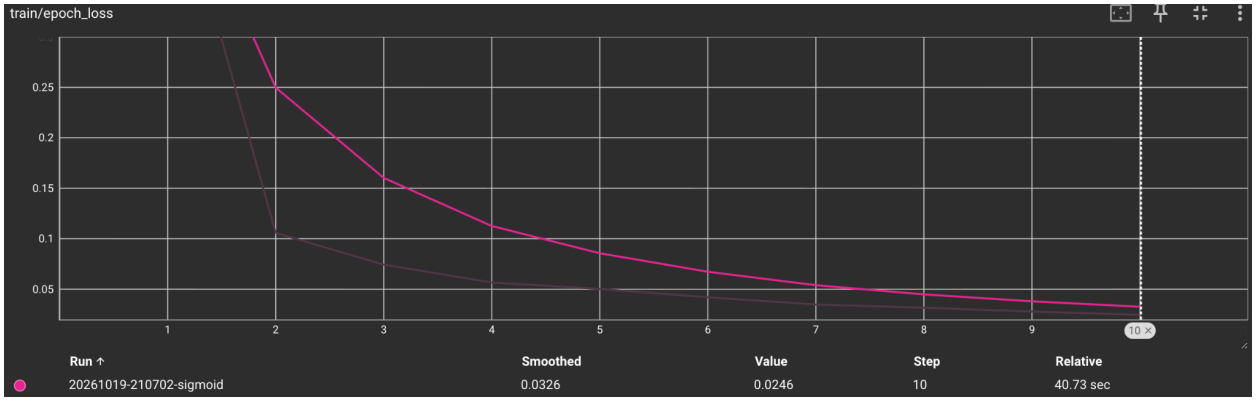
tanh

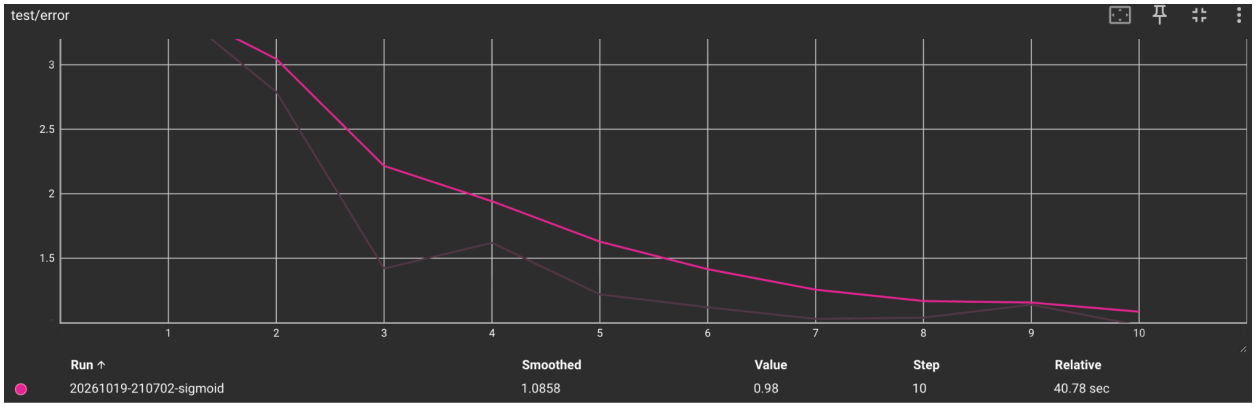


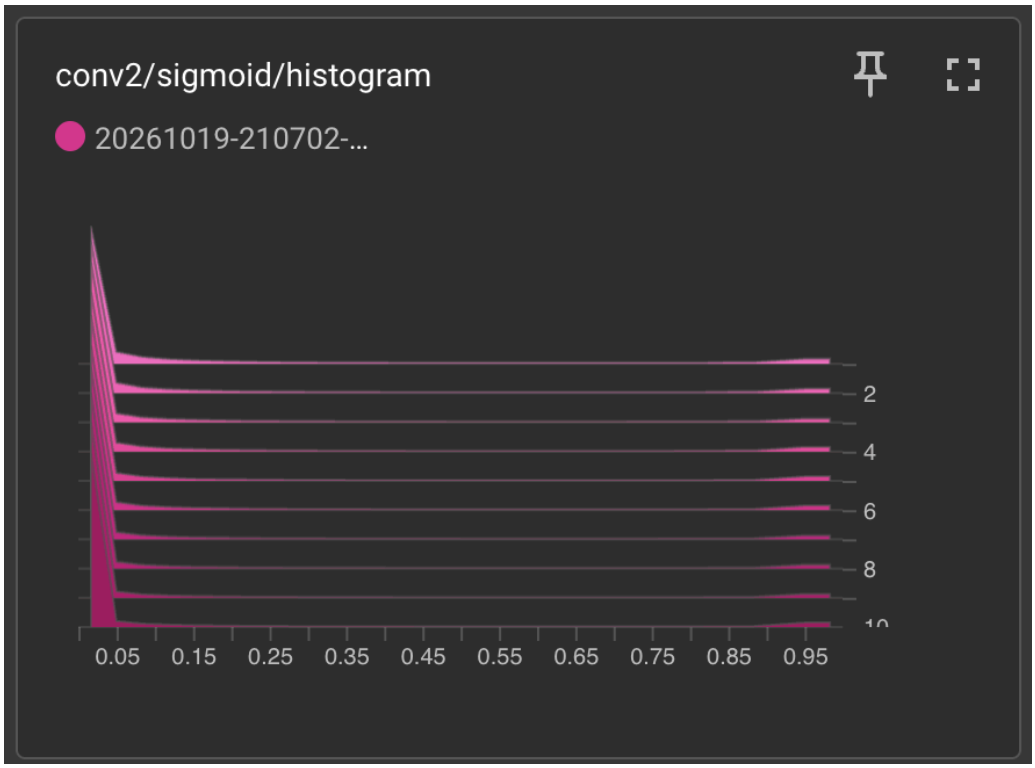



conv2/tanh/histogram
20261019-210234-...
2
4
6
8
-0.9
-0.7
-0.5
-0.3
-0.1
0.1
0.3
0.5
0.7
0.9

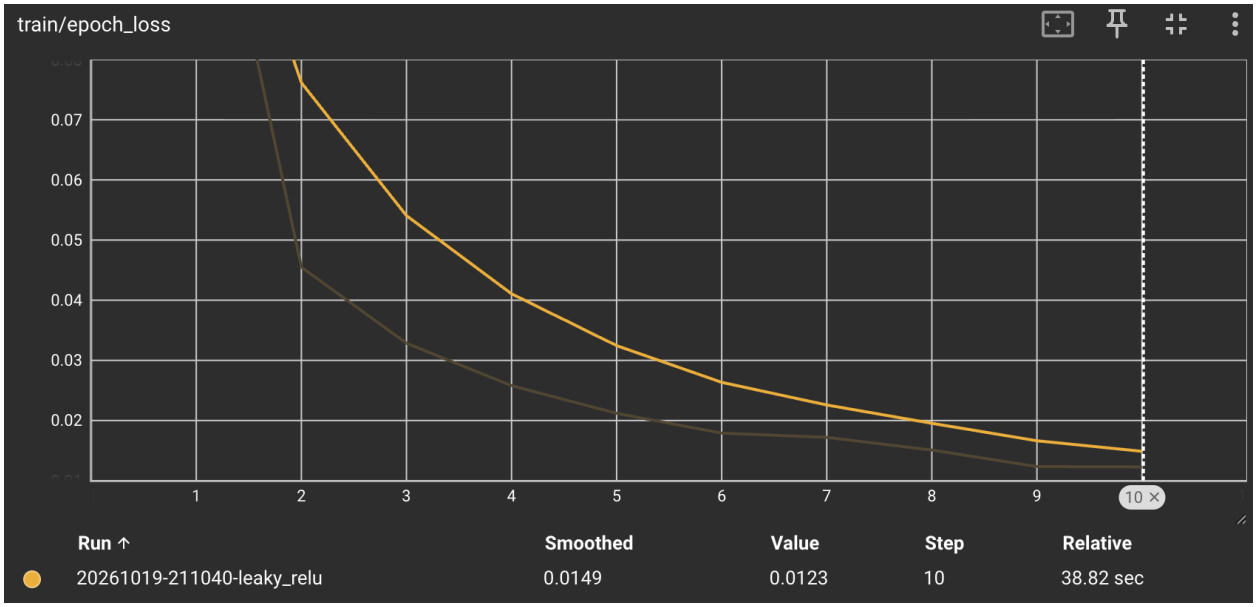
sigmoid

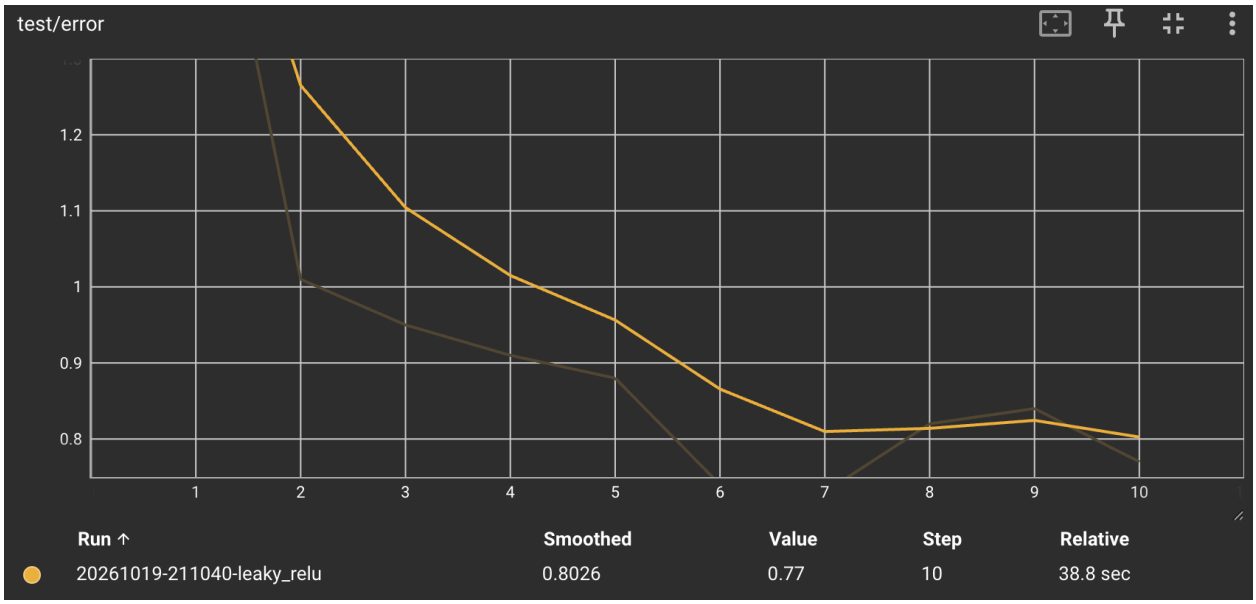

train/epoch_loss
0.25
0.2
0.15
0.1
0.05
1
2
3
4
5
6
7
8
9
10 ×
Run ↑
Smoothed
Value
Step
Relative
20261019-210702-sigmoid
0.0326
0.0246
10
40.73 sec

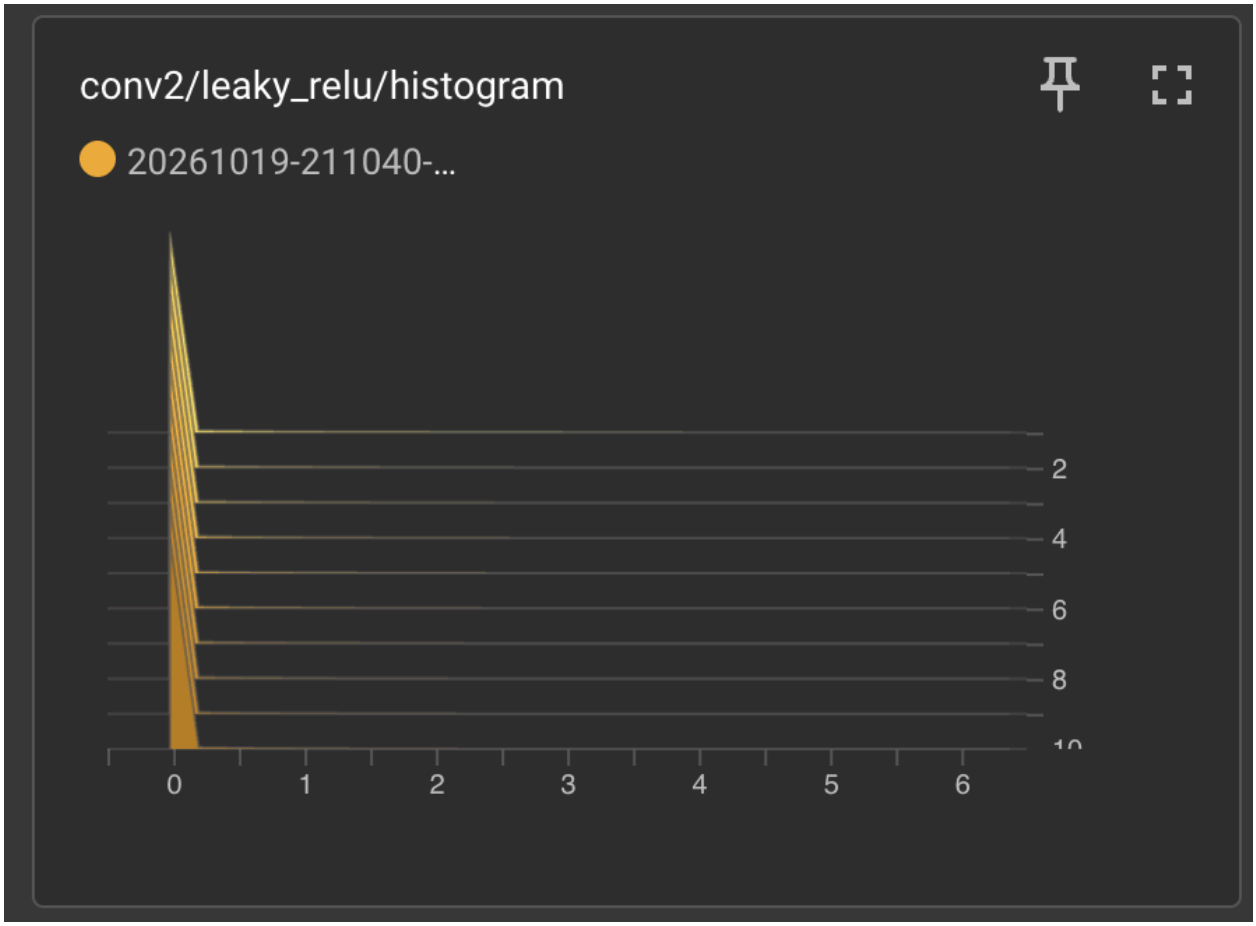

test/error
3
2.5
2
1.5
1
2
3
4
5
6
7
8
9
10
Run ↑
Smoothed
Value
Step
Relative
20261019-210702-sigmoid
1.0858
0.98
10
40.78 sec


conv2/sigmoid/histogram
20261019-210702-...
2
4
6
8
0.05
0.15
0.25
0.35
0.45
0.55
0.65
0.75
0.85
0.95

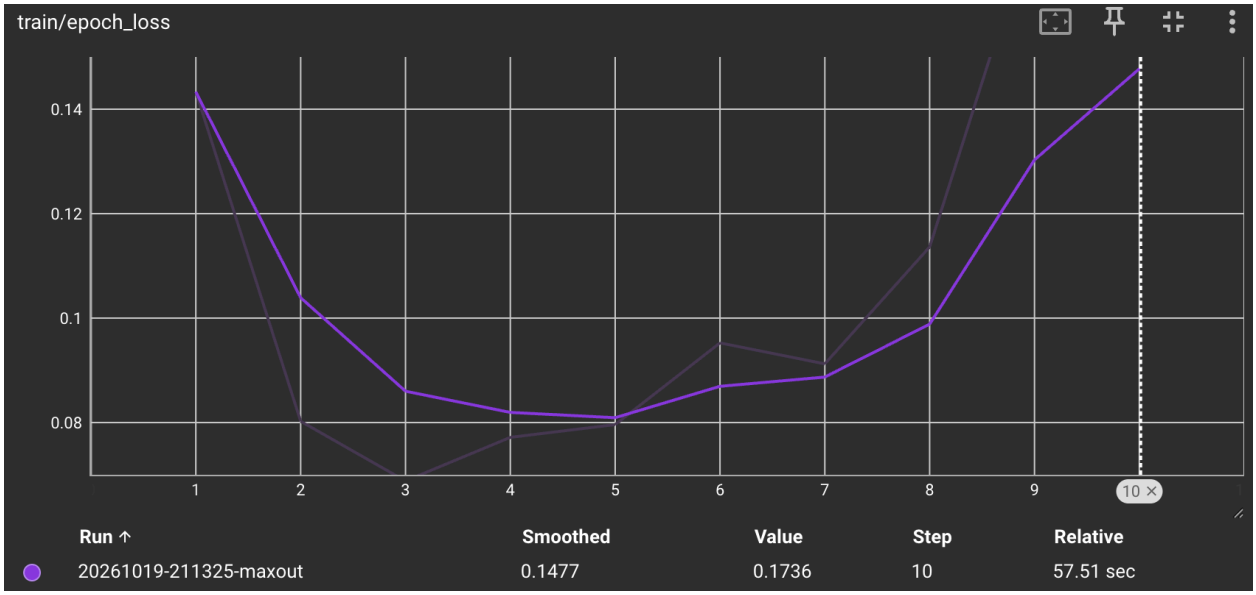
Leaky-ReLU

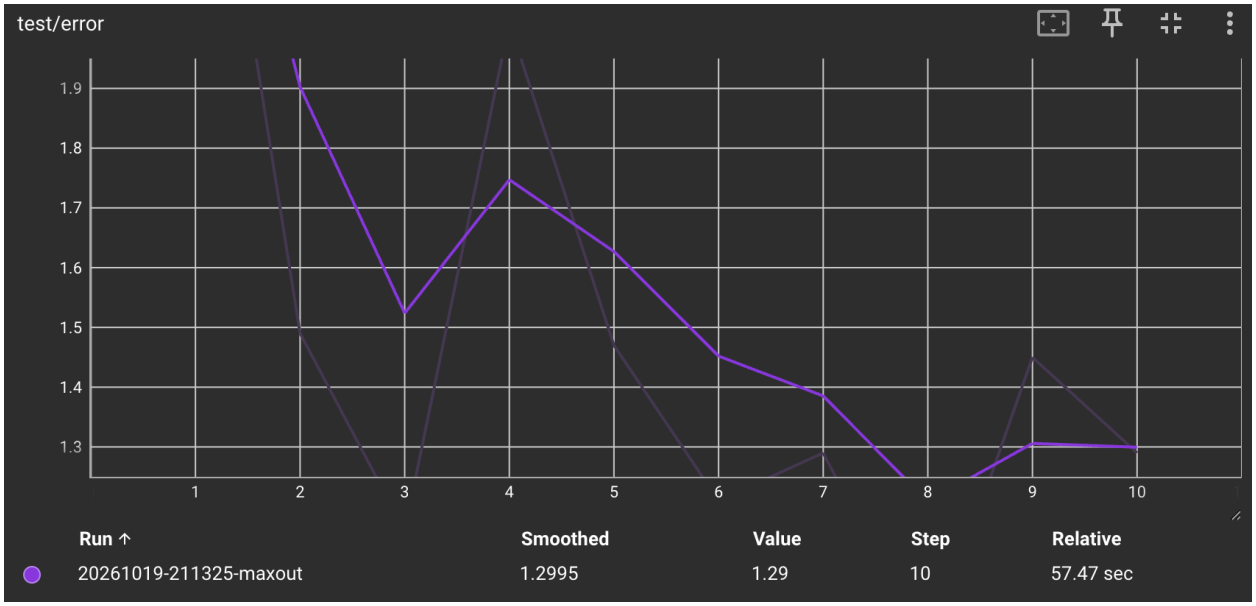

train/epoch_loss
0.07
0.06
0.05
0.04
0.03
0.02
1
2
3
4
5
6
7
8
9
10 ×
Run ↑
Smoothed
Value
Step
Relative
20261019-211040-leaky_relu
0.0149
0.0123
10
38.82 sec

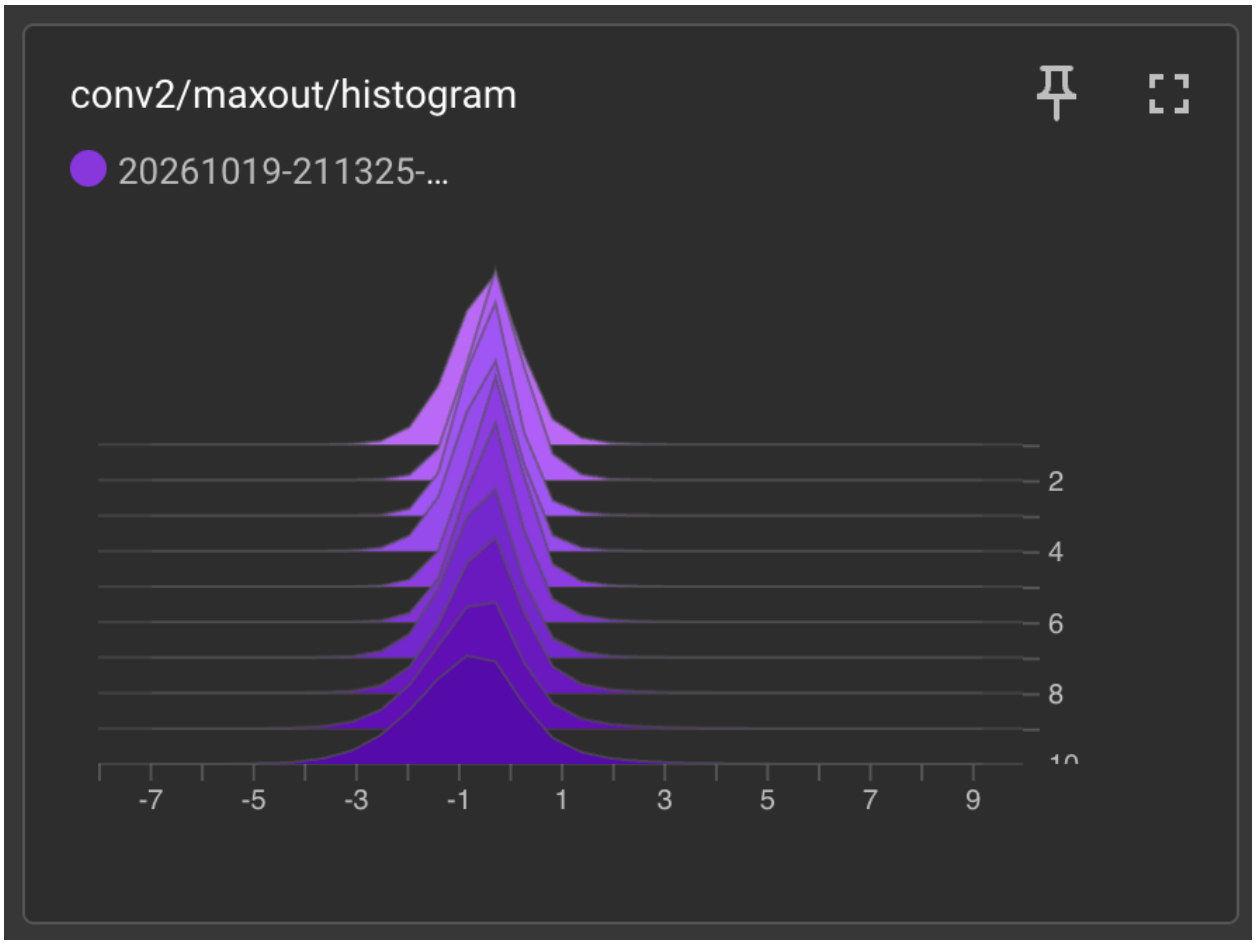

test/error
1.2
1.1
1
0.9
0.8
1
2
3
4
5
6
7
8
9
10
Run ↑
Smoothed
Value
Step
Relative
20261019-211040-leaky_relu
0.8026
0.77
10
38.8 sec


conv2/leaky_relu/histogram
20261019-211040-...
2
4
6
8
10
0
1
2
3
4
5
6

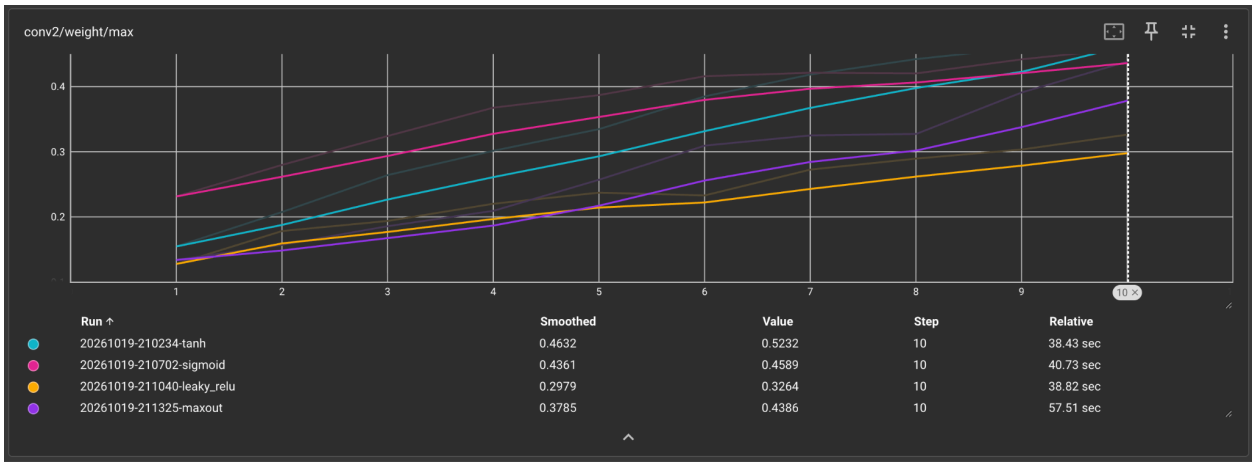
MaxOut


train/epoch_loss
0.14
0.12
0.1
0.08
1
2
3
4
5
6
7
8
9
10
Run
Smoothed
Value
Step
Relative
20261019-211325-maxout
0.1477
0.1736
10
57.51 sec

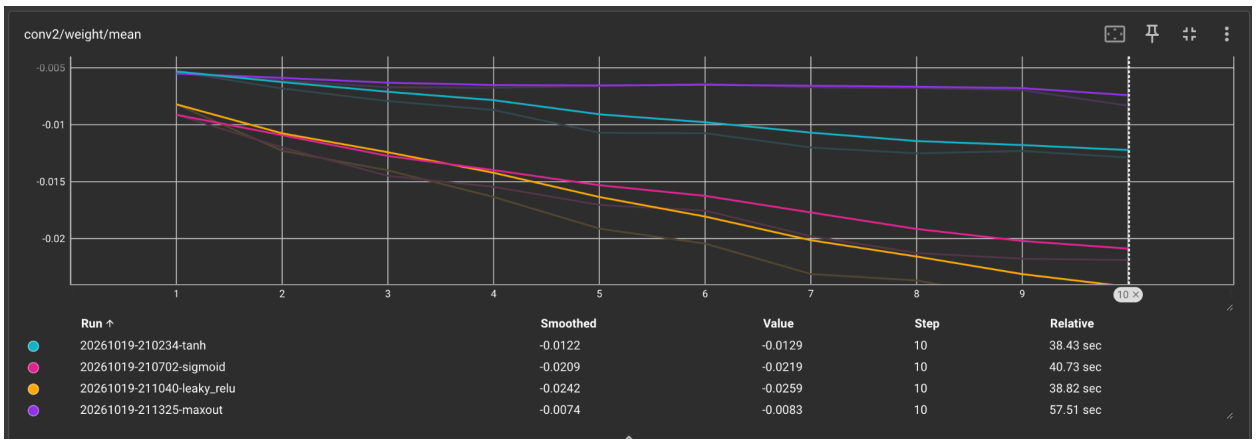

test/error
1.9
1.8
1.7
1.6
1.5
1.4
1.3
1
2
3
4
5
6
7
8
9
10
Run ↑
Smoothed
Value
Step
Relative
20261019-211325-maxout
1.2995
1.29
10
57.47 sec


conv2/maxout/histogram
20261019-211325-...
2
4
6
8
10
-7
-5
-3
-1
1
3
5
7
9

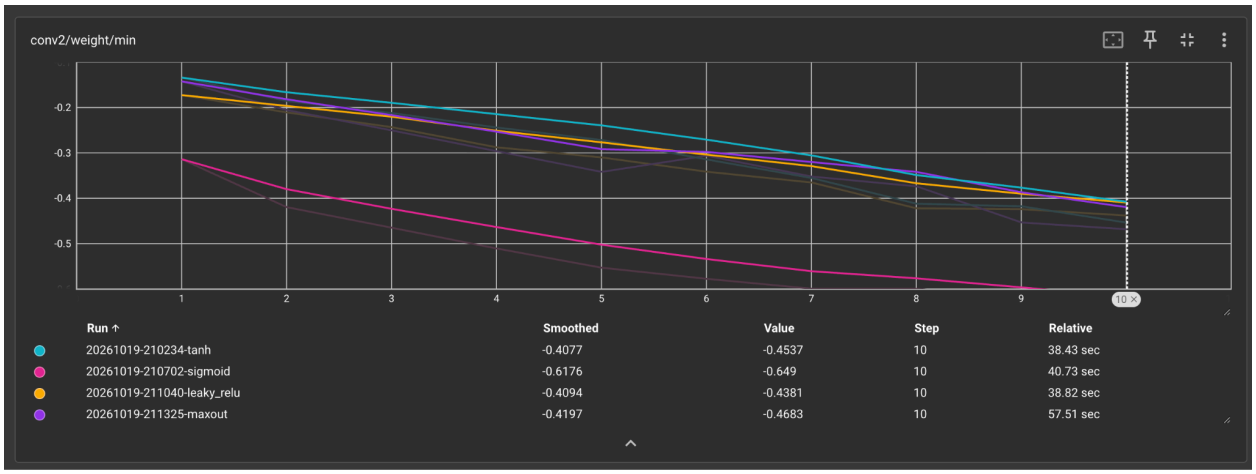
Comparing the Activation Functions



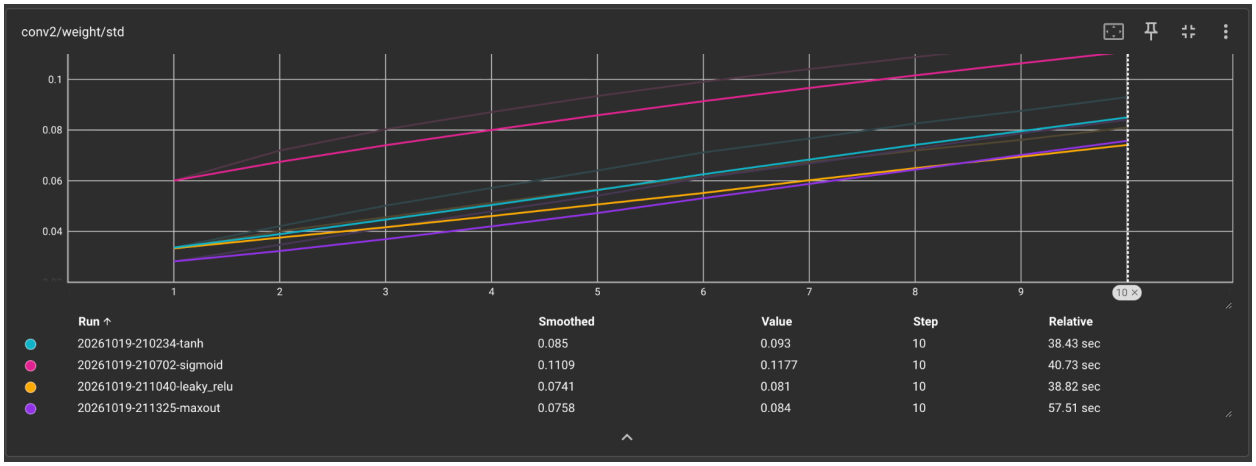
Run	Smoothed	Value	Step	Relative
20261019-210234-tanh	0.4632	0.5232	10	38.43 sec
20261019-210702-sigmoid	0.4361	0.4589	10	40.73 sec
20261019-211040-leaky_relu	0.2979	0.3264	10	38.82 sec
20261019-211325-maxout	0.3785	0.4386	10	57.51 sec

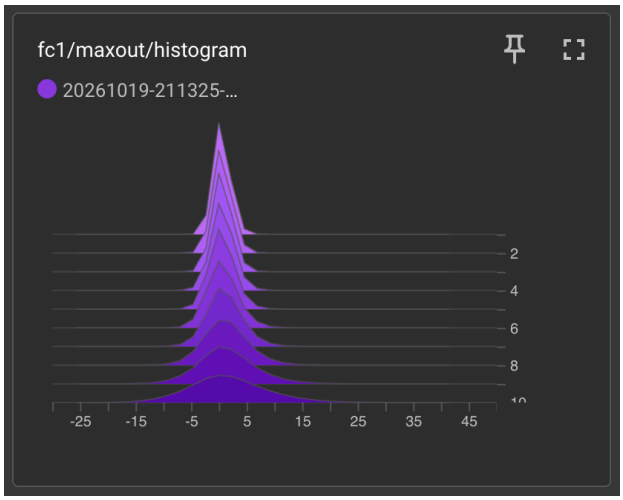


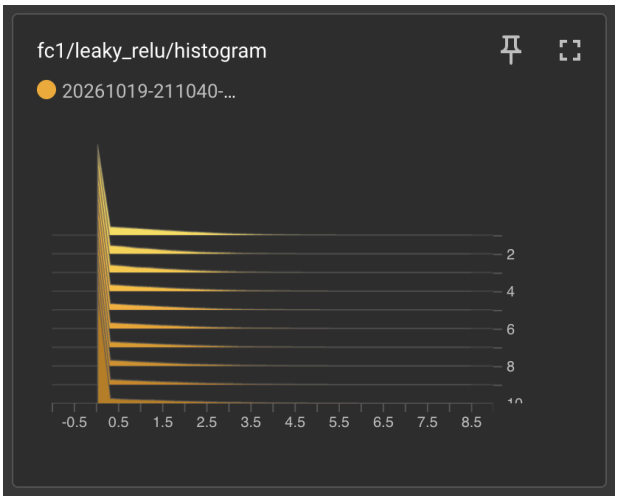
Run	Smoothed	Value	Step	Relative
20261019-210234-tanh	-0.0122	-0.0129	10	38.43 sec
20261019-210702-sigmoid	-0.0209	-0.0219	10	40.73 sec
20261019-211040-leaky_relu	-0.0242	-0.0259	10	38.82 sec
20261019-211325-maxout	-0.0074	-0.0083	10	57.51 sec

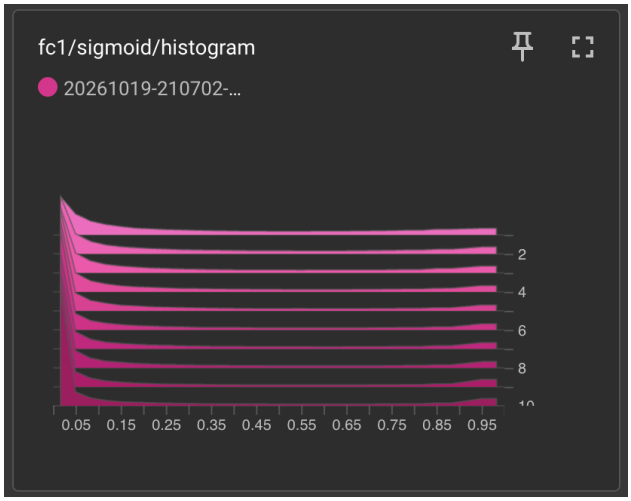


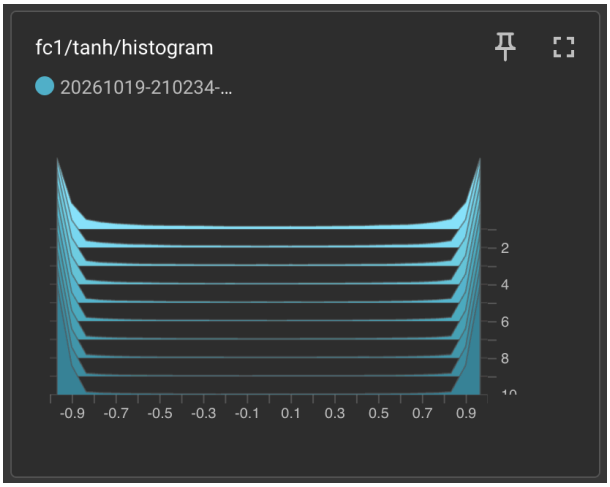
Run	Smoothed	Value	Step	Relative
20261019-210234-tanh	-0.4077	-0.4537	10	38.43 sec
20261019-210702-sigmoid	-0.6176	-0.649	10	40.73 sec
20261019-211040-leaky_relu	-0.4094	-0.4381	10	38.82 sec
20261019-211325-maxout	-0.4197	-0.4683	10	57.51 sec


conv2/weight/std
0.1
0.08
0.06
0.04
1
2
3
4
5
6
7
8
9
10
Run
Smoothed
Value
Step
Relative
20261019-210234-tanh 0.085 0.093 10 38.43 sec
20261019-210702-sigmoid 0.1109 0.1177 10 40.73 sec
20261019-211040-leaky_relu 0.0741 0.081 10 38.82 sec
20261019-211325-maxout 0.0758 0.084 10 57.51 sec


fc1/maxout/histogram
20261019-211325-...
2
4
6
8
10
-25
-15
-5
5
15
25
35
45


fc1/leaky_relu/histogram
20261019-211040-...
2
4
6
8
10
-0.5
0.5
1.5
2.5
3.5
4.5
5.5
6.5
7.5
8.5

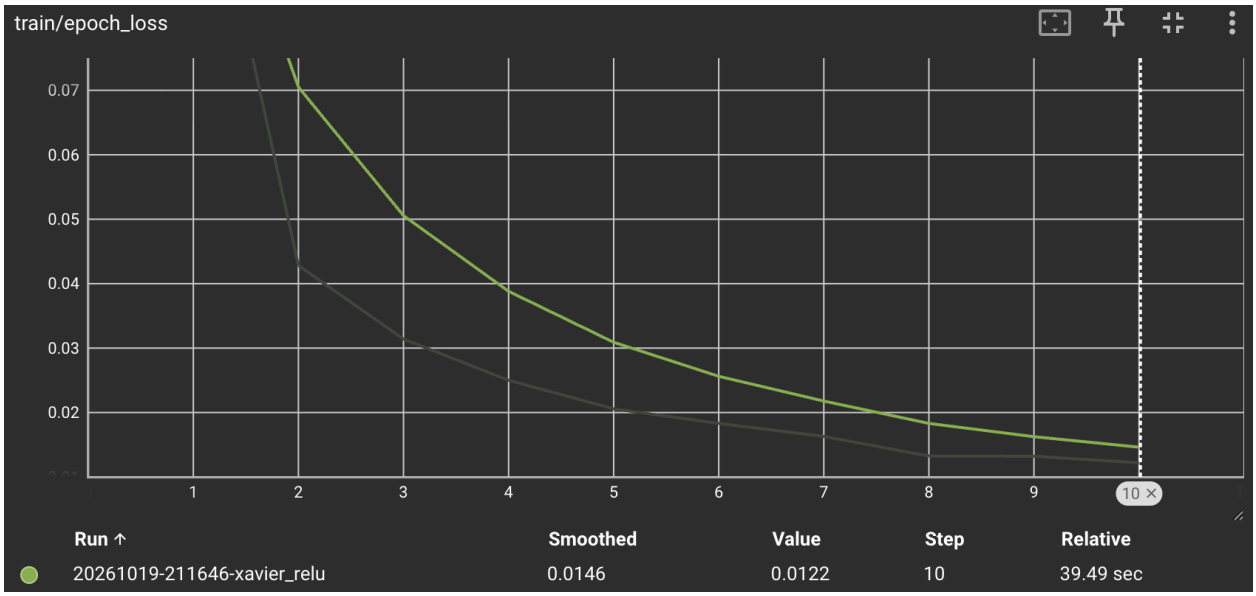

fc1/sigmoid/histogram
20261019-210702-...
2
4
6
8
10
0.05
0.15
0.25
0.35
0.45
0.55
0.65
0.75
0.85
0.95

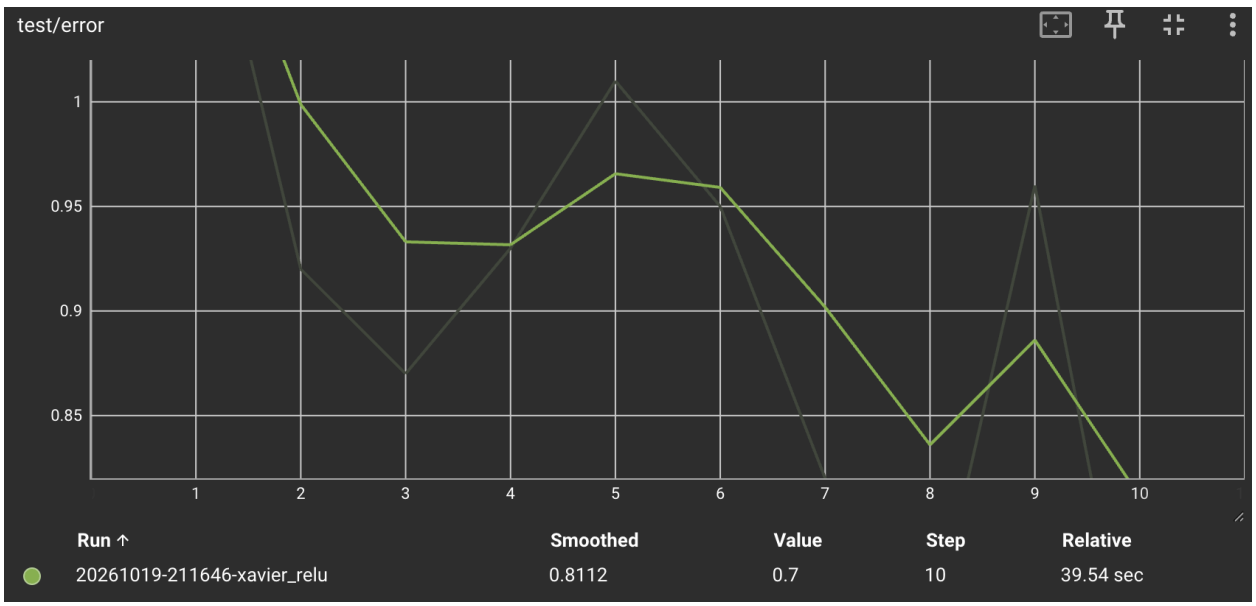

fc1/tanh/histogram
20261019-210234-...
2
4
6
8
10
-0.9
-0.7
-0.5
-0.3
-0.1
0.1
0.3
0.5
0.7
0.9

Initialization Technique

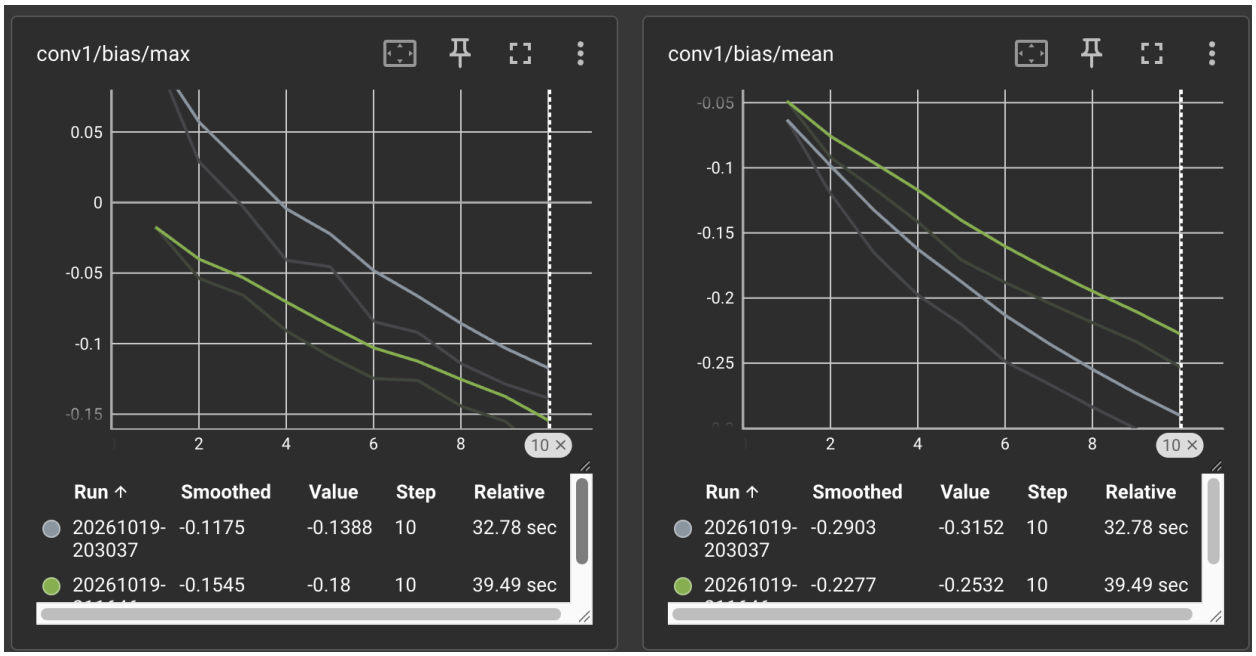
For the following runs, I reset the activation function back to ReLU.

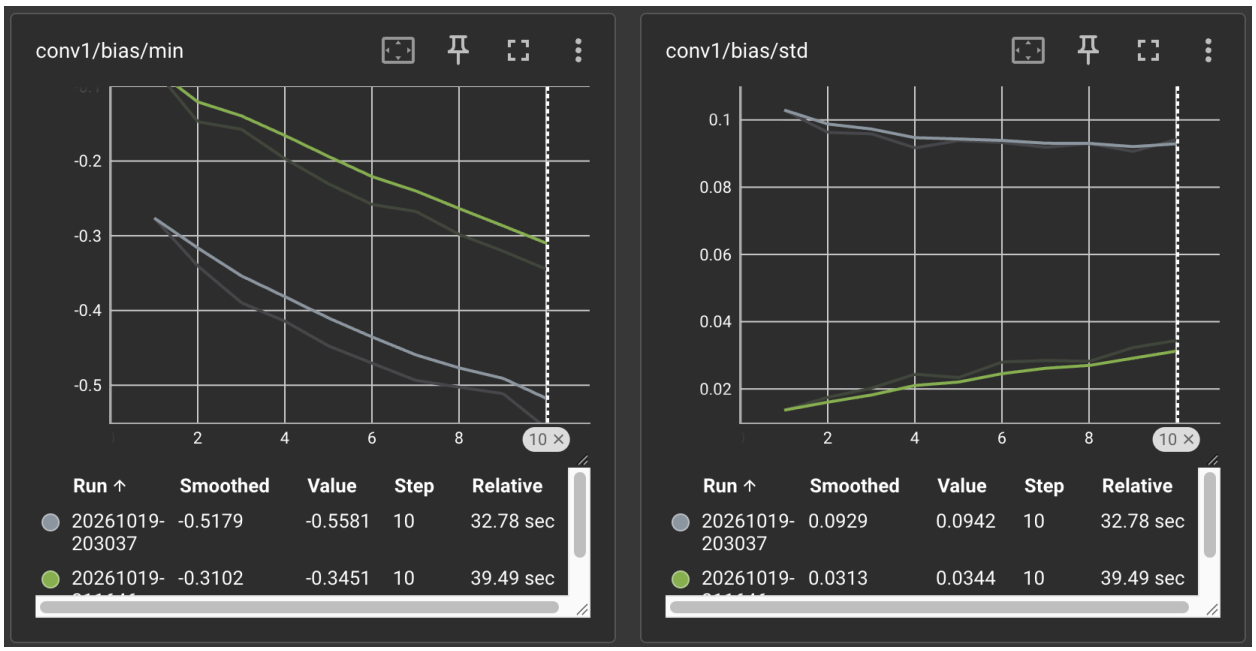
Xavier

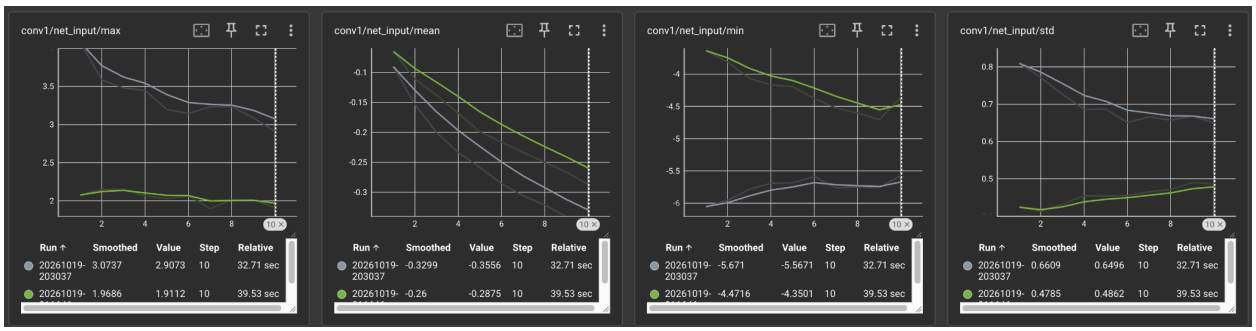


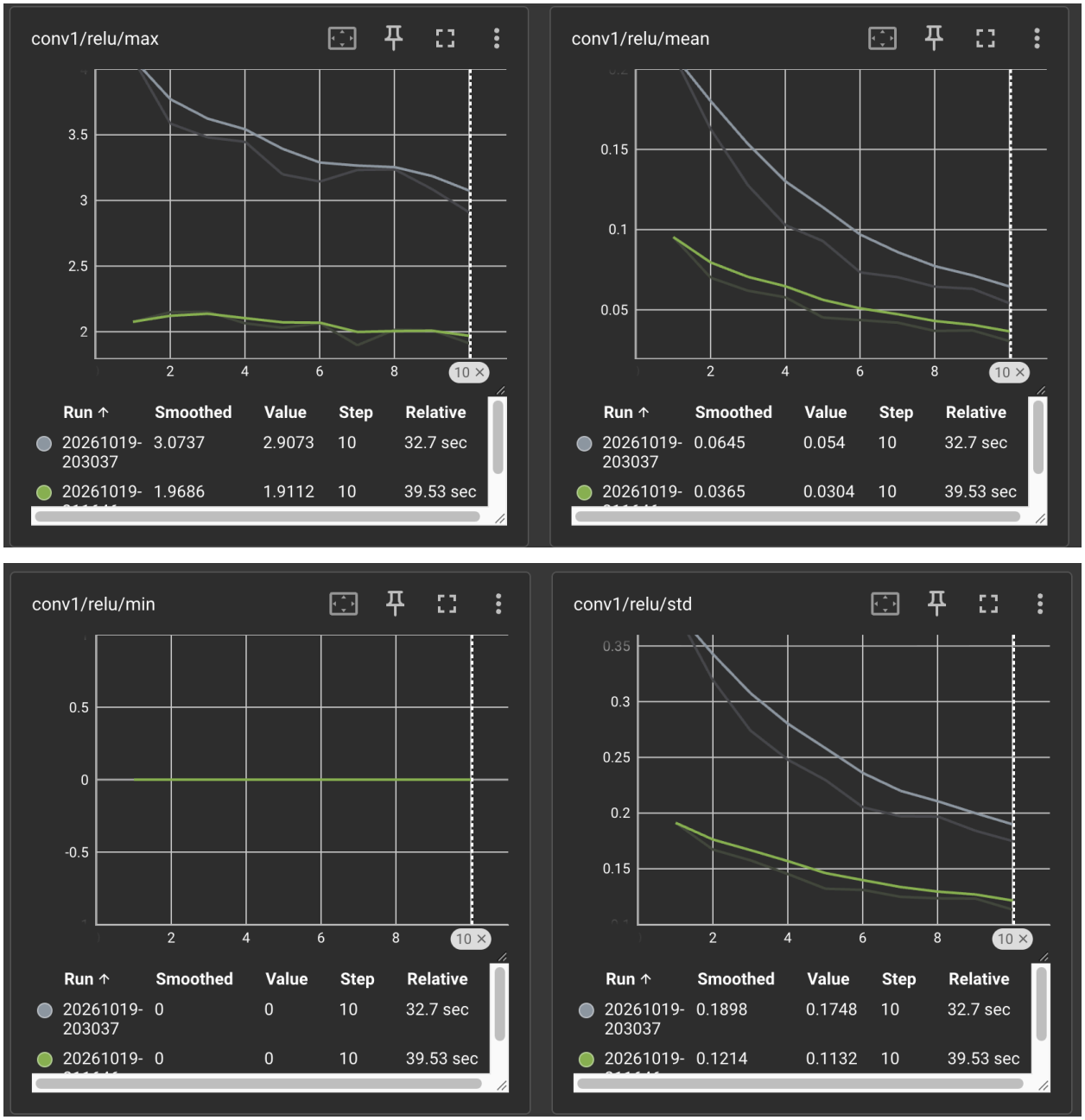


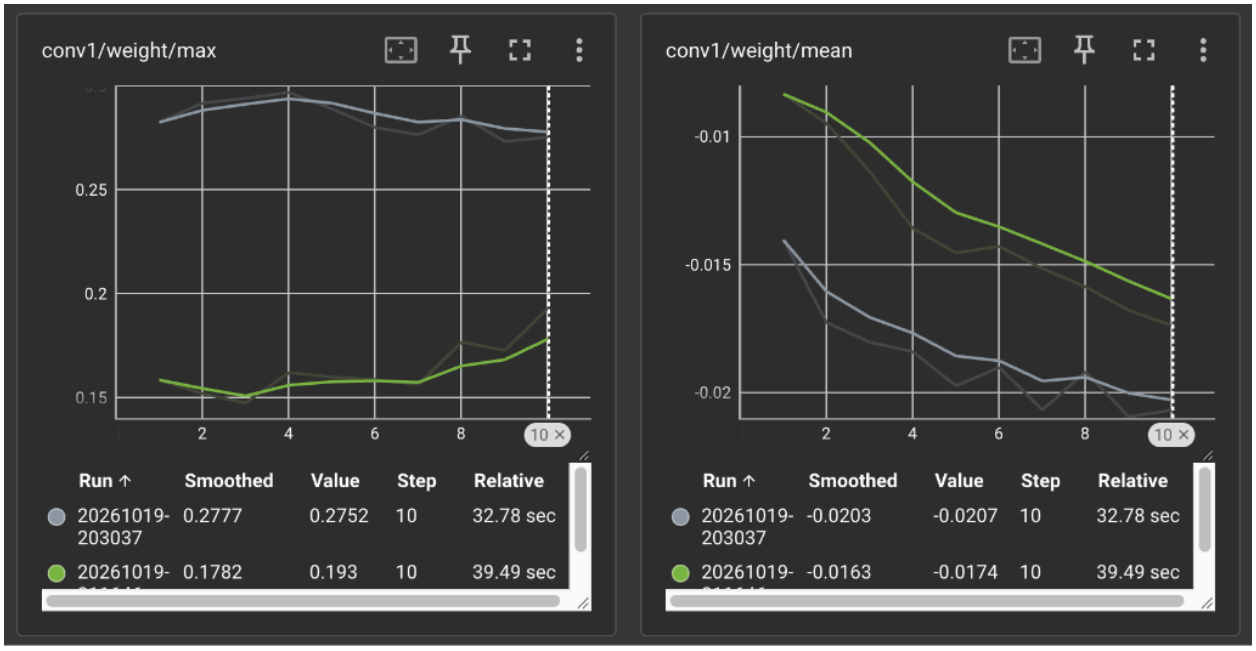
Initialization Comparison

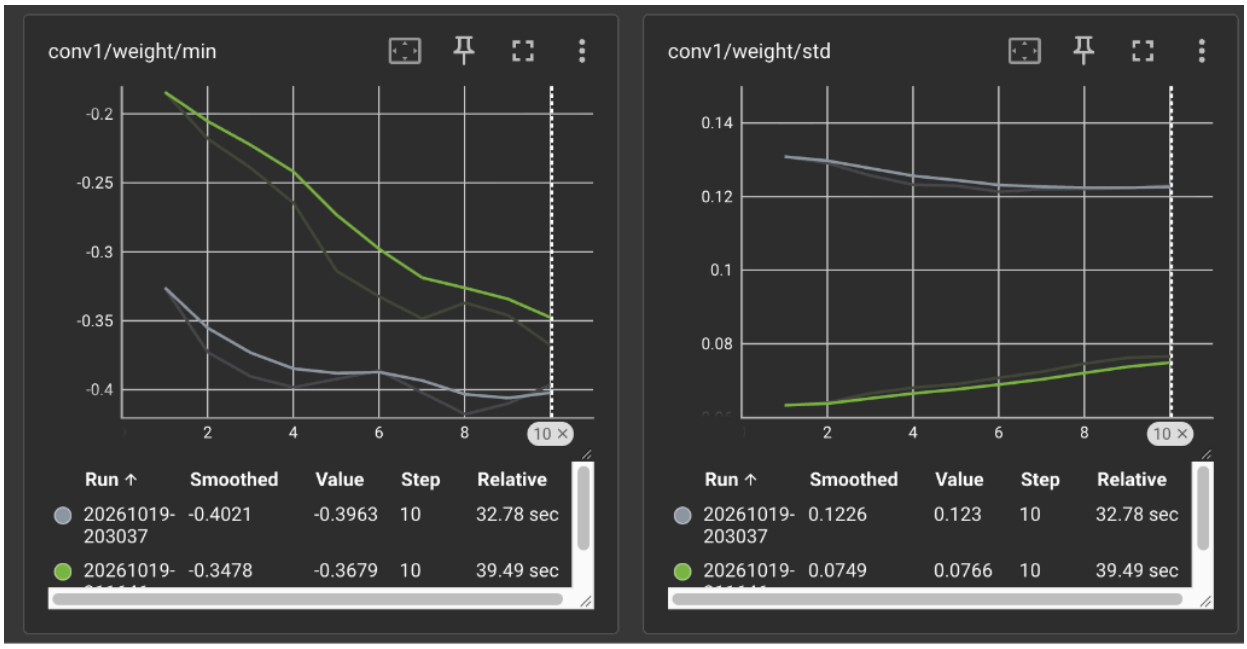







conv1/relu/max
3.5
3
2.5
2
2
4
6
8
10
Run
Smoothed
Value
Step
Relative
20261019-203037
3.0737
2.9073
10
32.7 sec
1.9686
1.9112
10
39.53 sec
conv1/relu/mean
0.15
0.1
0.05
2
4
6
8
10
Run
Smoothed
Value
Step
Relative
20261019-203037
0.0645
0.054
10
32.7 sec
0.0365
0.0304
10
39.53 sec
conv1/relu/min
0.5
0
-0.5
2
4
6
8
10
Run
Smoothed
Value
Step
Relative
20261019-203037
0
0
10
32.7 sec
0
0
10
39.53 sec
conv1/relu/std
0.35
0.3
0.25
0.2
0.15
2
4
6
8
10
Run
Smoothed
Value
Step
Relative
20261019-203037
0.1898
0.1748
10
32.7 sec
0.1214
0.1132
10
39.53 sec

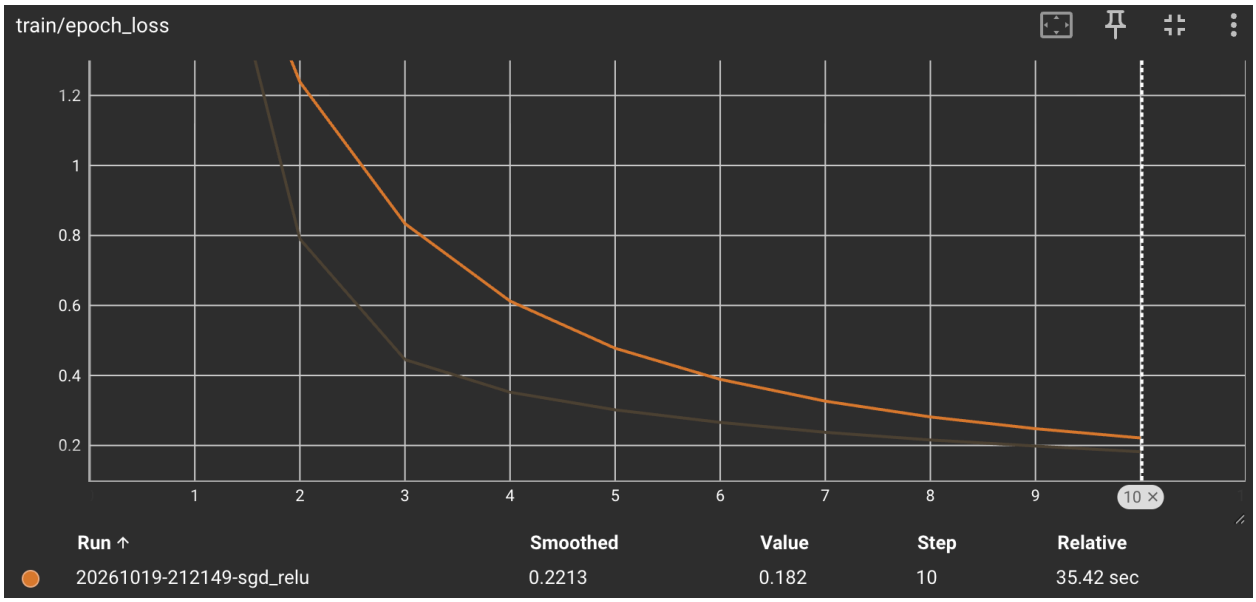

conv1/weight/max
0.25
0.2
0.15
2
4
6
8
10 ×
Run ↑ Smoothed Value Step Relative
20261019-203037 0.2777 0.2752 10 32.78 sec
20261019- 0.1782 0.193 10 39.49 sec
conv1/weight/mean
-0.01
-0.015
-0.02
2
4
6
8
10 ×
Run ↑ Smoothed Value Step Relative
20261019-203037 -0.0203 -0.0207 10 32.78 sec
20261019- -0.0163 -0.0174 10 39.49 sec

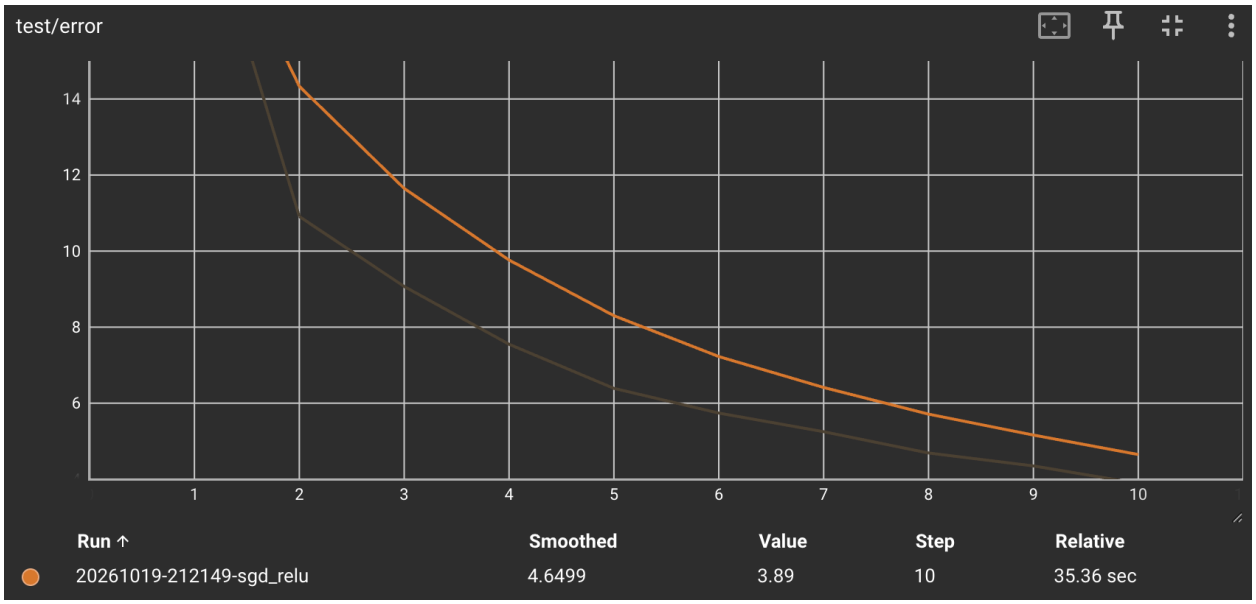

conv1/weight/min
-0.2
-0.25
-0.3
-0.35
-0.4
2
4
6
8
10 ×
Run ↑ Smoothed Value Step Relative
20261019-203037 -0.4021 -0.3963 10 32.78 sec
20261019- -0.3478 -0.3679 10 39.49 sec
conv1/weight/std
0.14
0.12
0.1
0.08
2
4
6
8
10 ×
Run ↑ Smoothed Value Step Relative
20261019-203037 0.1226 0.123 10 32.78 sec
20261019- 0.0749 0.0766 10 39.49 sec

Training Algorithms

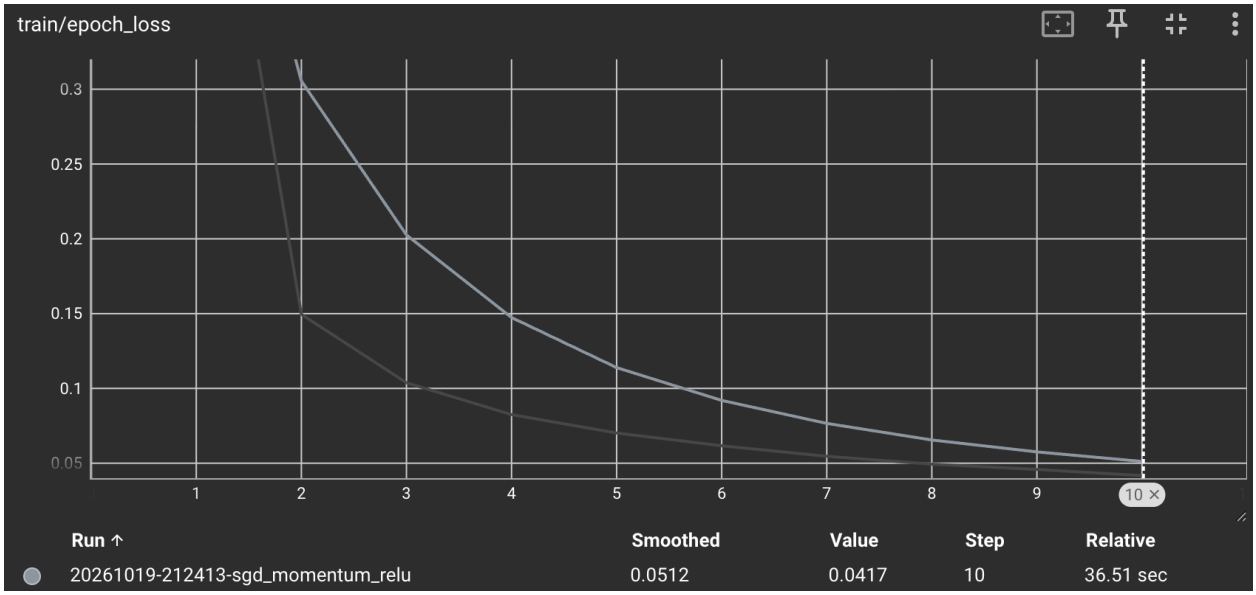
For the following runs, I reset the initialization technique back to the default layer initialization from PyTorch.

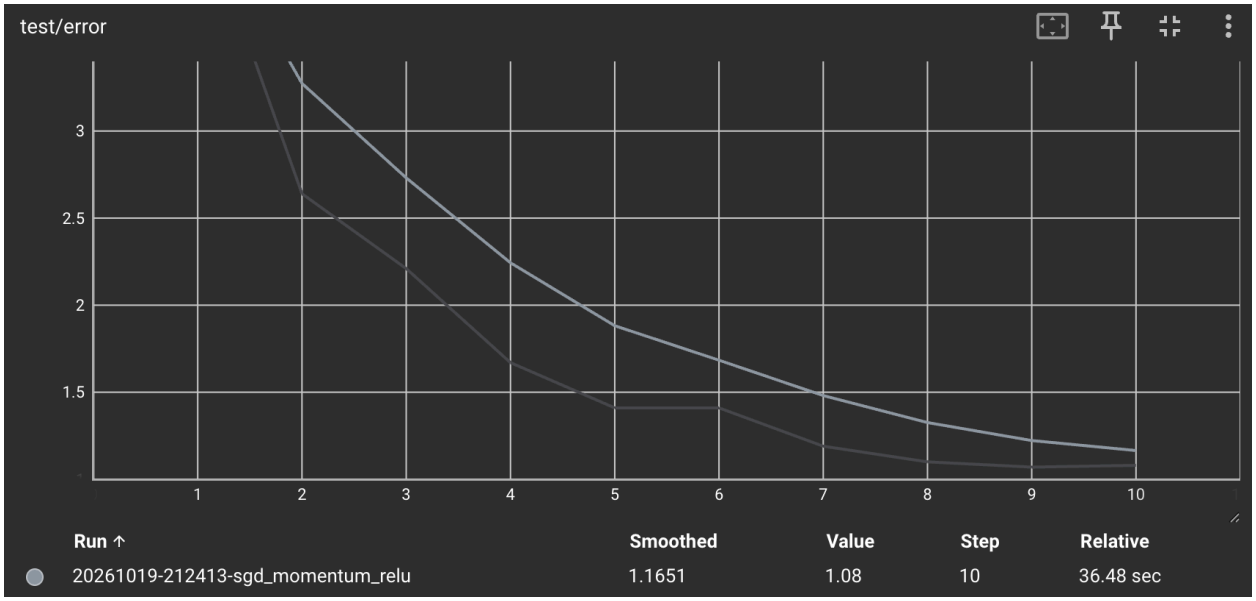
SGD



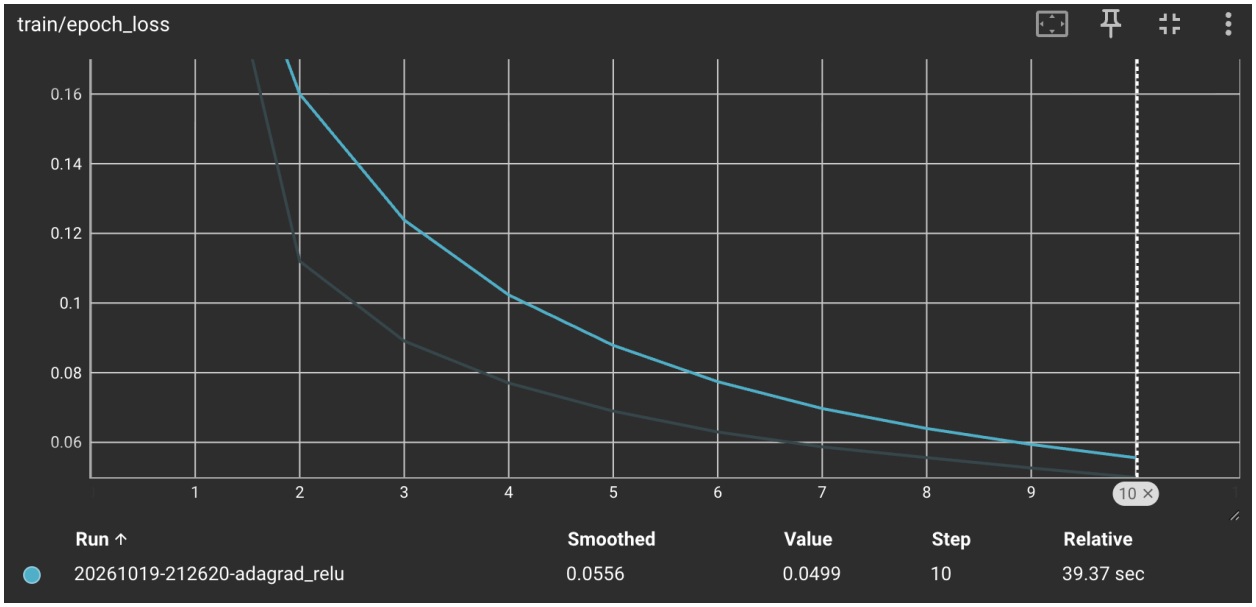


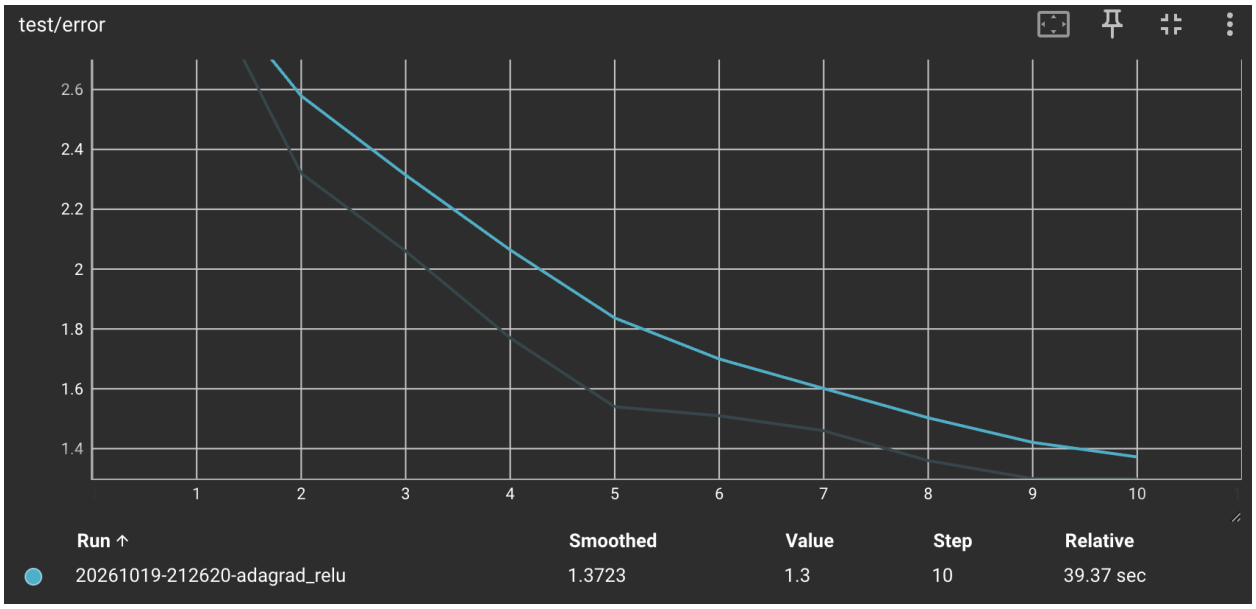
SGD with Momentum=0.9



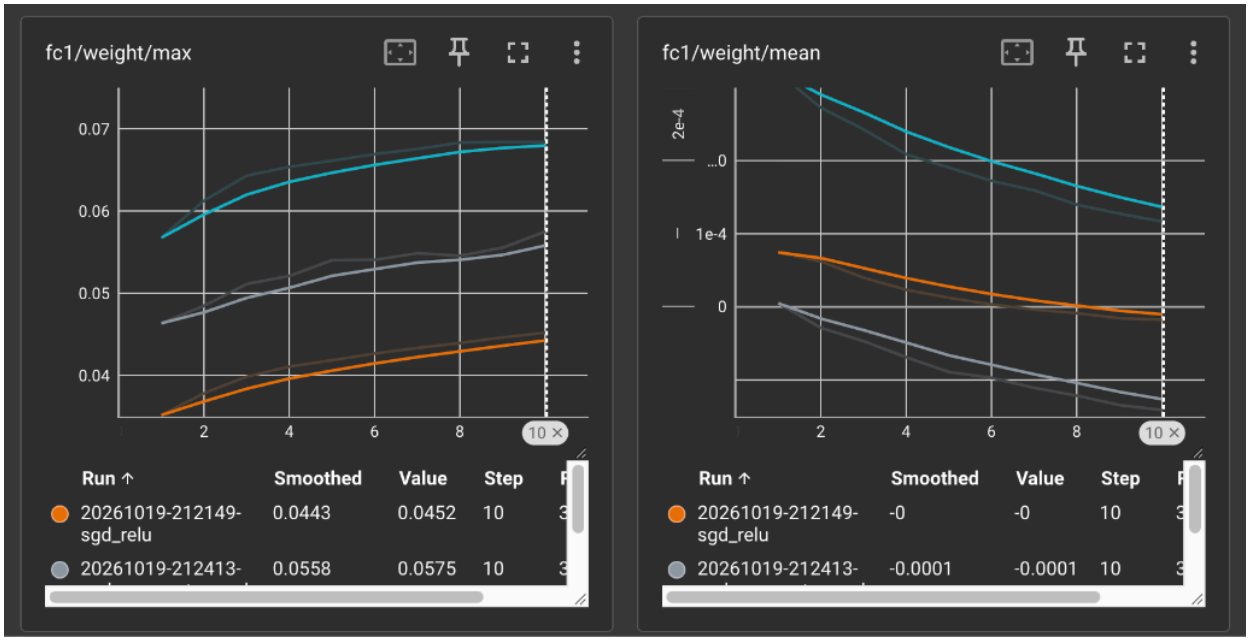


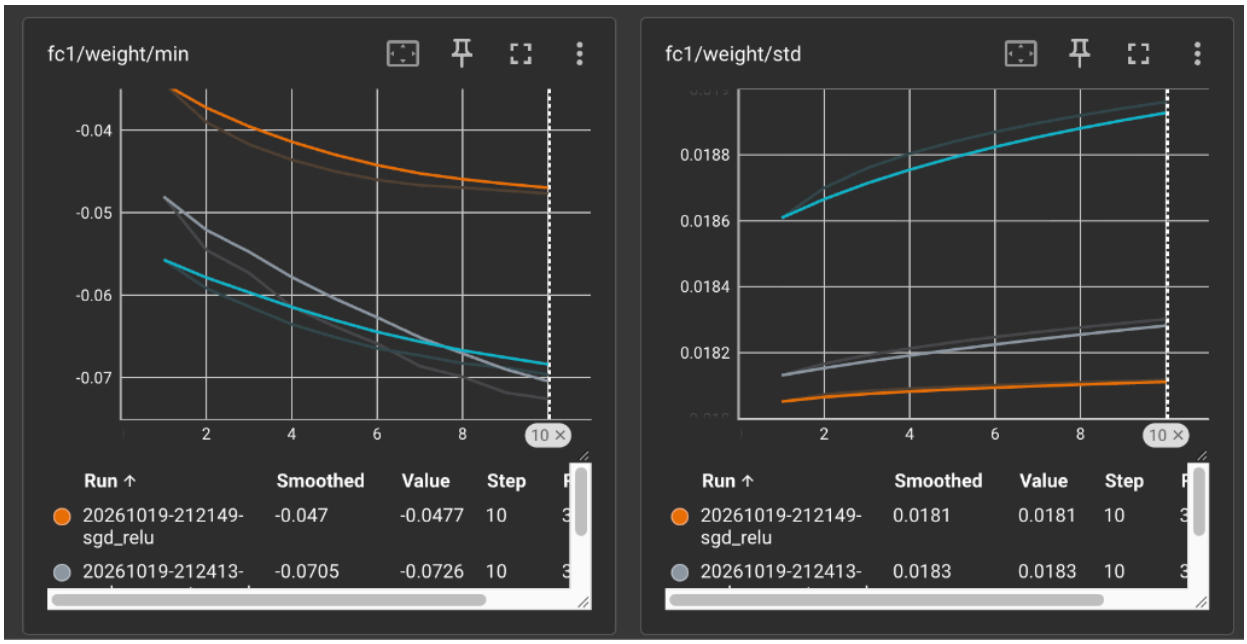
Adagrad


train/epoch_loss
0.16
0.14
0.12
0.1
0.08
0.06
1
2
3
4
5
6
7
8
9
10
Run
Smoothed
Value
Step
Relative
20261019-212620-adagrad_relu
0.0556
0.0499
10
39.37 sec


test/error
2.6
2.4
2.2
2
1.8
1.6
1.4
1
2
3
4
5
6
7
8
9
10
Run
Smoothed
Value
Step
Relative
20261019-212620-adagrad_relu
1.3723
1.3
10
39.37 sec

Training Algorithm Comparison





Observations

Across the 10 epoch activation runs, Leaky ReLU achieved the lowest final training loss (0.0123) and a 0.77% test error. Tanh and Sigmoid had similar final test errors (1.00% and 0.98%), although Sigmoid began with higher loss and error. Maxout's training loss rose in later epochs, and it finished at 1.29% test error; it also used more trainable parameters, so its results are not a direct like-for-like comparison. The activation histograms reflect their different output ranges: Tanh spans negative and positive values, Sigmoid stays between 0 and 1, and Leaky ReLU concentrates near zero with a positive tail; Maxout has a broader spread. Xavier initialization kept the first convolution's weight standard deviation below the baseline's and finished at 0.70% test error, compared with 0.74% for the original Adam run. At the shared learning rate of 0.001, SGD finished at 3.89% test error, compared with 1.08% for SGD with momentum and 1.30% for Adagrad.